Cohrint

Complete Developer & Admin Guidebook

Version 1.3 · April 2026

INTERNAL — NOT FOR PUBLIC DISTRIBUTION

Table of Contents

Press Ctrl+A then F9 to update page numbers.

Cohrint — Complete Developer & Admin Guidebook

Version 1.2 · April 2026 · INTERNAL

Table of Contents

1. Product Overview
2. Infrastructure Setup
3. Database Schema (Complete)
4. Authentication & Authorization
5. Complete API Reference
6. Dashboard Features
7. Semantic Cache — Deep Dive
8. Prompt Registry — Deep Dive
9. Agent Trace DAG — Deep Dive
10. Alert System
11. Rate Limiting
12. Client Libraries & SDKs
13. Integration Guides
14. Security & Governance
15. Email Infrastructure
16. CI/CD & Deployment
17. Test Infrastructure
18. Business Algorithms
19. Public Benchmark Dashboard
20. Pricing & Plans
21. Operational Runbook
22. Quick Reference Card

1. Product Overview

What Cohrint Does

An application integrates the Cohrint SDK (Python or JS). Every LLM API call the app makes is transparently intercepted; the SDK extracts cost, token, latency, and metadata from the response and POSTs it to `api.cohrint.com`. The Worker stores it in D1 (SQLite). The dashboard (`app.html`) polls or streams from the same API to render charts, KPI cards, and team breakdowns. The **Semantic Cache layer** intercepts prompts before they reach the LLM

and returns cached responses for semantically equivalent prompts, reducing cost. The **Prompt Registry** lets admins version and A/B-compare prompt templates with per-version cost attribution. The **Benchmark Dashboard** surfaces anonymized industry percentile rankings (k-anonymity floor: 5 orgs). Admins set budgets, alerts fire via Slack when thresholds are crossed, and team members each get scoped keys so they see only their team's data.

Technology Stack

| Layer | Technology | Why |

|---|---|---|

| API Worker | Cloudflare Workers + Hono | Edge-globally-distributed, zero cold starts, TypeScript |

| Database | Cloudflare D1 (SQLite) | Serverless SQLite, no infra, free tier sufficient for MVP |

| Cache/Pub-Sub | Cloudflare KV | Rate limiting counters, SSE broadcast, alert throttle, session tokens |

| Vector Store | Cloudflare Vectorize | BGE-small-en-v1.5 (384-dim) embeddings for semantic cache |

| AI Inference | Cloudflare Workers AI | Embedding generation for semantic cache (`@cf/baai/bge-small-en-v1.5`) |

| Frontend | Cloudflare Pages | Static hosting, global CDN, auto-deploys from `main` |

| Email | Resend API | Transactional email, 3k/month free, custom domain |

| SDK (Python) | `cohrint` on PyPI | OpenAI + Anthropic proxy wrappers |

| SDK (JS) | `cohrint` on npm | OpenAI + Anthropic proxy wrappers, streaming support |

| MCP Server | `cohrint-mcp/` | VS Code, Cursor, Windsurf integration |

| CI/CD | GitHub Actions | Deploy on push to `main`, test on every branch |

System Architecture

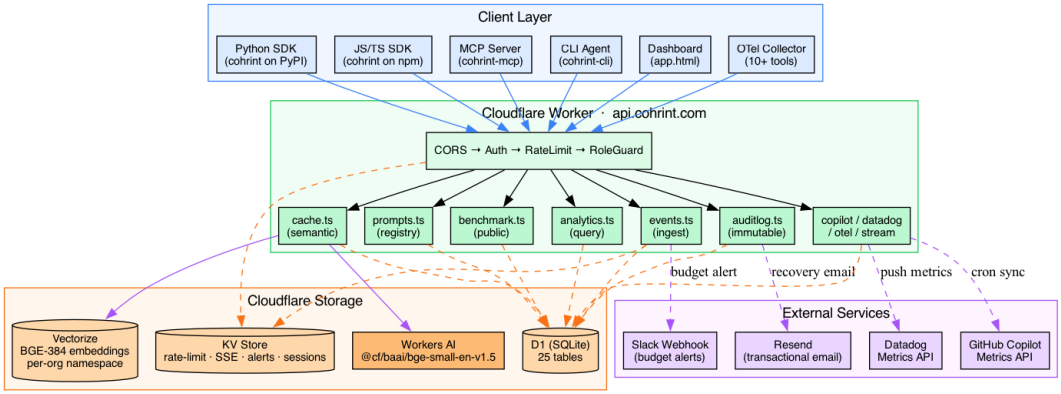

Figure 1 — Cohrint System Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                              CLIENT LAYER                            │
│                                                                      │
│  Python SDK    JS SDK    MCP Server    CLI      Browser Dashboard    │
│  (cohrint)   (cohrint)  (cohrint-mcp) (cohrint-cli)  (app.html)      │
└──────────────────────────────────────────────────────────────────────┘
                            │  HTTPS  Bearer crt_... / Cookie
                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│                Cloudflare Worker  —  api.cohrint.com                 │
│                                                                      │
│  corsMiddleware → authMiddleware → rateLimiter → roleGuard           │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐               │
│  │  events.ts   │  │ analytics.ts │  │  cache.ts    │               │
│  │  (ingest)    │  │  (query)     │  │  (semantic)  │               │
│  └──────────────┘  └──────────────┘  └──────────────┘               │
│        │                │                  │                         │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐               │
│  │  prompts.ts  │  │benchmark.ts  │  │  auditlog.ts │               │
│  │  (registry)  │  │  (public)    │  │  (immutable) │               │
│  └──────────────┘  └──────────────┘  └──────────────┘               │
└──────────────────────────────────────────────────────────────────────┘
          │                                    │
    ┌──────────────┐                     ┌──────────────┐
    │  Cloudflare D1 │                   │  Cloudflare KV │
    │  (SQLite)      │                   │  (ephemeral)   │
    │                │                   │                │
    │  events        │                   │  rl:{org}:{min}│
    │  orgs          │                   │  stream:{org}  │
    │  org_members   │                   │  slack:{org}   │
    │  sessions      │                   │  alert:{org}   │
    │  prompts       │                   │  recover:{tok} │
    │  prompt_vers.  │                   │  copilot:token │
    │  semantic_cache│                   │  sse:{org}     │
    │  audit_events  │                   └──────────────┘
    │  benchmark_*   │
    │  cross_plat.   │          ┌──────────────────────┐
    │  otel_events   │          │  Cloudflare Vectorize  │
    └──────────────┘            │  (semantic embeddings) │
                                │  namespace: org_id     │
                                │  model: bge-small-en   │
                                └──────────────────────┘
                                          ▲
                                ┌──────────────┐
                                │ Workers AI   │
                                │ @cf/baai/bge-  │
                                │ small-en-v1.5  │
                                └──────────────┘
```

Request Lifecycle Sequence Diagram

```
Client                  Worker                    D1              KV
  |                       |                        |               |
  |──POST /v1/events──────►|                        |               |
  |                       |                        |               |
  |                       |──corsMiddleware()──────►|               |
  |                       |  ← 204 if OPTIONS       |               |
  |                       |                        |               |
  |                       |──authMiddleware()──────────────────────►
  |                       |  1. parse Cookie header |               |
  |                       |  2. GET sessions WHERE  |               |
  |                       |     token=? expires>now►               |
  |                       |  OR SHA-256(Bearer key) |               |
  |                       |     GET orgs/members ──►               |
  |                       |  set: orgId,role,scope  |               |
  |                       |                        |               |
  |                       |──checkRateLimit()──────────────────────►
  |                       |  GET rl:{org}:{minute}  |      ┌──────────┐
  |                       |◄─ count (or 0)          |      │  count   │
  |                       |  PUT rl:{org}:{min} N+1 |      └──────────┘
  |                       |  ← 429 if >= RPM_LIMIT  |               |
  |                       |                        |               |
  |                       |──roleGuard()            |               |
  |                       |  viewer? → 403          |               |
  |                       |                        |               |
  |                       |──checkFreeTierLimit()──►               |
  |                       |  COUNT events this month|               |
  |                       |  ← 429 if free + >50k   |               |
  |                       |                        |               |
  |                       |──INSERT OR IGNORE events►              |
  |                       |                        |               |
  |                       |──broadcastEvent()──────────────────────►
  |                       |  PUT stream:{org}:latest|      ┌──────────┐
  |                       |  TTL=60s                |      │ payload  │
  |                       |                        |      └──────────┘
  |                       |──logAudit()────────────►               |
  |                       |  INSERT audit_events    |               |
  |◄──201 {ok,id}─────────|                        |               |
```

2. Infrastructure Setup

wrangler.toml Bindings

```
name = "cohrint-api"
routes = [{ pattern = "api.cohrint.com/*", zone_name = "cohrint.com" }]

[[d1_databases]]
binding = "DB"
# database_id in wrangler.toml (gitignored)
# retrieve: wrangler d1 list

[[kv_namespaces]]
binding = "KV"
# id in wrangler.toml (gitignored)
# retrieve: wrangler kv namespace list

[ai]
binding = "AI"          # Workers AI — used by semantic cache

[[vectorize]]
binding = "VECTORIZE"   # Vectorize index — used by semantic cache
# index_name in wrangler.toml (gitignored)
# retrieve: wrangler vectorize list

[vars]
ENVIRONMENT = "production"
ALLOWED_ORIGINS = "https://cohrint.com,https://www.cohrint.com,https://cohrint.pages.dev"
RATE_LIMIT_RPM = "1000"

# Secrets (set via: wrangler secret put <NAME>)
# RESEND_API_KEY           — email sending
# SUPERADMIN_SECRET        — /v1/superadmin/* gate
# VANTAGE_CI_SECRET        — bypass signup rate limiting in CI
# TOKEN_ENCRYPTION_SECRET  — AES-256-GCM key for Copilot PAT + Datadog key encryption
# DEMO_API_KEY             — viewer-scoped key used by POST /v1/auth/demo
```

> *database_id, KV id, and Vectorize index_name are in wrangler.toml which is gitignored. Retrieve via wrangler d1 list / wrangler kv namespace list / wrangler vectorize list.*

D1 Migrations

```
# Apply all pending migrations to production
npx wrangler d1 migrations apply vantage-events --remote

# Apply a specific migration file
npx wrangler d1 execute vantage-events --file
./cohrint-worker/migrations/0017_benchmark_metric_name.sql

# Query production
npx wrangler d1 execute vantage-events --command "SELECT COUNT(*) FROM events"

# Backup
npx wrangler d1 export vantage-events --output backup-$(date +%Y%m%d).sql
```

Vectorize Index Creation

```
# Create the semantic cache index
wrangler vectorize create cohrint-semantic-cache \
  --dimensions=384 \
  --metric=cosine

# List indexes
wrangler vectorize list
```

Workers AI Binding

Add to `wrangler.toml`:

```
[ai]
binding = "AI"
```

Then in worker code: `env.AI.run('@cf/baai/bge-small-en-v1.5', { text: [prompt] })`

Secrets List

| Secret | Purpose | Required |

|---|---|---|

| `RESEND_API_KEY` | Transactional email (recovery + invites) | Optional (silent no-op if missing) |

| `TOKEN_ENCRYPTION_SECRET` | AES-256-GCM key for Copilot PAT + Datadog API key | **Throws on startup if missing** |

| `SUPERADMIN_SECRET` | Gates `/v1/superadmin/*` platform admin routes | Required for superadmin ops |

| `VANTAGE_CI_SECRET` | Bypasses signup rate limiting in CI test suites | Required for CI |

| `DEMO_API_KEY` | Viewer-scoped key for `POST /v1/auth/demo` | Required for demo mode |

Deploy Commands

```
# Deploy worker
cd cohrint-worker && npm run deploy   # runs: wrangler deploy


# Deploy frontend
npx wrangler pages deploy ./cohrint-frontend --project-name=cohrint --branch=main


# TypeScript check (no emit)
cd cohrint-worker && npm run typecheck   # runs: tsc --noEmit
```

3. Database Schema (Complete)

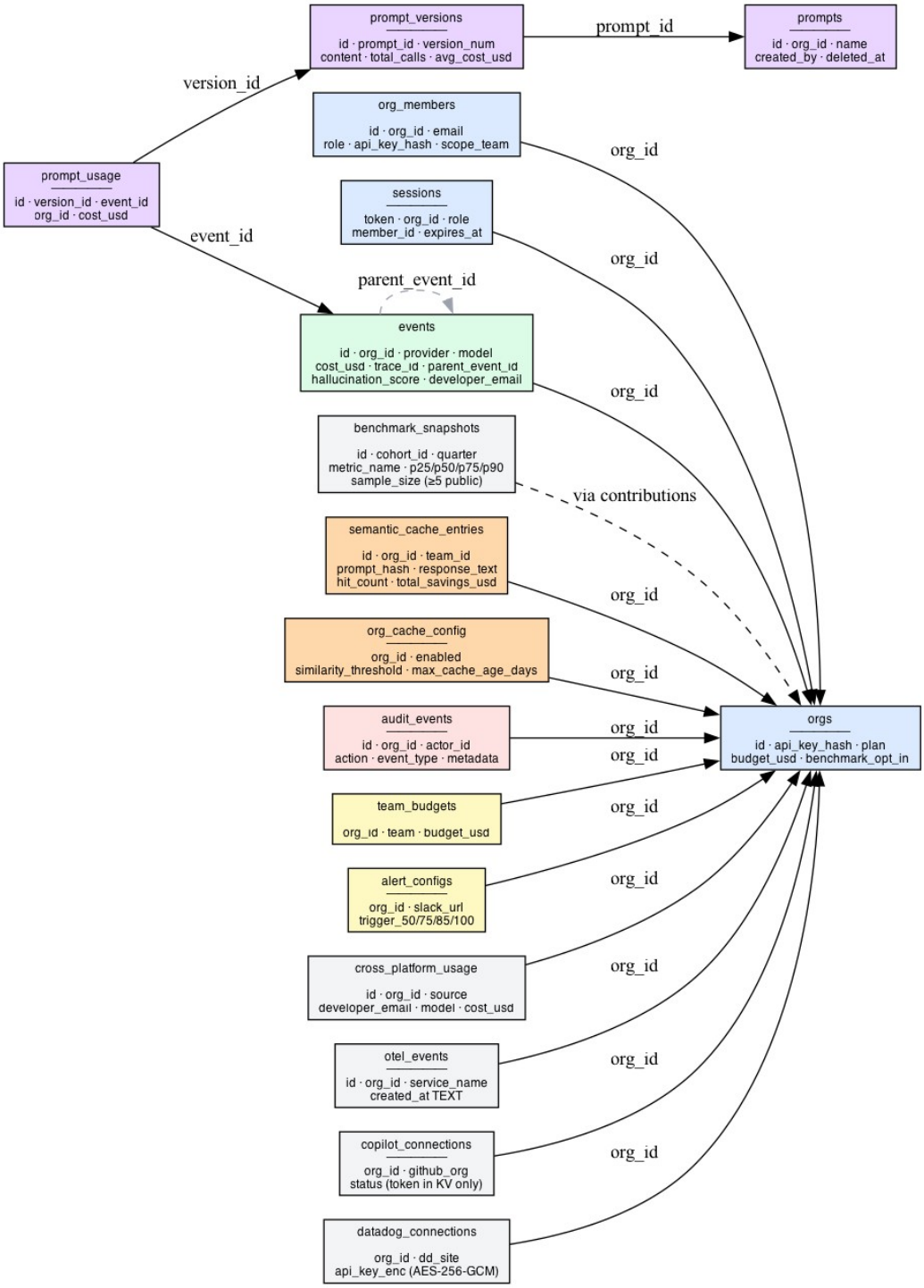

Figure 6 — Database ERD

CRITICAL: Date Column Type Divergence

`created_at` is NOT consistent across tables. Binding the wrong type causes SQLite to silently coerce to `0` — filters then match every row with no runtime error.

```
┌──────────────────────────────────────────────────────────────────────┐
│  INTEGER (unix epoch seconds)     TEXT (YYYY-MM-DD HH:MM:SS)         │
│  bind: Math.floor(Date.now()/1000)  bind: "2026-04-17 00:00:00"    |
│  default: unixepoch()             default: datetime('now')           |
|                                                                       |
│  events                           cross_platform_usage               |
│  orgs                             otel_events                        |
│  org_members                      benchmark_snapshots                |
│  alert_configs                    copilot_connections                |
│  team_budgets                     datadog_connections                |
│  sessions                         prompts                            |
│  alert_log                        prompt_versions                    |
│  platform_*                       prompt_usage                       |
│  audit_events                     semantic_cache_entries             |
|                                   org_cache_config                   |
└──────────────────────────────────────────────────────────────────────┘
```

Helper functions — use these, never raw date math:

- **INTEGER tables:** `sinceUnix(days)`, `todayUnix()` in `analytics.ts`
- **TEXT tables:** `sqliteDateSince(days)`, `sqliteTodayStart()`, `sqliteMonthStart()` in `crossplatform.ts`
- For INTEGER hour extraction: `strftime('%H', created_at, 'unixepoch')`
- Never use ISO 8601 `T/Z` format (`2026-03-24T00:00:00Z`) in TEXT WHERE clauses — `T` sorts differently than space.

Database ERD

```
orgs ──────────────────────────────────────────────────────────────────┐
 │ id (PK)                                                              │
 │ api_key_hash, plan, budget_usd, benchmark_opt_in                     │
 │                                                                      │
 ├──── org_members                    ┌── sessions                      │
 │      id (PK)                       │    token (PK)                   │
 │      org_id (FK→orgs)              │    org_id (FK→orgs)             │
 │      role, scope_team, email       │    role, member_id, expires_at  │
 │      api_key_hash                  └─────────────────────────────────┘
 │                                                                      │
 ├──── events                                                           │
 │      (id, org_id) PK                                                 │
 │      provider, model, cost_usd                                       │
 │      trace_id, parent_event_id ◄── DAG edges                         │
 │      hallucination_score, ...                                        │
 │                                                                      │
 ├──── team_budgets                   ┌── alert_configs                 │
 │      (org_id, team) PK             │    org_id (PK)                  │
 │      budget_usd                    │    slack_url, trigger_*         │
 │                                    └─────────────────────────────────┘
 ├──── cross_platform_usage                                             │
 │      id (PK), org_id, source                                         │
 │      developer_email, model, cost_usd                                │
 │      created_at TEXT                                                 │
 │                                                                      │
 ├──── otel_events                                                      │
 │      id (PK), org_id, service_name                                   │
 │      created_at TEXT                                                 │
 │                                                                      │
 ├──── prompts                        ┌── prompt_versions               │
 │      id (PK), org_id               │    id (PK)                      │
 │      name UNIQUE, deleted_at       │    prompt_id (FK→prompts)       │
 │                                    │    version_num, content         │
 │                                    │    total_calls, avg_cost_usd    │
 │                                    └──── prompt_usage                │
 │                                           version_id (FK)            │
 │                                           event_id, cost_usd         │
 │                                                                      │
 ├──── semantic_cache_entries         ┌── org_cache_config              │
 │      id (PK), org_id, team_id      │    org_id (PK)                  │
 │      prompt_hash, model            │    enabled, similarity_thresh   │
 │      response_text, cost_usd       │    min_prompt_length            │
 │      hit_count, total_savings_usd  │    max_cache_age_days           │
 │      vectorize_id                  └─────────────────────────────────┘
 │      created_at TEXT                                                 │
 │                                                                      │
 ├──── audit_events                                                     │
 │      id (PK), org_id, actor_id                                       │
 │      action, event_type, metadata                                    │
 │                                                                      │
 ├──── benchmark_cohorts ──────────── benchmark_snapshots               │
 │      id (PK)                        cohort_id (FK)                   │
 │      size_band, industry            quarter, metric_name             │
 │                                     p25/p50/p75/p90                  │
 │                                     sample_size (≥5 to be public)    │
 │                                  ── benchmark_contributions          │
 │                                     org_id, snapshot_id              │
 │                                                                      │
 ├──── copilot_connections            ┌── datadog_connections           │
 │      org_id (UNIQUE)               │    org_id (UNIQUE)              │
 │      github_org, status            │    dd_site                      │
 │      (token in KV only)            │    api_key_enc (AES-256-GCM)    │
 │                                    └─────────────────────────────────┘
 └──── budget_policies                                                  │
        id (PK), org_id                                                 │
        scope, scope_value, budget_usd                                  │
```

```
        enforcement ('alert'|'block')                                    |
```

3.1 `orgs`

```
CREATE TABLE orgs (
  id            TEXT PRIMARY KEY,     -- slug: "mycompany-a3f2"
  api_key_hash  TEXT NOT NULL,        -- SHA-256 of raw key (never store raw)
  api_key_hint  TEXT,                 -- "crt_mycompa..." (first 12 chars + ...)
  name          TEXT,
  email         TEXT UNIQUE,
  plan          TEXT DEFAULT 'free',  -- 'free' | 'team' | 'enterprise'
  budget_usd    REAL DEFAULT 0,       -- monthly limit; 0 = not set
  benchmark_opt_in INTEGER NOT NULL DEFAULT 0,
  account_type  TEXT DEFAULT 'organization',
  industry      TEXT,                 -- 'tech'|'finance'|'healthcare'|'other'
  created_at    INTEGER               -- unix timestamp
);
```

3.2 `org_members`

```
CREATE TABLE org_members (
  id            TEXT PRIMARY KEY,     -- 8-char hex random
  org_id        TEXT NOT NULL,        -- FK → orgs.id
  email         TEXT NOT NULL,
  name          TEXT,
  role          TEXT NOT NULL,        -- 'superadmin'|'ceo'|'admin'|'member'|'viewer'
  api_key_hash  TEXT NOT NULL,
  api_key_hint  TEXT,
  scope_team    TEXT,                 -- NULL = see all; non-null = filtered to team
  team_id       TEXT,                 -- internal team assignment (for cache namespace)
  created_at    INTEGER
);
```

3.3 `sessions`

```
CREATE TABLE sessions (
  token       TEXT PRIMARY KEY,       -- 64-char hex (32 random bytes)
  org_id      TEXT NOT NULL,
  role        TEXT NOT NULL,
  member_id   TEXT,                   -- NULL for owner sessions
  expires_at  INTEGER NOT NULL        -- unix timestamp, 30-day TTL
);
```

3.4 `events` — Core data table

```sql
CREATE TABLE events (
  id                  TEXT NOT NULL,
  org_id              TEXT NOT NULL,
  provider            TEXT,
  model               TEXT,
  prompt_tokens       INTEGER DEFAULT 0,
  completion_tokens   INTEGER DEFAULT 0,
  cache_tokens        INTEGER DEFAULT 0,
  total_tokens        INTEGER DEFAULT 0,
  cost_usd            REAL DEFAULT 0,
  latency_ms          INTEGER DEFAULT 0,
  team                TEXT,
  project             TEXT,
  user_id             TEXT,
  feature             TEXT,
  endpoint            TEXT,
  environment         TEXT DEFAULT 'production',
  is_streaming        INTEGER DEFAULT 0,
  stream_chunks       INTEGER DEFAULT 0,
  trace_id            TEXT,              -- agent trace grouping
  parent_event_id     TEXT,              -- parent span → DAG edge
  agent_name          TEXT,
  span_depth          INTEGER DEFAULT 0,
  tags                TEXT,              -- JSON object
  sdk_language        TEXT,
  sdk_version         TEXT,
  hallucination_score REAL,
  faithfulness_score  REAL,
  relevancy_score     REAL,
  consistency_score   REAL,
  toxicity_score      REAL,
  efficiency_score    REAL,
  created_at          INTEGER NOT NULL,  -- unix timestamp
  prompt_hash         TEXT,
  cache_hit           INTEGER NOT NULL DEFAULT 0,
  developer_email     TEXT,
  PRIMARY KEY (id, org_id)               -- dedup per org (INSERT OR IGNORE)
);
```

3.5 `semantic_cache_entries`

```sql
CREATE TABLE semantic_cache_entries (
  id                TEXT PRIMARY KEY,   -- UUID = also vectorize_id
  org_id            TEXT NOT NULL,
  team_id           TEXT,               -- optional team namespace
  prompt_hash       TEXT NOT NULL,      -- SHA-256 or first 16 chars for dedup
  prompt_text       TEXT NOT NULL,      -- full prompt text
  model             TEXT NOT NULL,
  response_text     TEXT NOT NULL,      -- cached LLM response
  prompt_tokens     INTEGER DEFAULT 0,
  completion_tokens INTEGER DEFAULT 0,
  cost_usd          REAL DEFAULT 0,     -- cost of original call (basis for savings)
  vectorize_id      TEXT,               -- Vectorize vector ID (matches id)
  hit_count         INTEGER DEFAULT 0,
  total_savings_usd REAL DEFAULT 0,
  last_hit_at       TEXT,               -- YYYY-MM-DD HH:MM:SS
  created_at        TEXT NOT NULL       -- YYYY-MM-DD HH:MM:SS
);
```

3.6 `org_cache_config`

```
CREATE TABLE org_cache_config (
  org_id               TEXT PRIMARY KEY,
  enabled              INTEGER DEFAULT 1,
  similarity_threshold REAL DEFAULT 0.92,   -- cosine similarity floor
  min_prompt_length    INTEGER DEFAULT 10,  -- chars; short prompts not cached
  max_cache_age_days   INTEGER DEFAULT 30,
  updated_at           TEXT                 -- YYYY-MM-DD HH:MM:SS
);
```

3.7 `prompts`, `prompt_versions`, `prompt_usage`

```
CREATE TABLE prompts (
  id          TEXT PRIMARY KEY,
  org_id      TEXT NOT NULL,
  name        TEXT NOT NULL,
  description TEXT,
  created_by  TEXT NOT NULL,      -- member email or 'owner'
  deleted_at  TEXT,               -- soft delete
  created_at  TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at  TEXT NOT NULL DEFAULT (datetime('now')),
  UNIQUE (org_id, name)           -- name unique per org
);

CREATE TABLE prompt_versions (
  id                  TEXT PRIMARY KEY,
  prompt_id           TEXT NOT NULL,    -- FK → prompts.id
  version_num         INTEGER NOT NULL, -- auto-incremented per prompt
  content             TEXT NOT NULL,    -- full prompt template text
  model               TEXT,             -- target model (nullable)
  notes               TEXT,
  created_by          TEXT NOT NULL,
  total_calls         INTEGER DEFAULT 0,
  total_cost_usd      REAL DEFAULT 0,
  avg_cost_usd        REAL DEFAULT 0,
  avg_prompt_tokens   INTEGER DEFAULT 0,
  avg_completion_tokens INTEGER DEFAULT 0,
  created_at          TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE prompt_usage (
  id                TEXT PRIMARY KEY,
  version_id        TEXT NOT NULL,     -- FK → prompt_versions.id
  event_id          TEXT NOT NULL,     -- FK → events.id
  org_id            TEXT NOT NULL,
  cost_usd          REAL DEFAULT 0,
  prompt_tokens     INTEGER DEFAULT 0,
  completion_tokens INTEGER DEFAULT 0,
  created_at        TEXT NOT NULL DEFAULT (datetime('now')),
  UNIQUE (version_id, event_id)        -- idempotent (INSERT OR IGNORE)
);
```

3.8 `cross_platform_usage`

```
CREATE TABLE cross_platform_usage (
  id              TEXT PRIMARY KEY,
  org_id          TEXT NOT NULL,
  source          TEXT NOT NULL,       -- 'otel'|'copilot'|'datadog'|'sdk'
  provider        TEXT,
  developer_id    TEXT,
  developer_email TEXT,
  model           TEXT,
  input_tokens    INTEGER DEFAULT 0,
  output_tokens   INTEGER DEFAULT 0,
  cache_tokens    INTEGER DEFAULT 0,
  cost_usd        REAL DEFAULT 0,
  commits         INTEGER DEFAULT 0,
  pull_requests   INTEGER DEFAULT 0,
  lines_added     INTEGER DEFAULT 0,
  active_time_s   INTEGER DEFAULT 0,
  period_date     TEXT,                -- YYYY-MM-DD
  created_at      TEXT                 -- YYYY-MM-DD HH:MM:SS
);
```

3.9 `benchmark_snapshots`

```
CREATE TABLE benchmark_snapshots (
  id           INTEGER PRIMARY KEY,
  cohort_id    INTEGER NOT NULL,   -- FK → benchmark_cohorts.id
  quarter      TEXT NOT NULL,      -- '2026-Q2'
  metric_name  TEXT NOT NULL,      -- 'cost_per_token'|'cost_per_dev_month'|'cache_hit_rate'
  model        TEXT,               -- NULL = all models
  sample_size  INTEGER DEFAULT 0,
  p25          REAL,
  p50          REAL,
  p75          REAL,
  p90          REAL,
  updated_at   TEXT,
  UNIQUE (cohort_id, quarter, metric_name, model)
);
```

3.10 `audit_events`

```sql
-- created_at is TEXT datetime('now') — NOT INTEGER
-- Paginate: WHERE created_at < ? using 'YYYY-MM-DD HH:MM:SS' strings
CREATE TABLE audit_events (
  id           TEXT PRIMARY KEY,
  org_id       TEXT NOT NULL,
  actor_id     TEXT,
  actor_email  TEXT,
  action       TEXT NOT NULL,
  target_id    TEXT,
  target_type  TEXT,
  metadata     TEXT,              -- JSON
  event_type   TEXT,              -- 'auth'|'admin'|'billing'
  created_at   TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_audit_events_org_created ON audit_events(org_id, created_at DESC);
```

3.11 Migration Registry

| File | Description |

|---|---|

| `0001_cross_platform_usage.sql` | `cross_platform_usage`, `otel_events`, `provider_connections`, `budget_policies` |

| `0003_audit_events.sql` | `audit_events` table |

| `0004_audit_event_type.sql` | `ALTER TABLE audit_events ADD COLUMN event_type TEXT` |

| `0005_otel_traces.sql` | `otel_traces` table |

| `0006_otel_sessions.sql` | `otel_sessions` table |

| `0007_prompt_hash.sql` | `ALTER TABLE events ADD COLUMN prompt_hash TEXT` + `cache_hit` |

| `0008_benchmark_opt_in.sql` | `ALTER TABLE orgs ADD COLUMN benchmark_opt_in INTEGER NOT NULL DEFAULT 0` |

| `0009_copilot_connections.sql` | `copilot_connections` table |

| `0010_platform_tables.sql` | `platform_pageviews`, `platform_sessions`, `benchmark_*` tables |

| `0011_benchmark_snapshots.sql` | Additional indexes on `benchmark_snapshots` |

| `0012_datadog_connections.sql` | `datadog_connections` table |

| `0013_schema_fixes.sql` | Additional indexes and fixes |

| `0014_drop_copilot_kv_key.sql` | Removes `kv_key` from `copilot_connections` (token KV-only) |

| 0015_semantic_cache.sql | semantic_cache_entries, org_cache_config tables |

| 0016_prompt_registry.sql | prompts, prompt_versions, prompt_usage tables |

| 0017_benchmark_metric_name.sql | ALTER TABLE benchmark_snapshots ADD COLUMN metric_name TEXT + p25/p50/p75/p90 |

All 25 D1 Tables

orgs, org_members, sessions, events, team_budgets, alert_configs,

cross_platform_usage, otel_events, otel_traces, otel_sessions,

provider_connections, budget_policies, audit_events,

copilot_connections, datadog_connections,

benchmark_cohorts, benchmark_snapshots, benchmark_contributions,

platform_pageviews, platform_sessions,

semantic_cache_entries, org_cache_config,

prompts, prompt_versions, prompt_usage

4. Authentication & Authorization

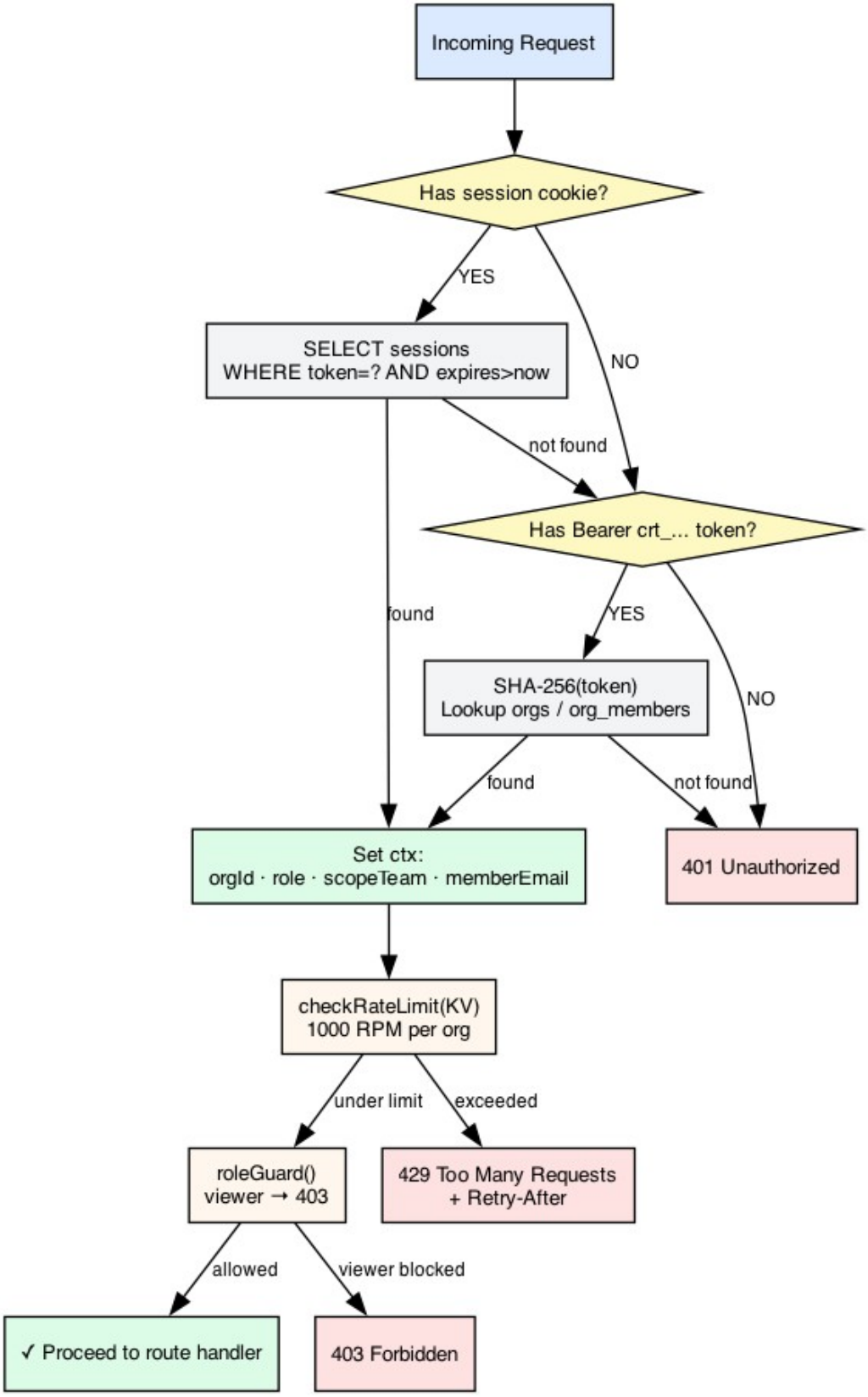
Incoming Request
Has session cookie?
YES
NO
SELECT sessions
WHERE token=? AND expires>now
not found
Has Bearer crt_... token?
found
YES
NO
SHA-256(token)
Lookup orgs / org_members
found
not found
Set ctx:
orgId · role · scopeTeam · memberEmail
401 Unauthorized
checkRateLimit(KV)
1000 RPM per org
under limit
exceeded
roleGuard()
viewer → 403
429 Too Many Requests
+ Retry-After
allowed
viewer blocked
✓ Proceed to route handler
403 Forbidden

Figure 2 — Authentication & Authorization Flow

API Key Format

```
crt_{orgId}_{16-hex-random}
 ^     ^          ^
 |     |          └── 16 bytes crypto.getRandomValues() = 128 bits entropy
 |     └──────────── org slug (for fast routing — no DB lookup to extract orgId)
 └────────────────── Cohrint namespace prefix
```

Only `SHA-256(rawKey)` is stored. Raw key shown exactly once. Forgotten = must rotate.

Auth Middleware Flow

```
Request arrives at authMiddleware
           │
           ▼
  Has Cookie: cohrint_session / __Host-cohrint_session / vantage_session ?
           │
      ┌────┴────┐
    YES         NO
     │           │
     ▼           ▼
  SELECT sessions  Has Authorization: Bearer crt_... / vnt_... ?
  WHERE token=?         │
  AND expires_at >      │
  unixepoch()     ┌─────┴─────┐
     │          YES          NO
     │           │            │
  Found?         ▼            ▼
  ┌──┴──┐   Extract orgId   logAuditRaw(auth.failed)
 YES    NO  from key (parts[1]) → 401
  │     │       │
  │   fall    SHA-256(key)
  │   through     │
  ▼             SELECT orgs WHERE api_key_hash=?
set ctx:          │
orgId,role,   Found (owner)?
scopeTeam,    ┌──┴──┐
memberId,    YES     NO → SELECT org_members WHERE api_key_hash=?
memberEmail   │            │
  │           ▼         Found? → set ctx from member row
  │      role='owner'       │
  │      scopeTeam=null     NO → logAuditRaw(auth.failed) → 401
  │
  ▼
checkRateLimit(KV, orgId, RPM)
  │
  ├── exceeded? → 429 + Retry-After header
  │
  └── ok → logAudit(auth.login) → next()
```

Role Hierarchy

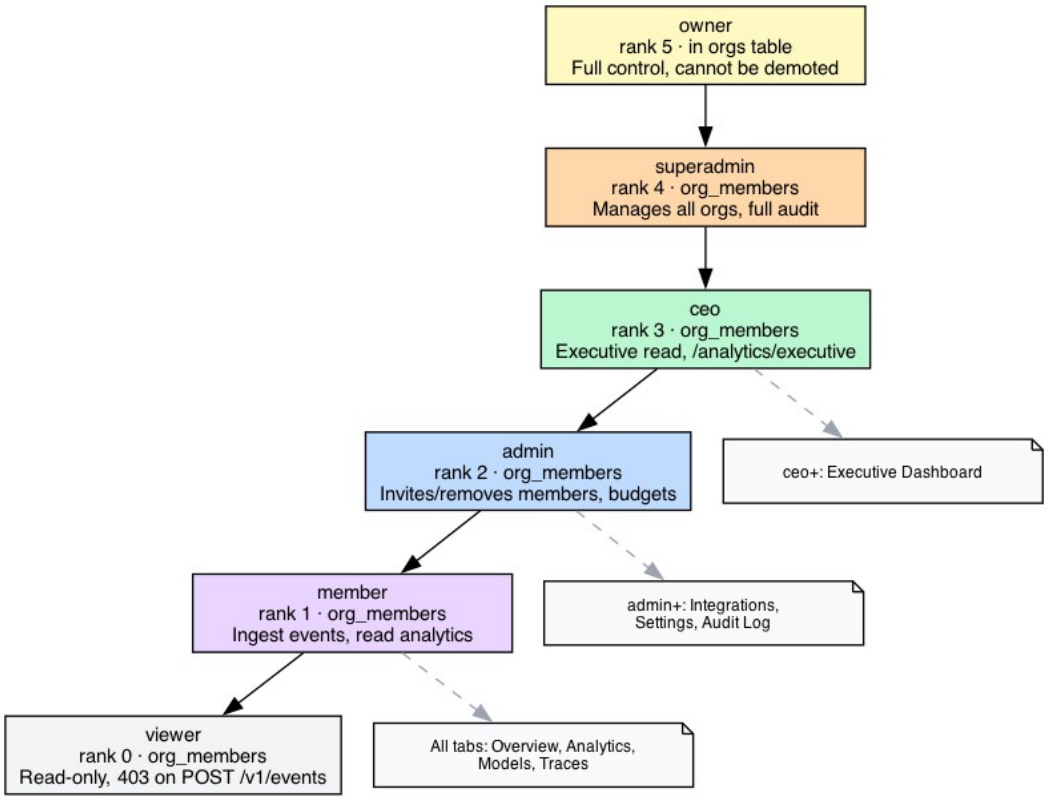

Figure 3 — RBAC Role Hierarchy

```
owner  ──────────────────────────── (in orgs table; cannot be demoted)
  │    privilege rank: 5
  ▼
superadmin ──────────────────────── (org_members, rank 4)
  │    platform-wide; manages all orgs; full audit access
  ▼
ceo ──────────────────────────────── (org_members, rank 3)
  │    executive read; GET /v1/analytics/executive only
  ▼
admin ────────────────────────────── (org_members, rank 2)
  │    invites/removes members; budgets; policies
  │    BLOCKED: cannot promote to superadmin/owner
  ▼
member ───────────────────────────── (org_members, rank 1)
  │    ingest events; read analytics (team-scoped if scope_team set)
  ▼
viewer ───────────────────────────── (org_members, rank 0)
     read-only; 403 on POST /v1/events and /v1/otel/*
```

Guards Table

Guard	Minimum rank	Allowed roles
`adminOnly`	admin (2)	owner, superadmin, ceo, admin
`executiveOnly`	ceo (3)	owner, superadmin, ceo
`superadminOnly`	superadmin (4)	owner, superadmin
inline viewer block	—	all except viewer

`hasRole(role, minimum)` uses `ROLE_RANK` lookup — always use this function, never compare role strings directly.

Session Security Properties

Property	Value	Reason
Cookie flags	`HttpOnly; SameSite=Lax; Secure`	XSS, CSRF, HTTPS-only
Cookie name	`__Host-cohrint_session` (prod)	Origin-bound, no Domain= attr required
`Domain`	`cohrint.com` (prod only)	Shared across `api.` and `app.`
TTL	30 days from creation (not last use)	Balance UX vs. security
Token entropy	256 bits (32 random bytes → 64 hex)	Unguessable
Storage	D1 `sessions` table	Consistent expiry + deletion

Key Recovery Flow

```
POST /v1/auth/recover { email }
  │
  ├── always returns 200 (never leak email existence)
  │
  ├── found in orgs table:
  │     generate 48-char hex token (24 random bytes)
  │     KV.put("recover:{token}", {orgId,'owner'}, TTL=3600s)
  │     email → one-click redeem link
  │
  └── found in org_members:
        email → hint only (admin must reissue member keys)


GET /v1/auth/recover/redeem?token=TOKEN
  ├── peek KV (don't delete — email scanners follow GETs)
  ├── valid → redirect /auth?confirm_token=TOKEN
  └── invalid → redirect /auth?recovery_error=expired


POST /v1/auth/recover/redeem { token }
  ├── KV.get → verify
  ├── KV.delete → consume (single-use)
  ├── generate new key → hash → UPDATE orgs
  └── return { ok, api_key, hint }
```

Critical: GET does not consume the token. Gmail/Outlook Safe Links follow GETs automatically. Only POST consumes.

5. Complete API Reference

Base URL: `https://api.cohrint.com`

Auth header: `Authorization: Bearer crt_{orgId}_{16hex}`

All endpoints return `Content-Type: application/json`.

5.1 Authentication

POST /v1/auth/signup

| Field | Type | Description |

|---|---|---|

| `email` | string | Org owner email |

| `name` | string | Org name |

| `plan` | string | `'free'` \| `'team'` \| `'enterprise'` |

Response: `{ ok, api_key, org_id, hint }`

```
curl -X POST https://api.cohrint.com/v1/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"email":"cto@acme.com","name":"Acme Corp","plan":"free"}'
```

POST /v1/auth/session (login)

| Field | Type | Description |

|---|---|---|

| `api_key` | string | `crt_...` raw key |

Response: `{ ok, role, org_id, plan }` + sets `__Host-cohrint_session` cookie.

Also returns `sse_token` (32-char hex, 120s TTL) for SSE connection.

POST /v1/auth/logout

Requires: session cookie. Deletes session from D1. Returns `{ ok: true }`.

POST /v1/auth/recover

| Field | Type | Description |

|---|---|---|

| `email` | string | Owner or member email |

Always returns `200 { ok: true }` — never leaks email existence.

GET /v1/auth/recover/redeem?token=TOKEN

Does NOT consume token (email scanner protection). Redirects to `/auth?confirm_token=TOKEN` or `/auth?recovery_error=expired`.

POST /v1/auth/recover/redeem

| Field | Type | Description |

|---|---|---|

| `token` | string | Recovery token from email link |

Consumes token (single-use). Returns `{ ok, api_key, hint }`.

POST /v1/auth/rotate (owner key rotation)

Auth: owner only. Generates new key, old key immediately invalid, sessions survive.

Returns: `{ ok, api_key, hint }`

POST /v1/auth/members/:id/rotate

Auth: admin+. Targets member by `member_id`. Sends email with new key.

Returns: `{ ok, hint }`

5.2 Members & Teams

POST /v1/auth/members (invite)

Auth: adminOnly. Escalation guard: cannot invite to `superadmin` or `owner`.

| Field | Type | Description |

|---|---|---|

| `email` | string | New member email |

| `name` | string | Display name |

| `role` | string | `'admin'\|'member'\|'viewer'\|'ceo'` |

| `scope_team` | string? | Team restriction (null = see all) |

Response: `{ member_id, api_key (once), hint, role, scope_team }`

```
curl -X POST https://api.cohrint.com/v1/auth/members \
  -H "Authorization: Bearer crt_..." \
  -H "Content-Type: application/json" \
  -d '{"email":"dev@acme.com","name":"Dev","role":"member","scope_team":"backend"}'
```

DELETE /v1/auth/members/:id

Auth: adminOnly. Soft-removes member. Returns `{ ok: true }`.

PATCH /v1/auth/members/:id

Auth: adminOnly. Update `role` and/or `scope_team`.

GET /v1/admin/overview

Auth: adminOnly. Returns combined payload:

```
{
  "org": { "id", "name", "email", "plan", "budget_usd", "budget_pct", "mtd_cost_usd" },
  "totals": { "total_cost_usd", "total_tokens", "total_requests", "avg_latency_ms" },
  "teams": [{ "team", "cost_usd", "budget_usd", "budget_pct" }],
  "members": [{ "id", "email", "name", "role", "scope_team", "hallucination_score" }],
  "period_days": 30
}
```

`members[].hallucination_score` = average across their events for the period (new in v2.0).

GET /v1/admin/budget-policies

Auth: adminOnly. Returns all budget policies for org.

POST /v1/admin/budget-policies

Auth: adminOnly. `enforcement: "block"` → `POST /v1/events` returns 429 when scope budget exceeded.

PUT /v1/admin/budget-policies/:id

Auth: adminOnly. Update budget/period/enforcement.

DELETE /v1/admin/budget-policies/:id

Auth: adminOnly.

5.3 Events (Ingest)

POST /v1/events

Auth: any (viewer gets 403). Free tier: 50,000 events/calendar-month.

Key request fields:

Field	Aliases	Description
`event_id` / `id`	—	Unique event ID (dedup via INSERT OR IGNORE)
`provider`	—	`'openai'|'anthropic'|'google'`
`model`	—	Model name string
`prompt_tokens`	`usage_prompt_tokens`	Input token count
`completion_tokens`	`usage_completion_tokens`	Output token count
`cache_tokens`	`usage_cached_tokens`	Cached token count

| `total_tokens` | `usage_total_tokens` | Total (or sum of above) |

| `cost_usd` | `cost_total_usd`, `cost_total_cost_usd`, `total_cost_usd` | Call cost in USD |

| `latency_ms` | — | End-to-end latency |

| `team` | — | Team tag for scoping |

| `trace_id` | — | Agent trace grouping |

| `parent_event_id` | — | Parent span for DAG |

| `span_depth` | — | Visual indentation hint |

| `developer_email` | — | Developer attribution |

Response: `201 { ok: true, id: event_id }`

```
curl -X POST https://api.cohrint.com/v1/events \
  -H "Authorization: Bearer crt_..." \
  -H "Content-Type: application/json" \
  -d '{
    "event_id": "evt_abc123",
    "provider": "anthropic",
    "model": "claude-sonnet-4-6",
    "prompt_tokens": 1500,
    "completion_tokens": 400,
    "cost_usd": 0.0105,
    "latency_ms": 1200,
    "team": "backend",
    "environment": "production"
  }'
```

POST /v1/events/batch

Max 500 events per request. Single D1 round-trip via `DB.batch().broadcastEvent()` on last event only.

```
curl -X POST https://api.cohrint.com/v1/events/batch \
  -H "Authorization: Bearer crt_..." \
  -H "Content-Type: application/json" \
  -d '{"events": [{...}, {...}]}'
# Response: { "ok": true, "accepted": 10, "failed": 0 }
```

PATCH /v1/events/:id/scores

Auth: any. Used by async LLM judge (Claude Opus 4.6). Fields: `hallucination_score`, `faithfulness_score`, `relevancy_score`, `consistency_score`, `toxicity_score`, `efficiency_score`. All nullable.

5.4 Analytics

All analytics endpoints require auth. Team-scoped members automatically get filtered results.

GET /v1/analytics/summary

No query params required. KV-cached 5 minutes. Bypass cache: add `?agent=<name>`.

Key response fields:

Field	Description
`today_cost_usd`	Cost since midnight UTC
`mtd_cost_usd`	Month-to-date cost
`projected_month_end_usd`	`daily_avg × days_in_month`
`daily_avg_cost_usd`	`mtd / days_elapsed`
`days_until_budget_exhausted`	`ceil((budget - mtd) / daily_avg)`, 0 when exceeded
`budget_usd`	Org monthly budget (0 = not set)
`session_cost_usd`	Last-30-min window cost
`total_tokens`	MTD total tokens
`request_count`	MTD request count
`cache_hit_rate`	Cache hit % (0–1.0)
`hallucination_score`	Average across all org events

| `last_event_at` | ISO timestamp of most recent event |

```
curl https://api.cohrint.com/v1/analytics/summary \
  -H "Authorization: Bearer crt_..."
```

GET /v1/analytics/kpis?period=N

N = days (max 365). Returns totals + averages + streaming count.

```
curl "https://api.cohrint.com/v1/analytics/kpis?period=30" \
  -H "Authorization: Bearer crt_..."
```

GET /v1/analytics/timeseries?period=N

N = days (max 365). Returns daily cost/tokens/requests array.

GET /v1/analytics/models?period=N

Top 25 models by cost for the period.

Response: `{ models: [{ model, provider, cost_usd, tokens, requests, avg_cost }] }`

GET /v1/analytics/teams?period=N

Per-team cost + budget percentage.

Response: `{ teams: [{ team, cost_usd, budget_usd, budget_pct, tokens, requests }] }`

GET /v1/analytics/traces?period=N

Agent trace summaries. N max 30 days. Top 100 traces ordered by `started_at DESC`.

SQL:

```
SELECT
  trace_id,
  MIN(agent_name)      AS name,
  COUNT(*)             AS spans,
  SUM(cost_usd)        AS cost,
  SUM(latency_ms)      AS latency,
  MAX(CASE WHEN parent_event_id IS NULL THEN 1 ELSE 0 END) AS has_root,
  MIN(created_at)      AS started_at
FROM events
WHERE org_id = ? AND trace_id IS NOT NULL AND created_at >= ?
GROUP BY trace_id
ORDER BY started_at DESC
LIMIT 100
```

GET /v1/analytics/traces/:traceId

Full span DAG for one trace. RBAC: members below `admin` rank see only their own spans.

SQL:

```
SELECT
  event_id AS id, parent_event_id AS parent_id,
  agent_name, model, provider, feature,
  span_depth, prompt_tokens, completion_tokens,
  cache_tokens, cost_usd, latency_ms, created_at
FROM events
WHERE org_id = ? AND trace_id = ?
  [AND team = ? if scopeTeam]
  [AND developer_email = ? if role < admin]
ORDER BY created_at ASC
```

GET /v1/analytics/cost?period=N

CI cost gate endpoint. Returns:

```
{ "today_cost_usd": 1.23, "period_cost_usd": 45.67, "period_days": 7 }
```

```
# GitHub Actions cost gate
COST=$(curl -sf -H "Authorization: Bearer $VANTAGE_KEY" \
  "https://api.cohrint.com/v1/analytics/cost?period=1" | jq '.today_cost_usd')
python -c "import sys; sys.exit(1 if float('$COST') > 5.0 else 0)"
```

GET /v1/analytics/executive

Auth: executiveOnly (ceo+). 30-day cross-source spend roll-up.

5.5 Semantic Cache

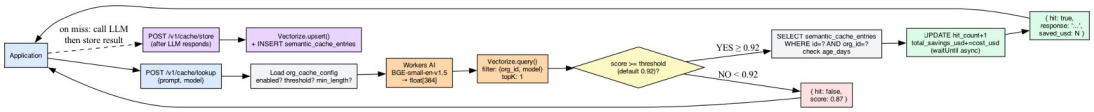

Figure 4 — Semantic Cache Flow

POST /v1/cache/lookup

```
curl -X POST https://api.cohrint.com/v1/cache/lookup \
  -H "Authorization: Bearer crt_..." \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Summarize this document...", "model": "gpt-4o"}'

# Hit response:
{
  "hit": true,
  "score": 0.9731,
  "response": "The document covers...",
  "prompt_tokens": 450,
  "completion_tokens": 120,
  "saved_usd": 0.00612,
  "cache_entry_id": "uuid-..."
}

# Miss response:
{ "hit": false, "score": 0.8412 }
```

POST /v1/cache/store

```
curl -X POST https://api.cohrint.com/v1/cache/store \
  -H "Authorization: Bearer crt_..." \
  -d '{
    "prompt": "Summarize this document...",
    "model": "gpt-4o",
    "response": "The document covers...",
    "prompt_tokens": 450,
    "completion_tokens": 120,
    "cost_usd": 0.00612,
    "prompt_hash": "sha256hexofprompt"
  }'
# → 201 { "stored": true, "cache_entry_id": "uuid-..." }
# → 200 { "stored": false, "reason": "duplicate", "cache_entry_id": "uuid-..." }
```

GET /v1/cache/stats

Returns hit rate, total savings USD, recent cache entries.

PATCH /v1/cache/config

Auth: adminOnly.

```
curl -X PATCH https://api.cohrint.com/v1/cache/config \
  -H "Authorization: Bearer crt_..." \
  -d '{"similarity_threshold": 0.88, "max_cache_age_days": 14}'
```

DELETE /v1/cache/entries/:id

Auth: adminOnly. Removes entry from both Vectorize and D1.

5.6 Prompt Registry

GET /v1/prompts

Auth: any. Lists all non-deleted prompts with version count + total cost.

POST /v1/prompts

Auth: adminOnly.

```
curl -X POST https://api.cohrint.com/v1/prompts \
  -H "Authorization: Bearer crt_..." \
  -d '{
    "name": "customer-support-v1",
    "description": "Main customer support prompt",
    "content": "You are a helpful support agent...",
    "model": "claude-sonnet-4-6"
  }'
```

GET /v1/prompts/:id

Auth: any. Returns prompt + version list (200-char preview per version).

POST /v1/prompts/:id/versions

Auth: adminOnly. Auto-increments `version_num`.

POST /v1/prompts/usage

Auth: any (SDK). Records LLM event attribution to a prompt version.

```
curl -X POST https://api.cohrint.com/v1/prompts/usage \
  -H "Authorization: Bearer crt_..." \
  -d '{
    "version_id": "uuid-...",
    "event_id": "evt-...",
    "cost_usd": 0.0032,
    "prompt_tokens": 800,
    "completion_tokens": 200
  }'
```

GET /v1/prompts/analytics/comparison

Auth: any. Returns cost comparison across all versions with delta percentages.

```
{
  "prompt": { "id", "name" },
  "versions": [
    { "version_num": 1, "avg_cost_usd": 0.0050, "total_calls": 1000, "cost_delta_pct":
null },
    { "version_num": 2, "avg_cost_usd": 0.0032, "total_calls": 800, "cost_delta_pct":
-36.0 },
    { "version_num": 3, "avg_cost_usd": 0.0041, "total_calls": 200, "cost_delta_pct": 28.1 }
  ]
}
```

5.7 Benchmarks

GET /v1/benchmark/summary

Public (no auth). Returns available metric+model combos with sample sizes.

GET /v1/benchmark/percentiles?metric=X&model=Y

Public (no auth). Returns p25/p50/p75/p90 for the latest quarter with `sample_size >= 5`.

Returns `404 "Insufficient data"` if total sample < 5.

```
curl "https://api.cohrint.com/v1/benchmark/percentiles?metric=cost_per_token&model=gpt-4o"
```

POST /v1/benchmark/contribute

Auth: owner or admin. Computes + upserts quarterly snapshot for calling org.

```
curl -X POST https://api.cohrint.com/v1/benchmark/contribute \
  -H "Authorization: Bearer crt_..."
# Response: { "ok": true }
# or { "ok": false, "reason": "not_opted_in" | "no_usage_data" }
```

5.8 Cross-Platform

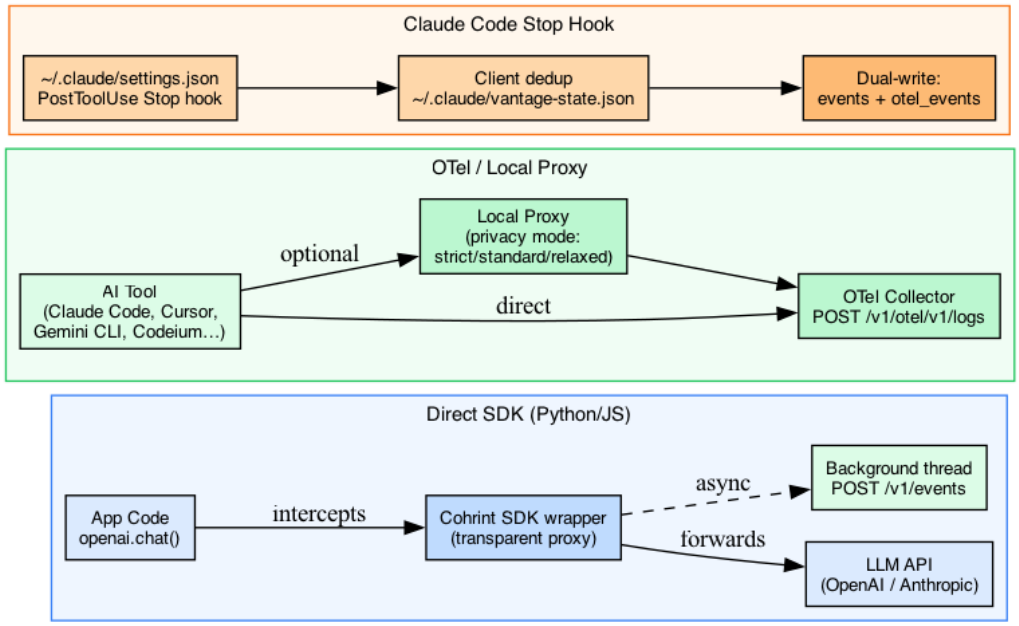

Figure 9 — SDK Integration Patterns

All `GET /v1/cross-platform/*` require adminOnly (PR #67).

GET /v1/cross-platform/summary

Spend roll-up across all sources (OTel + Copilot + Datadog + SDK).

GET /v1/cross-platform/developers

Per-developer cost breakdown across all connected tools.

GET /v1/cross-platform/models

Cost by model across all data sources.

GET /v1/cross-platform/live

Recent cross-platform events (last 50).

GET /v1/cross-platform/budget

Cross-platform budget status.

5.9 Integrations

POST /v1/copilot/connect

Auth: adminOnly.

```
curl -X POST https://api.cohrint.com/v1/copilot/connect \
  -H "Authorization: Bearer crt_..." \
  -d '{"github_org": "acme-corp", "pat_token": "ghp_..."}'
# PAT encrypted via AES-256-GCM before KV storage — never stored in D1
```

DELETE /v1/copilot/disconnect

Auth: adminOnly. Deletes connection + KV token.

POST /v1/copilot/sync

Auth: adminOnly. Manual sync trigger (runs same flow as Sunday UTC cron).

GET /v1/copilot/status

Auth: any. Returns `{ status, github_org, last_sync }`.

POST /v1/datadog/connect

Auth: adminOnly.

```
curl -X POST https://api.cohrint.com/v1/datadog/connect \
  -H "Authorization: Bearer crt_..." \
  -d '{"api_key": "dd_api_...", "dd_site": "datadoghq.com"}'
# Allowed sites: datadoghq.com, datadoghq.eu, ddog-gov.com, us3.datadoghq.com,
us5.datadoghq.com
```

DELETE /v1/datadog/disconnect

Auth: adminOnly.

POST /v1/datadog/sync

Auth: adminOnly. Manual sync.

GET /v1/datadog/status

Auth: any. Returns `{ status, dd_site, last_push }`.

POST /v1/otel/v1/logs (OTel ingest)

Accepts OpenTelemetry OTLP format. Auth: Bearer key. Writes to `otel_events` + `cross_platform_usage`.

```
curl -X POST https://api.cohrint.com/v1/otel/v1/logs \
  -H "Authorization: Bearer crt_..." \
  -H "Content-Type: application/json" \
  -d '{"resourceLogs": [...]}'
```

GET /v1/otel/v1/metrics

Returns OTel-sourced metrics for org.

5.10 Alerts & Audit

POST /v1/alerts/config

Auth: adminOnly. Configure Slack webhook + thresholds.

```
curl -X POST https://api.cohrint.com/v1/alerts/config \
  -H "Authorization: Bearer crt_..." \
  -d '{"slack_webhook_url": "https://hooks.slack.com/..."}'
```

GET /v1/alerts/config

Auth: adminOnly. Returns current alert configuration.

GET /v1/audit-log?limit=50&before=TIMESTAMP

Auth: adminOnly. Cursor-based pagination via `before` param (ISO format `YYYY-MM-DD HH:MM:SS`). Sorted DESC by `created_at`. This endpoint itself is NOT logged.

```
curl "https://api.cohrint.com/v1/audit-log?limit=50&before=2026-04-17+00:00:00" \
  -H "Authorization: Bearer crt_..."
```

5.11 Real-Time Streaming

GET /v1/stream/:orgId

SSE connection. Auth via `?sse_token=` (one-time, 120s TTL) or `?token=crt_...`.

Polls KV `stream:{orgId}:latest` every 2s. Sends `data: {event json}` on new events, `: ping` otherwise. Auto-closes at 25s (Worker wall-clock limit). Client must auto-reconnect.

```
const evtSource = new EventSource(
  `https://api.cohrint.com/v1/stream/${orgId}?sse_token=${sseToken}`
);
evtSource.onmessage = (e) => console.log(JSON.parse(e.data));
```

6. Dashboard Features

6.1 KPI Cards

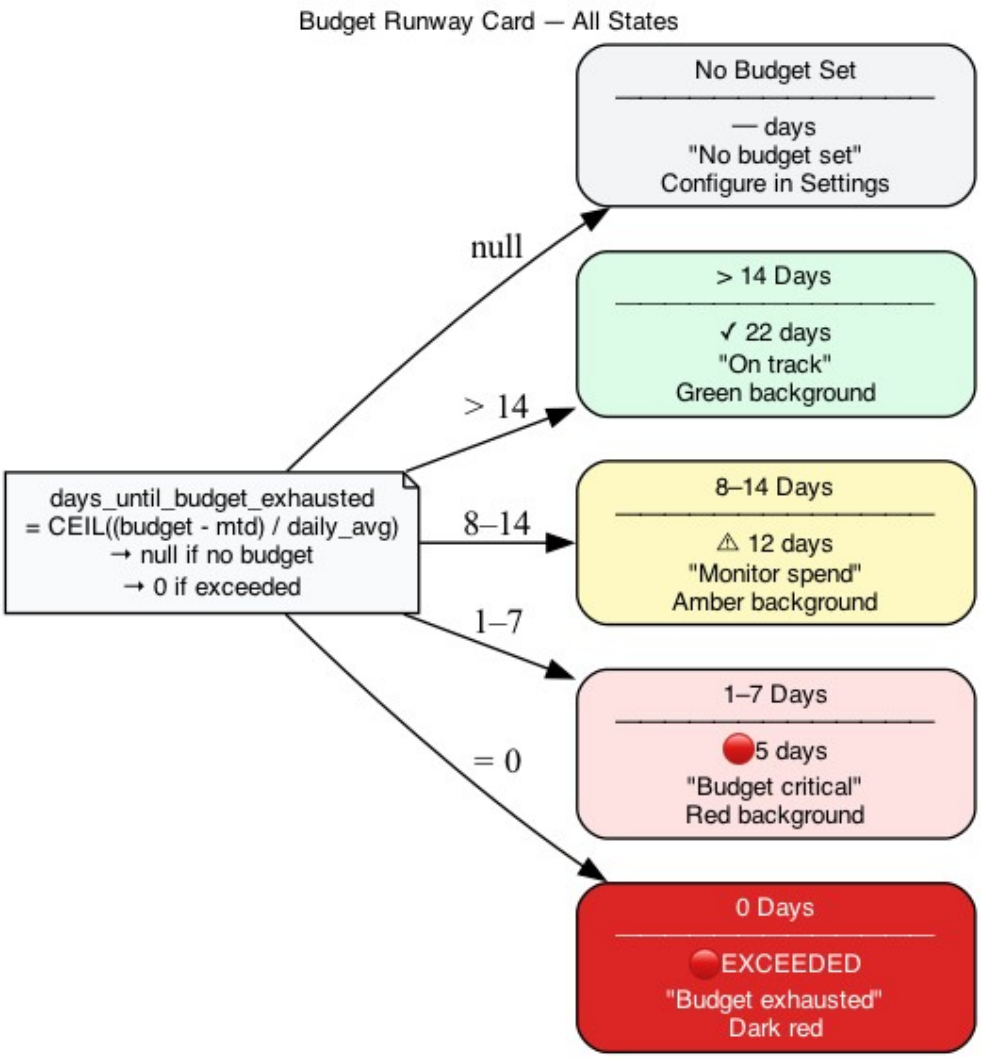

Figure 17 — Budget Runway Card States

| Card | Field | Description |

|---|---|---|

| Today Cost | `today_cost_usd` | Cost since midnight UTC |

| MTD Cost | `mtd_cost_usd` | Month-to-date spend |

| Session Cost | `session_cost_usd` | Last 30-min window |

| Token Count | `total_tokens` | MTD total tokens |

| Request Count | `request_count` | MTD requests |

| Projected Month-End | `projected_month_end_usd` | `daily_avg × days_in_month` — shown in amber |

| Budget Runway | `days_until_budget_exhausted` | Color-coded: red ≤7d, amber ≤14d, green >14d, "Exceeded" at 0 |

| Cache Hit Rate | `cache_hit_rate` | 0–1.0 percentage |

6.2 Charts & Tabs

- **Timeseries cost chart** — Chart.js line/bar chart, daily cost for selected period
- **Models tab** — Top 25 models by cost with token breakdown
- **Teams tab** — Per-team cost + budget% bar chart
- **Traces tab** — Agent Trace DAG visualization, collapsible tree with depth/latency/cost per node
- **Cross-Platform tab** — Per-developer spend across all tools (admin+ only)

6.3 RBAC Tab Visibility (PR #67)

Dashboard Tab Visibility by Role

Role	Overview	Analytics	Models	Traces	Integrations	Settings	Audit Log	Executive View
viewer	✅	✅	✅	–	–	–	–	–
member	✅	✅	✅	✅	–	–	–	–
admin	✅	✅	✅	✅	✅	✅	✅	–
ceo	✅	✅	✅	✅	–	–	–	✅
superadmin/owner	✅	✅	✅	✅	✅	✅	✅	✅

Figure 18 — Tab Visibility by Role

```
User logs in
     │
     ▼
GET /v1/auth/session → role
     │
     ├── role ∈ {owner, superadmin, ceo, admin}
     │     │
     │     ├── show: All tabs including Cross-Platform, Integrations
     │     ├── show: Cost per Developer card in Overview
     │     ├── show: Budgets & Alerts tab
     │     └── show: Per-developer hallucination in members table
     │
     └── role ∈ {member, viewer}
           │
           ├── hide: Cross-Platform tab (display:none)
           ├── hide: Cost per Developer card
           ├── hide: Budgets & Alerts tab
           ├── nav('integrations') → redirected to nav('overview')
           └── members table omits hallucination_score column
```

Tab visibility by role:

| Tab / Feature | owner | superadmin | ceo | admin | member | viewer |

|---|---|---|---|---|---|---|

| Overview, Analytics, Models, Traces | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

| Cross-Platform tab | ✓ | ✓ | ✓ | ✓ | — | — |

| Integrations view | ✓ | ✓ | ✓ | ✓ | — | — |

| Budgets & Alerts | ✓ | ✓ | ✓ | ✓ | — | — |

| Executive view | ✓ | ✓ | ✓ | — | — | — |

| Admin panel | ✓ | ✓ | — | — | — | — |

| Audit Log | ✓ | ✓ | ✓ | ✓ | — | — |

| Per-dev hallucination | ✓ | ✓ | ✓ | ✓ | — | — |

6.4 Real-Time Stream

SSE token generated at login (`GET /v1/auth/session`), 120s TTL, one-time use. Browser `EventSource` connects to `/v1/stream/:orgId?sse_token=X`. Worker polls KV every 2s, streams new events as `data: {...}`. Connection closes at 25s (Worker limit). Client auto-reconnects via `EventSource` built-in retry. On tab exit, `evtSource.close()` tears down connection.

6.5 Frontend File Structure

```
cohrint-frontend/
├── index.html        — Landing page
├── app.html          — Main SPA dashboard (auth-gated)
├── auth.html         — Sign-in
├── signup.html       — Signup
├── docs.html         — API documentation
├── calculator.html   — Cost calculator
├── trust.html        — Security architecture, privacy modes
├── report.html       — State of AI Coding Spend 2026 (email-gated)
├── sw.js             — Service Worker (offline)
├── manifest.json     — PWA manifest
├── _headers          — Cloudflare Pages headers (CSP, cache control)
└── _redirects        — URL routing rules
```

7. Semantic Cache — Deep Dive

Architecture

```
Application                    Worker (/v1/cache/*)        Vectorize + Workers AI
     |                                |                          |
     |──POST /v1/cache/lookup─────────►|                          |
     |  { prompt, model }             |                          |
     |                                |──getOrgCacheConfig()──►D1|
     |                                |  enabled? min_length?    |
     |                                |                          |
     |                                |──embedText(prompt)───────►
     |                                |  AI.run(bge-small-en)    |
     |                                |◄─ float[384] embedding───|
     |                                |                          |
     |                                |──VECTORIZE.query()───────►
     |                                |  filter: {org_id, model} |
     |                                |  topK: 1                 |
     |                                |◄─ { id, score }──────────|
     |                                |                          |
     |                                |  score >= threshold?     |
     |                                |  ┌─YES──────────────────►|
     |                                |  │  SELECT * FROM        |
     |                                |  │  semantic_cache_entries|
     |                                |  │  WHERE id=? org_id=?  |
     |                                |  │  check age_days       |
     |                                |  │  UPDATE hit_count+1   |
     |◄──{ hit:true, response }───────|  │  (waitUntil async)    |
     |                                |  └──────────────────────── |
     |                                |  NO → { hit:false, score }|
     |◄──{ hit:false }────────────────|                          |
```

Embedding Model

- **Model:** `@cf/baai/bge-small-en-v1.5` via Cloudflare Workers AI
- **Dimensions:** 384
- **Metric:** Cosine similarity
- **Per-org namespace isolation:** Vectorize metadata filter `{ org_id }` — org A cannot retrieve org B's vectors

Configuration Defaults

| Parameter | Default | Range | Description |

|---|---|---|---|

| `enabled` | `1` | 0/1 | Master on/off switch per org |

| `similarity_threshold` | `0.92` | 0.0–1.0 | Cosine similarity floor for a hit |

| `min_prompt_length` | `10` | chars | Prompts below this are not cached |

| `max_cache_age_days` | `30` | days | Entries older than this are evicted |

Cache Hit/Miss Decision Algorithm

23. Check `org_cache_config.enabled` — skip if disabled
24. Check `len(prompt) >= min_prompt_length` — skip short prompts
25. Embed prompt via `AI.run('@cf/baai/bge-small-en-v1.5', { text: [prompt] })` → `float[384]`
26. Query Vectorize: `topK=1`, filter `{ org_id, model }` (+ `team_id` if scoped)
27. If `score >= similarity_threshold`: fetch D1 entry, check age, return hit
28. If `age_days > max_cache_age_days`: evict entry (async `waitUntil`), return miss
29. Otherwise: return miss with score

Cost Savings Calculation

When a cache hit occurs:

- `saved_usd` = `cost_usd` from the original stored call (the cost that would have been incurred)
- `UPDATE semantic_cache_entries SET hit_count = hit_count + 1, total_savings_usd = total_savings_usd + saved_usd`
- Dashboard shows cumulative `total_savings_usd` across all entries

Team Namespace Isolation

When the authenticated member has `scopeTeam` set, `team_id` is included in both the Vectorize metadata filter and the D1 WHERE clause. A scoped member for `team='backend'` cannot retrieve a cache entry stored by `team='frontend'`, even if the prompt is semantically equivalent. Enforced at both layers.

Stale Entry Eviction

On a hit, checks entry age:

```
SELECT (julianday('now') - julianday(created_at)) AS age_days
FROM semantic_cache_entries WHERE id = ?
```

If `age_days > max_cache_age_days`:

30. `VECTORIZE.deleteByIds([id])` — remove from vector index

31. `DELETE FROM semantic_cache_entries WHERE id = ?` — remove from D1
32. Returns `{ hit: false, reason: 'entry_expired' }`

Duplicate Prevention

Before embedding+storing:

```
SELECT id FROM semantic_cache_entries
WHERE org_id = ? AND prompt_hash = ? AND model = ?
```

If found: returns `{ stored: false, reason: 'duplicate', cache_entry_id }` without hitting Vectorize.

8. Prompt Registry — Deep Dive

Flow: Create → Version → Track Usage

```
POST /v1/prompts (admin+)
  → creates prompt record (name unique per org)
  → optionally creates first version inline

POST /v1/prompts/:id/versions (admin+)
  → auto-increments version_num
  → stores full content + target model

SDK: POST /v1/prompts/usage (any)
  → links event_id to version_id
  → updates rolling average cost on prompt_versions

GET /v1/prompts/analytics/comparison
  → returns versions sorted by version_num ASC
  → computes cost_delta_pct between consecutive versions
```

Version Comparison Example

```
v1  avg_cost=$0.0050  total_calls=1000   cost_delta_pct=null
v2  avg_cost=$0.0032  total_calls=800    cost_delta_pct=-36.0  ← 36% cheaper
v3  avg_cost=$0.0041  total_calls=200    cost_delta_pct=+28.1  ← regression
```

Formula: `delta = ((v_n.avg_cost - v_{n-1}.avg_cost) / v_{n-1}.avg_cost) * 100`

Rolling Average Update

On each `POST /v1/prompts/usage`:

```
new_total_calls = old_total_calls + 1
new_total_cost  = old_total_cost + cost_usd
new_avg_cost    = new_total_cost / new_total_calls
new_avg_prompt  = (old_avg_prompt * old_calls + new_tokens) / new_calls
```

Executed as `DB.batch([INSERT OR IGNORE prompt_usage, UPDATE prompt_versions SET ...])` — single round-trip.

RBAC

| Operation | Required role |

|---|---|

| List / read prompts | any authenticated |

| Version analytics comparison | any authenticated |

| Create prompt | admin+ |

| Add new version | admin+ |

| Soft delete | admin+ |

| Record usage (SDK) | any authenticated |

Soft delete: `UPDATE prompts SET deleted_at = datetime('now')`. Never hard-deletes. `GET /v1/prompts` excludes `WHERE deleted_at IS NOT NULL`.

9. Agent Trace DAG — Deep Dive

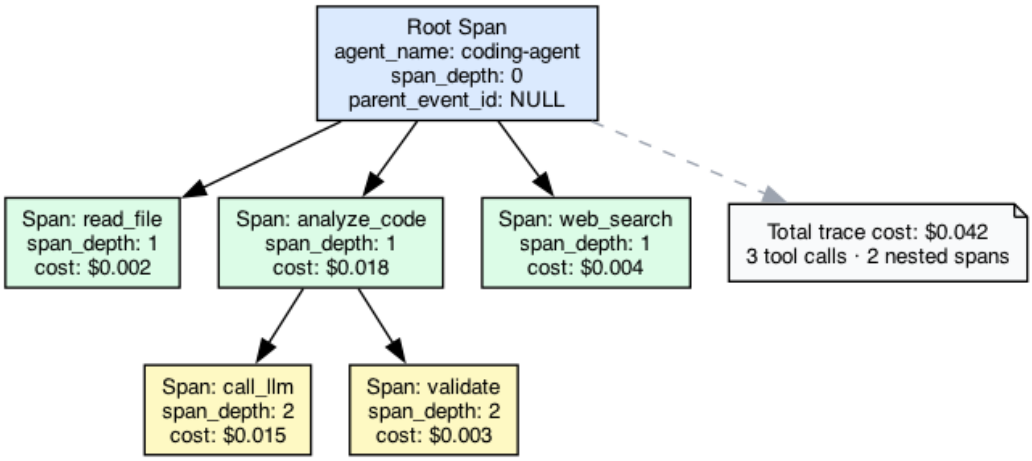

Figure 5 — Agent Trace DAG

Data Model

Three columns on `events` table enable the DAG:

Column	Type	Description
`trace_id`	TEXT	Groups all spans in one agent session
`parent_event_id`	TEXT	NULL = root span; otherwise FK to `events.id`
`span_depth`	INTEGER	Visual indentation hint (0 = root, 1 = child, etc.)

Example: Multi-Agent Trace

```
import requests, uuid

trace_id = str(uuid.uuid4())

# Root span
root_id = str(uuid.uuid4())
requests.post("https://api.cohrint.com/v1/events", json={
  "event_id": root_id,
  "trace_id": trace_id,
  "parent_event_id": None,
  "agent_name": "ResearchAgent",
  "span_depth": 0,
  "model": "claude-sonnet-4-6",
  "cost_usd": 0.0050
}, headers={"Authorization": "Bearer crt_..."})

# Child span
requests.post("https://api.cohrint.com/v1/events", json={
  "event_id": str(uuid.uuid4()),
  "trace_id": trace_id,
  "parent_event_id": root_id,
  "agent_name": "SummarizerAgent",
  "span_depth": 1,
  "model": "claude-haiku-4-5",
  "cost_usd": 0.0012
})
```

DAG Reconstruction in Frontend

```
spans returned ordered by created_at ASC
  │
  ▼
Build adjacency map: id → [children]
  root = spans where parent_id IS NULL
  │
  ▼
Recursive tree render: root → children → grandchildren
  span_depth provides visual indentation hint
```

RBAC Enforcement on Trace Detail

```
GET /v1/analytics/traces/:traceId
          │
          ▼
  check role
  isPrivileged (admin+)?
  ┌── YES: no devClause — see all spans
  └── NO:  AND developer_email = ?  — only own spans
```

Members below `admin` rank can only see spans where `developer_email` matches their own email. Team scoping also applied if `scopeTeam` is set.

DAG Sequence Diagram

```
Client (dashboard)          Worker                  D1
     |                        |                     |
     |—GET /analytics/traces——▶|                     |
     |                        |—SELECT trace summary▶|
     |◀—{ traces: [...] }———— |◀——————————————————— |
     |                        |                     |
     |—GET /traces/:traceId——▶|                     |
     |                        |  check role         |
     |                        |  isPrivileged?       |
     |                        |  ┌── YES: no devClause
     |                        |  └── NO:  AND developer_email=?
     |                        |—SELECT spans————————▶|
     |◀—{ spans: [...] }———— |◀——————————————————— |
     |                        |                     |
     |  Build DAG in JS       |                     |
     |  root = parent_id=null |                     |
     |  children via Map      |                     |
     |  Render tree UI        |                     |
```

10. Alert System

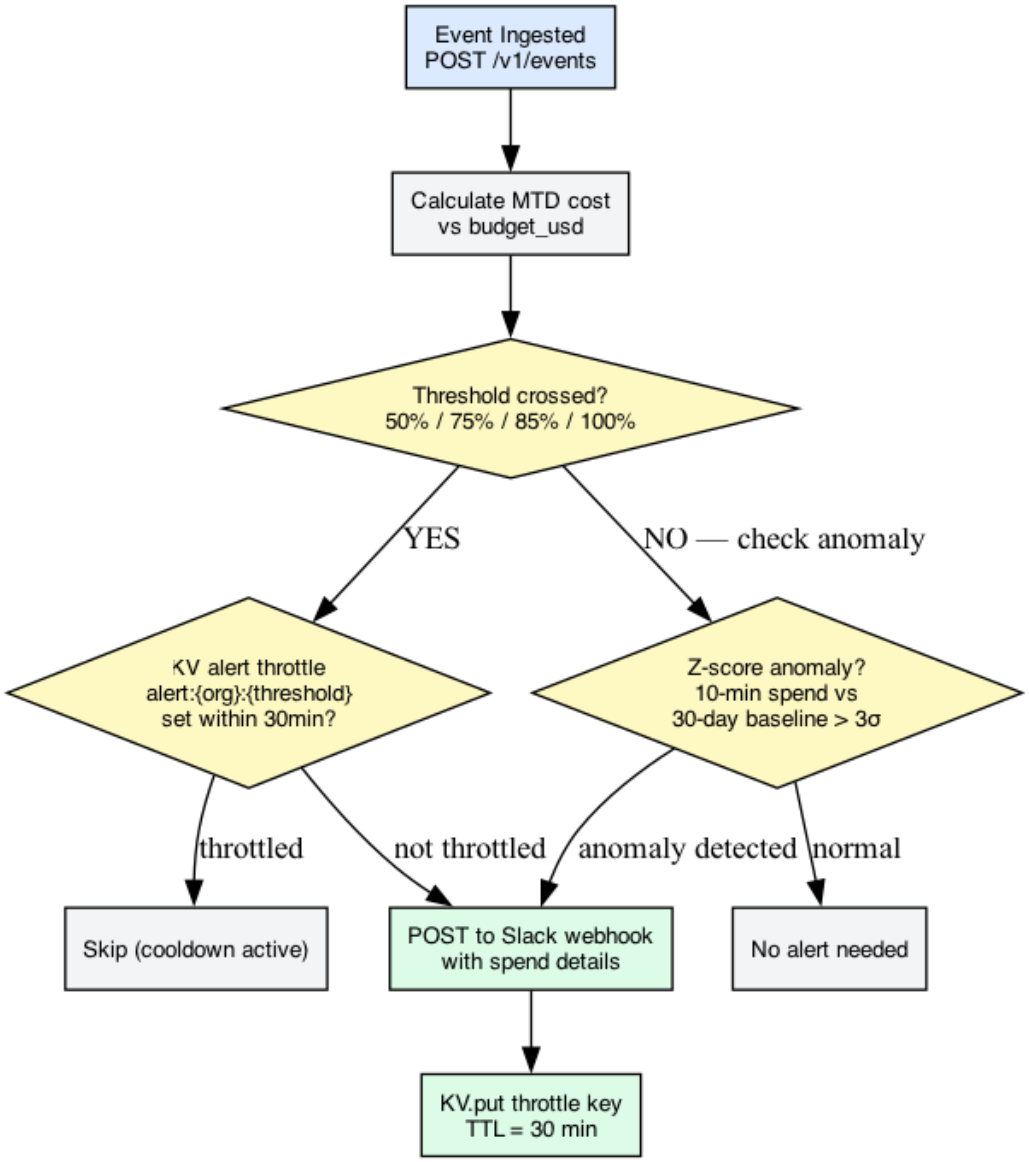

Figure 10 — Alert System

Budget Alert Algorithm

```
async function maybeSendBudgetAlert(db, kv, orgId, mtdCost, budgetUsd) {
  const pct = (mtdCost / budgetUsd) * 100;

  let alertType: string | null = null;
  if (pct >= 100) alertType = 'budget_100';
  else if (pct >= 80) alertType = 'budget_80';
  if (!alertType) return;

  // Throttle: one alert per type per hour
  const throttleKey = `alert:${orgId}:${alertType}`;
  if (await kv.get(throttleKey)) return;

  const slackUrl = await kv.get(`slack:${orgId}`);
  if (!slackUrl) return;   // webhook not configured

  await sendSlackMessage(slackUrl, budgetAlertPayload);
  await kv.put(throttleKey, '1', { expirationTtl: 3600 });
}
```

Thresholds: 80% and 100%

Throttle: Once per hour per threshold via KV key `alert:{orgId}:{alertType}` TTL=3600s

Webhook storage: Slack URL cached in KV from D1 on configuration

Graduated Budget Thresholds

The `budget_policies` table supports graduated thresholds at 50% / 75% / 85% / 100% for per-team and per-org budgets. `enforcement` field:

- `'alert'` → Slack webhook only
- `'block'` → `POST /v1/events` returns 429 when scope budget exceeded

Z-Score Anomaly Detection (planned/partial)

```
def is_anomaly(current_hour_cost, historical_costs):
    mean = statistics.mean(historical_costs)
    std  = statistics.stdev(historical_costs)
    z_score = (current_hour_cost - mean) / max(std, 0.001)
    return z_score > 3.0  # 3-sigma threshold
```

- 10-minute window vs 30-day baseline
- 3σ threshold
- Run as Cloudflare Cron Trigger

- KV alert throttle: 30-min cooldown to prevent duplicate Slack alerts

KV Keys for Alert System

| Key | TTL | Description |

|---|---|---|

| `alert:{orgId}:budget_80` | 3600s | 80% alert throttle |

| `alert:{orgId}:budget_100` | 3600s | 100% alert throttle |

| `slack:{orgId}` | No TTL | Cached Slack webhook URL |

11. Rate Limiting

Fixed-Window Counter via KV

```
async function checkRateLimit(kv: KVNamespace, orgId: string, limitRpm: number):
Promise<boolean> {
  const key   = `rl:${orgId}:${Math.floor(Date.now() / 60_000)}`;  // 1-minute bucket
  const raw   = await kv.get(key);
  const count = raw ? parseInt(raw, 10) : 0;
  if (count >= limitRpm) return false;
  await kv.put(key, String(count + 1), { expirationTtl: 70 });     // 70s TTL (60s + 10s
skew)
  return true;
}
```

- Rate limited **per-org** (shared across all members)
- Default: `RATE_LIMIT_RPM = 1000` (configurable via env var)
- KV key: `rl:{orgId}:{minuteBucket}` with 70s TTL
- 429 response includes: `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining: 0`
- KV write failure = allow (graceful degradation, non-blocking)
- Known tradeoff: burst at minute boundary (2x allowed for ~1 second). Acceptable for background telemetry.

Unblock an Org

```
# Delete stuck rate limit key
npx wrangler kv key delete --namespace-id=... "rl:myorg:$(date +%s | awk '{print
int($1/60)}')"
```

12. Client Libraries & SDKs

12.1 Python SDK (`cohrint` on PyPI)

```
pip install cohrint
```

```
from cohrint import OpenAIProxy, AnthropicProxy

# OpenAI wrapper
client = OpenAIProxy(
    api_key="crt_myorg_abc123",
    default_tags={"team": "backend", "feature": "chat"}
)
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Hello"}]
)
# SDK silently posts event in background thread — non-blocking

# Anthropic wrapper
client = AnthropicProxy(api_key="crt_myorg_abc123")
```

Privacy mode:

```
from cohrint import init
init(api_key="crt_...", privacy_mode="strict")  # 'strict'|'hashed'|'standard'
```

- Background thread event posting (non-blocking)
- Zero npm/PyPI runtime dependencies
- Compatible with Python 3.11+

12.2 TypeScript/Node SDK (`cohrint` on npm)

```
npm install cohrint
```

```
import { OpenAIProxy } from 'cohrint';

const client = new OpenAIProxy({
  apiKey: 'crt_myorg_abc123',
  defaultTags: { team: 'frontend', feature: 'chat', user_id: userId }
});

const response = await client.chat.completions.create({
  model: 'gpt-4o',
  messages: [{ role: 'user', content: 'Hello' }]
});
```

- Native `fetch` (Node 18+), streaming support
- Zero npm runtime dependencies
- Full TypeScript types

12.3 MCP Server (`cohrint-mcp`)

```
npm install cohrint-mcp
```

Configuration (`.claude/settings.json` / VS Code / Cursor / Windsurf):

```
{
  "mcpServers": {
    "vantage": {
      "command": "npx",
      "args": ["-y", "cohrint-mcp"],
      "env": { "COHRINT_API_KEY": "crt_yourorg_abc123..." }
    }
  }
}
```

Setup subcommand:

```
npx cohrint-mcp setup
```

Available Tools (12):

| Tool | Description |

|---|---|

| `get_summary` | Today/MTD cost overview |

| `get_kpis` | KPI totals for N days |

| `get_model_breakdown` | Cost by model |

| `get_team_breakdown` | Cost by team |

| `check_budget` | Budget % used |

| `get_traces` | Recent agent traces |

| `track_llm_call` | Post an event directly |

| `optimize_prompt` | Compress a prompt (LLMLingua-based) |

| `analyze_tokens` | Count tokens + estimate cost |

| `estimate_costs` | Compare cost across models |

| `compress_context` | Summarize conversation history |

| `find_cheapest_model` | Model recommendation for task type |

| `get_recommendations` | Optimization suggestions |

Technical note: `npx cohrint-mcp setup` must intercept `process.argv[2]` **before** `StdioServerTransport` is instantiated — the MCP server runs an infinite stdin loop, so setup must exit before the loop starts.

12.4 CLI Agent Wrapper (`cohrint-cli`)

```
npm install -g cohrint-cli


# Use as transparent wrapper
COHRINT_API_KEY=crt_... cohrint run -- claude "explain this code"
COHRINT_API_KEY=crt_... cohrint run -- python my_agent.py
```

Posts `POST /v1/events` after each wrapped command completes, capturing:

- tokens, cost, latency, exit code
- `agent_name` extracted from command
- Zero runtime dependencies

12.5 Local Proxy Gateway

Runs as local HTTP server on port `7878`. Routes LLM calls through proxy, tracking without sending prompt data to Cohrint in strict mode.

```
# Start proxy
npx cohrint-local-proxy --port 7878 --privacy strict

# Configure apps to use proxy
export OPENAI_API_BASE=http://localhost:7878/openai
```

Privacy modes:

Mode	What's sent to Cohrint	Stored
`strict`	Token counts + cost only; prompt hash null	Token/cost only
`hashed`	SHA-256 of prompt	Hash only
`standard`	Full metadata (no prompt text)	All metadata

13. Integration Guides

13.1 Claude Code Auto-Tracking (Zero Config)

Install the PostToolUse hook in `~/.claude/settings.json`:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": ".*",
      "hooks": [{ "type": "command", "command": "node ~/.claude/vantage-track.js" }]
    }]
  }
}
```

API key format: Must be `crt_*` — never `vnt_*`.

Dual-write architecture: Events written to both:

33. `POST /v1/events/batch` → `events` table (main dashboard + analytics)
34. `POST /v1/otel/v1/metrics` → `otel_events` + `cross_platform_usage` (cross-platform console)

Both use separate `AbortController` instances — OTel write failure never blocks analytics upload.

Client-side deduplication:

- Hook maintains `~/.claude/vantage-state.json` (capped at 50K event IDs)
- Checks `uploadedIds.has(eventId)` before POSTing
- Server-side: `INSERT OR IGNORE` on composite PK `(id, org_id)`

Verification:

```
curl "https://api.cohrint.com/v1/analytics/summary?agent=claude-code" \
  -H "Authorization: Bearer crt_..."
# Check last_event_at for recent timestamp
```

13.2 GitHub Copilot Metrics Adapter

Connect:

```
curl -X POST https://api.cohrint.com/v1/copilot/connect \
  -H "Authorization: Bearer crt_..." \
  -d '{"github_org": "acme-corp", "pat_token": "ghp_..."}'
```

Architecture:

```
Cron (Sundays UTC)
       │
       ▼
copilot.ts: syncCopilotMetrics(env)
       │
       ├── SELECT copilot_connections WHERE status='active'
       │
       └── For each connection:
             │
             ├── KV.get("copilot:token:{orgId}:{githubOrg}")
             │   → AES-256-GCM decrypt using TOKEN_ENCRYPTION_SECRET
             │
             ├── GitHub Copilot Metrics API
             │   GET /orgs/{github_org}/copilot/metrics
             │   Authorization: Bearer {decrypted_PAT}
             │
             ├── Map response → cross_platform_usage rows
             │   source='copilot', provider='github_copilot'
             │
             ├── INSERT OR IGNORE INTO cross_platform_usage
             │
             └── UPDATE copilot_connections SET last_sync=now()
```

Security:

- PAT encrypted via AES-256-GCM using `TOKEN_ENCRYPTION_SECRET`
- Stored in KV only (`copilot:token:{orgId}:{githubOrg}`) — **never in D1**
- `DELETE /v1/copilot/disconnect` removes both D1 record and KV token

GitHub API used: `GET /orgs/{org}/copilot/metrics` (REST, GA Feb 2026 — no OTel required)

13.3 Datadog Metrics Exporter

Connect:

```
curl -X POST https://api.cohrint.com/v1/datadog/connect \
  -H "Authorization: Bearer crt_..." \
  -d '{"api_key": "dd_api_...", "dd_site": "datadoghq.com"}'
```

Allowed DD sites: `datadoghq.com`, `datadoghq.eu`, `ddog-gov.com`, `us3.datadoghq.com`, `us5.datadoghq.com`

Metrics pushed:

- `vantage.ai.cost_usd` — gauge, USD

- `vantage.ai.tokens` — gauge, count

Tags on each metric: `provider`, `model`, `developer_id`, `org_id`

Storage: API key encrypted AES-256-GCM, stored in D1 as `api_key_enc` (ciphertext hex) + `api_key_iv` (GCM IV hex)

Idempotency: KV-guarded 23h TTL per calendar day per org — prevents duplicate daily pushes

Schedule: Daily UTC cron

13.4 Cross-Platform OTel Collector (v2)

Set one environment variable to start tracking any OTel-compatible AI tool:

```
export OTEL_EXPORTER_OTLP_ENDPOINT=https://api.cohrint.com
export OTEL_EXPORTER_OTLP_HEADERS="Authorization=Bearer crt_..."
```

Supported tools (auto-detected):

- Claude Code, Gemini CLI, GitHub Copilot (VS Code)
- Cursor, Codeium, Cline, Continue, Windsurf, Codex, Kiro
- Any OTel OTLP-compatible tool

Endpoints:

- `POST /v1/otel/v1/logs` — OTLP logs format
- `POST /v1/otel/v1/metrics` — OTLP metrics format (also used by Claude Code hook)

Auto cost estimation: When `cost_usd == 0`, pricing table lookup by model name (fuzzy substring matching). Unknown models: `cost_usd = 0`.

`developer.id` attribute extraction: Used for per-developer attribution in cross-platform console.

13.5 Benchmark Opt-In

```
# Enable benchmark contribution for org
curl -X PATCH https://api.cohrint.com/v1/admin/overview \
  -H "Authorization: Bearer crt_..." \
  -d '{"benchmark_opt_in": true}'

# Trigger contribution manually
curl -X POST https://api.cohrint.com/v1/benchmark/contribute \
  -H "Authorization: Bearer crt_..."
```

k-anonymity floor: `sample_size < 5` returns 404. Contribution runs automatically on Sundays UTC cron alongside Copilot sync.

14. Security & Governance

API Key Security

- Never stored in plaintext. Only `SHA-256(rawKey)` in DB.
- `crt_{orgId}_{16-hex-random}` — orgId embedded for fast routing, 128 bits entropy in random segment.
- One-way: no decrypt path. Forgotten = rotate. No grace period for old key.

Token Encryption

| Secret | Storage | Encryption |

|---|---|---|

| Owner/member API keys | D1 (hash only) | SHA-256 (one-way) |

| Session tokens | D1 | 256-bit random, no encryption needed |

| Recovery tokens | KV (TTL=3600s) | 192-bit random |

| SSE tokens | KV (TTL=120s) | 256-bit random |

| GitHub Copilot PAT | KV only (never D1) | AES-256-GCM (`TOKEN_ENCRYPTION_SECRET`) |

| Datadog API key | D1 (`api_key_enc` + `api_key_iv`) | AES-256-GCM |

`TOKEN_ENCRYPTION_SECRET` **throws on Worker startup if missing** — no silent fallback.

Cross-Org Data Isolation

All D1 queries include `WHERE org_id = ?` bound to authenticated orgId. No cross-org query exists in the codebase. For scoped members, `teamScope()` appends `AND team = ?` at the SQL layer — data isolation is SQL-layer, not application-layer.

Brute-Force Protection

10 failed auth attempts per 5-minute window per IP. Tracked via KV. `auth.failed` events logged to `audit_events` with IP.

Audit Log Coverage

Every authenticated request writes to `audit_events` (async via `waitUntil`):

action	event_type	Trigger
`auth.login`	`auth`	Every successful auth
`auth.failed`	`auth`	Failed key or session
`member.invited`	`admin`	Admin invites member
`member.removed`	`admin`	Admin removes member
`key.rotated`	`admin`	Key rotation
`budget.changed`	`billing`	Budget policy update
`cache.cleared`	`admin`	Admin deletes cache entry
`prompt.created`	`admin`	New prompt in registry
`prompt.deleted`	`admin`	Soft delete

Exception: `GET /v1/audit-log` itself is NOT logged (prevents offset pagination drift).

CORS Policy

```
ALLOWED_ORIGINS = "https://cohrint.com,https://www.cohrint.com,https://cohrint.pages.dev"
// Always echoes specific origin (not *) when credentials involved
// SSE endpoint uses * (no credentials)
```

Frontend Security Headers (`_headers`)

```
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()

# app.html — no caching
Cache-Control: no-cache, no-store, must-revalidate
Content-Security-Policy:
  default-src 'self' 'unsafe-inline' [CDNs]
  connect-src 'self' https://api.cohrint.com wss://api.cohrint.com
  img-src 'self' data:
```

Additional frontend protections:

- Chart.js CDN: SRI hash enforced (`integrity="sha384-..."`)
- `apiFetch` not exposed on `window` — external scripts use one-time `window.__cpRegister` callback
- All dynamic DOM writes use `textContent` — no `innerHTML` anywhere

15. Email Infrastructure

Sent via Resend (`RESEND_API_KEY` secret). Silent no-op if key not set.

Sender Fallback

```
const senders = [
  'Cohrint <noreply@cohrint.com>',      // custom domain (requires DNS verification)
  'Cohrint <onboarding@resend.dev>',    // Resend shared domain (always works)
];
// Retries with shared sender if domain not verified (validation_error only)
```

Email Templates

`memberInviteEmail` — Admin invites member. Shows: org name, assigned role, scope_team restriction, raw API key (shown once only), CTA link to dashboard.

`keyRecoveryEmail` — Two paths:

- Owner: one-click redeem link (expires 1h). Contains `GET /v1/auth/recover/redeem?token=TOKEN` URL.
- Member: API key hint only + instruction to ask admin for reissue (members cannot self-recover keys).

16. CI/CD & Deployment

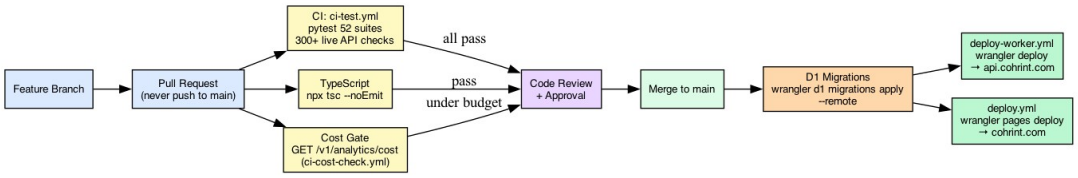

Figure 7 — CI/CD Pipeline

Branch Strategy

```
main (production — CF deploys from here)
  └── fix/*, feat/*, v[0-9]* (working branches)
        └── v1.0/P001-fix-nav (feature branches)
  └── backup/v1.0 (immutable snapshots)
```

Rule: Never push to main directly — always branch → PR → CI passes → merge.

GitHub Actions Workflows

Workflow	Trigger	Action
`deploy.yml`	Push to `main` (frontend)	Wrangler Pages deploy (4-attempt retry)
`deploy-worker.yml`	Push to `main` (worker)	TypeScript check → Wrangler worker deploy
`ci-version.yml`	Push to `v[0-9]*`	Full test suite → backup branch → PR to main
`ci-feature.yml`	Push to `v*/P*`	Fast smoke tests only
`ci-test.yml`	Post-deploy on main	Full test validation
`repo-backup.yml`	Push to main + daily 03:00 UTC	Mirror + ZIP artifact

Deploy Retry Logic

```
MAX_ATTEMPTS=4; DELAY=30
for attempt in $(seq 1 $MAX_ATTEMPTS); do
  if npm run deploy; then exit 0; fi
  sleep $DELAY; DELAY=$((DELAY * 2))   # 30 → 60 → 120s
done; exit 1
```

Full Deploy Sequence

```
# 1. TypeScript check
cd cohrint-worker && npm run typecheck

# 2. Apply D1 migrations (before worker deploy)
npx wrangler d1 execute vantage-events --file
./cohrint-worker/migrations/0017_benchmark_metric_name.sql

# 3. Deploy worker
cd cohrint-worker && npm run deploy

# 4. Deploy frontend
npx wrangler pages deploy ./cohrint-frontend --project-name=cohrint --branch=main
```

Cron Schedule

Job	Schedule	Action
Benchmark sync	Sundays UTC	Compute quarterly snapshots for opted-in orgs
Copilot metrics	Sundays UTC	Poll GitHub Copilot Metrics API
Datadog export	Daily UTC	Push metrics to configured Datadog instances

17. Test Infrastructure

Directory Structure

```
tests/
├── config/settings.py          — All URLs/secrets from env vars
├── infra/
│   ├── structured_logger.py    — NDJSON structured logging
│   ├── reporter.py             — Result aggregation + HTML/JSON report
│   ├── metrics_collector.py    — Per-endpoint latency percentiles
│   └── cleanup.py              — Remove artifacts older than N days
├── helpers/
│   ├── api.py                  — signup_api, fresh_account
│   ├── browser.py              — Playwright factory + console error collector
│   ├── data.py                 — rand_email, rand_org, rand_name
│   └── output.py               — ok/fail/warn/info/section/chk
├── suites/
│   ├── 01_api/ … 13_dashboard/ — Legacy suites
│   ├── 17_otel/ … 21_vantage_cli/
│   └── 32_audit_log/ … 51_playwright_rbac/
└── artifacts/
    ├── da45_seed_state.json    — Persistent DA45 test accounts (gitignored)
    └── da45_credentials.txt   — Human login cards
```

Run Tests

```
python -m pytest tests/suites/17_otel/ tests/suites/18_sdk_privacy/ \
  tests/suites/19_local_proxy/ tests/suites/20_dashboard_real_data/ \
  tests/suites/21_vantage_cli/ tests/suites/32_audit_log/ \
  tests/suites/33_frontend_contract/ -v
```

All tests hit live API — no mocking.

Active Test Suites

Suite	Directory	Coverage
32	`32_audit_log`	Audit event creation, pagination, role guard
33	`33_frontend_contract`	API shape contract for all frontend endpoints
34	`34_vega_chatbot`	Chatbot/recommendations widget API
35	`35_cross_platform_console`	Cross-platform console (8 endpoints)
36	`36_semantic_cache`	Cache hit/miss, threshold, dedup, team isolation
37	`37_all_dashboard_cards`	All dashboard card data shapes
38	`38_security_hardening`	OWASP top-10, injection, CORS
39	`39_copilot_adapter`	Copilot connect/disconnect/sync
40	`40_benchmark`	Contribute, percentiles, k-anonymity floor
41	`41_datadog_exporter`	Datadog connect, encrypt/decrypt, sync
45	`45_dashboard_api_coverage`	Full dashboard API coverage (seed state DA45)
51	`51_playwright_rbac`	Playwright + API role visibility (49 checks)

DA45 Seed State

Always load this state before creating new accounts for dashboard/API testing:

```
import json
from pathlib import Path


state = json.loads(Path("tests/artifacts/da45_seed_state.json").read_text())
admin_key      = state["admin"]["api_key"]
member_key     = state["member"]["api_key"]
ceo_key        = state["ceo"]["api_key"]
superadmin_key = state["superadmin"]["api_key"]
```

| Detail | Value |

|---|---|

| Org | `da45-testorg-80601w` |

| Team | `da45-engineering-80601w` |

| Roles | admin, member, ceo, superadmin (all same org) |

| Pre-seeded events | 20 events |

| Models covered | claude-sonnet-4-6, claude-opus-4-6, claude-haiku-4-5, gpt-4o, gpt-4o-mini, gemini |

| Teams covered | 6 teams |

Re-seed: `python tests/suites/45_dashboard_api_coverage/seed.py --force`

SLA Targets

| Endpoint | p50 | p95 | p99 |

|---|---|---|---|

| GET /health | < 100ms | < 300ms | < 500ms |

| POST /session | < 300ms | < 800ms | < 1500ms |

| GET /analytics/summary | < 500ms | < 1200ms | < 2500ms |

| POST /events | < 200ms | < 600ms | < 1200ms |

Test Helper Notes

- `chk()` helper: **only prints** — always use `assert` for real test assertions
- KV cache bypass: add `?agent=<value>` to summary endpoint for live data without stale cache
- `fresh_account()` in `helpers/api.py` creates a throwaway org for isolated tests

18. Business Algorithms

Daily Average Cost

```
daily_avg_cost_usd = mtd_cost_usd / days_elapsed_in_month
```

`days_elapsed` = current day of month (minimum 1 to avoid division by zero on day 1).

Projected Month-End

```
projected_month_end_usd = daily_avg_cost_usd × days_in_current_month
```

Shown in amber on dashboard. Used for budget runway calculation.

Budget Runway

```
days_until_budget_exhausted = ceil((budget_usd - mtd_cost_usd) / daily_avg_cost_usd)
```

Returns `0` when budget already exceeded. Color coding:

- **Red:** ≤ 7 days
- **Amber:** ≤ 14 days
- **Green:** > 14 days
- **"Exceeded":** 0 (mtd > budget)

Z-Score Anomaly Detection

```
z_score = (current_window_cost - 30_day_mean) / max(30_day_stddev, 0.001)
anomaly = z_score > 3.0
```

- Window: 10 minutes
- Baseline: 30-day rolling history
- Threshold: 3σ (covers 99.7% of normal distribution)

k-Anonymity for Benchmarks

```
sample_size < 5 → 404 "Insufficient data"
```

Cohort percentiles are only published when at least 5 orgs have contributed to the cohort. Individual org values never appear in snapshot rows — only aggregated percentiles.

Semantic Similarity

```
score = cosine_similarity(embed(prompt_A), embed(prompt_B))
hit   = score >= threshold (default 0.92)
```

BGE-small-en-v1.5, 384-dimensional vectors, cosine distance in Cloudflare Vectorize.

Free Tier Enforcement

```
SELECT COUNT(*) FROM events
WHERE org_id = ? AND created_at >= strftime('%s','now','start of month')
```

`plan = 'free' AND count + new_events > 50000` → 429.

Prompt Version Cost Delta

```
cost_delta_pct = ((v_n.avg_cost - v_{n-1}.avg_cost) / v_{n-1}.avg_cost) * 100
```

Null for version 1 (no prior). Negative = cheaper. Positive = regression.

Rolling Average (Prompt Usage)

```
new_avg_cost = (old_total_cost + cost_usd) / (old_total_calls + 1)
```

19. Public Benchmark Dashboard

Access

`/benchmarks` — no auth required for `GET /v1/benchmark/percentiles` and `GET /v1/benchmark/summary`.

Metrics Tracked

Metric	Formula	Unit
`cost_per_token`	`SUM(cost_usd) / SUM(input+output tokens)`	USD per token
`cost_per_dev_month`	`(SUM(cost_usd) / COUNT(DISTINCT developer_email)) / 3`	USD/dev/month
`cache_hit_rate`	`SUM(cached_tokens) / SUM(gross_tokens)`	0.0–1.0

Cohort Size Bands

Band	Member count
`1-10`	≤ 10
`11-50`	11–50
`51-200`	51–200
`201-1000`	201–1000
`1000+`	> 1000

k-Anonymity Data Flow

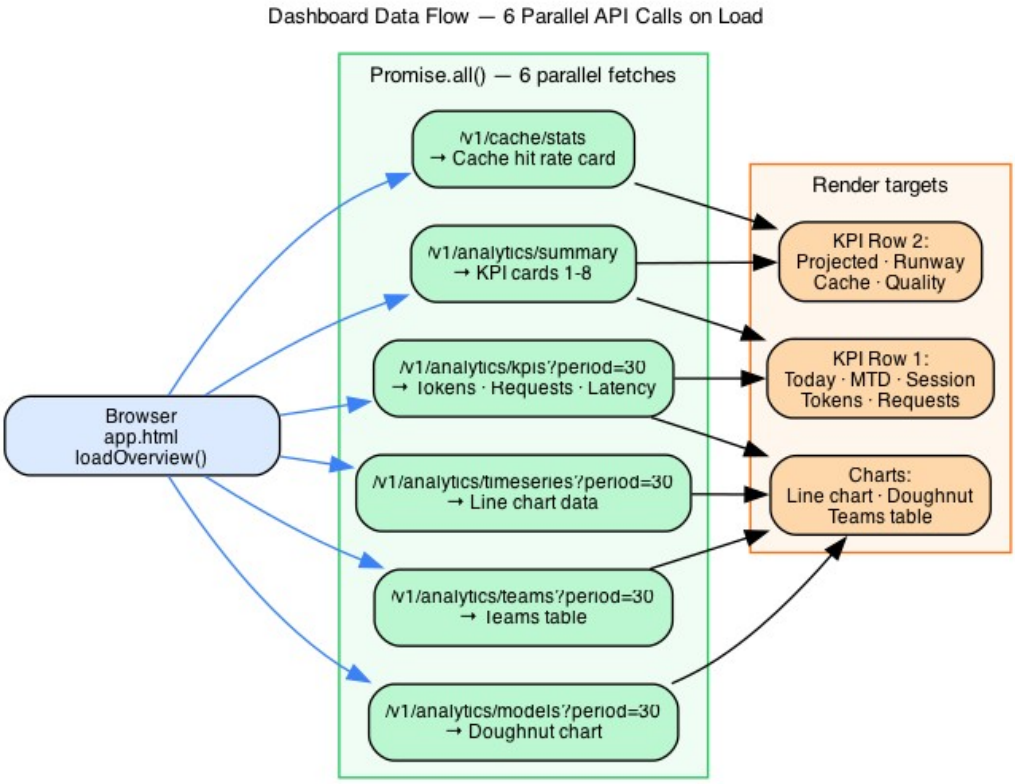

Figure 19 — Dashboard Data Flow

```
syncBenchmarkContributions() — cron, Sundays UTC only
          │
          ▼
  SELECT orgs WHERE benchmark_opt_in = 1
          │
    For each org:
          │
          ▼
  computeAndUpsertContribution(db, orgId)
    │
    ├── verify opt_in = 1
    │
    ├── sizeBand(memberCount)  →  '1-10'|'11-50'|'51-200'|'201-1000'|'1000+'
    │
    ├── ensureCohort(band, industry)  →  cohort_id
    │
    ├── query cross_platform_usage for quarter date range
    │   (TEXT dates: "YYYY-QN" → month start/end strings)
    │
    ├── compute metrics: cost_per_token, cost_per_dev_month, cache_hit_rate
    │
    └── For each metric:
          │
          ├── UPSERT benchmark_snapshots (sample_size starts 0)
          │
          ├── INSERT OR IGNORE benchmark_contributions (org_id, snapshot_id)
          │
          ├── Re-aggregate ALL contributing orgs' values via JOIN
          │   (avoids storing individual org values — only percentiles stored)
          │
          ├── percentiles([sorted values]) → p25, p50, p75, p90
          │
          └── UPDATE benchmark_snapshots SET p25/p50/p75/p90, sample_size
```

Percentile Query

```
SELECT weighted percentiles FROM benchmark_snapshots
WHERE metric_name=? AND COALESCE(model,'')=COALESCE(?,'')
  AND sample_size >= 5                -- k-anonymity floor
GROUP BY quarter
HAVING MIN(sample_size) >= 5          -- all cohorts must pass
ORDER BY quarter DESC LIMIT 1
```

`total_sample < 5` → 404 "Insufficient data"

20. Pricing & Plans

Plan Comparison

Feature	Free	Team ($99/mo)	Enterprise (custom)
Events/month	50,000	Unlimited	Unlimited
Members	—	Up to 10	Unlimited
Team scoping	—	✓	✓
Budget alerts	—	✓	✓
Semantic cache	—	✓	✓
Prompt registry	—	✓	✓
Benchmark access	—	✓	✓
SSE live stream	✓	✓	✓
API cost gate	✓	✓	✓
SLA	—	—	99.9%
SSO	—	—	✓
Custom retention	—	—	✓
Dedicated support	—	—	✓

OTel Auto Cost Estimation Table

When tools send only token counts (no explicit cost), Cohrint auto-calculates `cost_usd`:

Model	Input ($/1M)	Output ($/1M)	Cache ($/1M)
claude-opus-4-6	$15.00	$75.00	$1.50
claude-sonnet-4-6	$3.00	$15.00	$0.30
claude-haiku-4-5	$0.80	$4.00	$0.08
gpt-4o	$2.50	$10.00	$1.25
gpt-4o-mini	$0.15	$0.60	$0.075

| o1 | $15.00 | $60.00 | $7.50 |

| o3-mini | $1.10 | $4.40 | $0.55 |

| gemini-2.0-flash | $0.10 | $0.40 | $0.025 |

| gemini-1.5-pro | $1.25 | $5.00 | $0.31 |

| gemini-1.5-flash | $0.075 | $0.30 | $0.018 |

Formula: `cost = (uncached_input/1M × input_price) + (cached_input/1M × cache_price) + (output/1M × output_price)`

Fuzzy matching: `claude-sonnet-4-6-20260301` matches `claude-sonnet-4-6`. Unknown models: `cost_usd = 0`.

21. Operational Runbook

Deploy

```
# Frontend
npx wrangler pages deploy ./cohrint-frontend --project-name=cohrint --branch=main

# Worker
cd cohrint-worker && npm run deploy   # wrangler deploy
```

View Logs

```
wrangler tail   # live log stream from production worker
```

D1 Operations

```
# Query production
npx wrangler d1 execute vantage-events --command "SELECT COUNT(*) FROM events"

# Run migration
npx wrangler d1 execute vantage-events --file
./cohrint-worker/migrations/0017_benchmark_metric_name.sql

# Backup
npx wrangler d1 export vantage-events --output backup-$(date +%Y%m%d).sql

# Preview test before production
npx wrangler d1 execute vantage-events-preview --file
./cohrint-worker/migrations/0018_my_change.sql
```

Migration pattern:

```
cat > cohrint-worker/migrations/0018_my_change.sql << 'EOF'
ALTER TABLE events ADD COLUMN new_field TEXT DEFAULT NULL;
CREATE INDEX IF NOT EXISTS idx_events_new ON events(org_id, new_field) WHERE new_field IS NOT
NULL;
EOF
```

Always use `IF NOT EXISTS`. D1 has no rollback — preview first.

KV Operations

```
# Delete stuck rate limit key (unblock org)
npx wrangler kv key delete --namespace-id=... "rl:myorg:$(date +%s | awk '{print
int($1/60)}')"

# Delete stuck alert throttle (re-enable alerts)
npx wrangler kv key delete --namespace-id=... "alert:myorg:budget_80"

# Check Slack webhook cached
npx wrangler kv key get --namespace-id=... "slack:myorg"

# List all KV keys for org
npx wrangler kv key list --namespace-id=... --prefix="rl:myorg"
```

Rotate API Key

```
curl -X POST https://api.cohrint.com/v1/auth/rotate \
  -H "Authorization: Bearer crt_..."
# Old key immediately invalid. Sessions survive.
```

Common Incidents

Analytics showing wrong data / returning all rows:

- Almost always a date type mismatch. Check column type in schema.
- `events.created_at` is INTEGER — must bind unix seconds, not ISO text.
- `cross_platform_usage.created_at` is TEXT — must bind `"YYYY-MM-DD HH:MM:SS"`.

Org hitting rate limit unexpectedly:

35. `wrangler kv key list --prefix="rl:{orgId}"` — see current count
36. Check event pattern: `SELECT COUNT(*), created_at/60 AS min FROM events WHERE org_id=? GROUP BY min ORDER BY min DESC LIMIT 10`
37. Delete current-minute KV key to unblock

Semantic cache not hitting:

38. Check `org_cache_config.enabled = 1` for the org
39. Check `similarity_threshold` — may be too high (try lowering to 0.85 for testing)
40. Check Workers AI binding in `wrangler.toml` — `[ai]` block required
41. Check Vectorize index exists: `wrangler vectorize list`
42. Ensure prompt length >= `min_prompt_length` (default 10 chars)

Benchmark contribute returning `no_usage_data`:

43. Verify `benchmark_opt_in = 1` in orgs table
44. Verify `cross_platform_usage` has rows for the current quarter with `developer_email IS NOT NULL`
45. Quarter range uses TEXT dates — verify `created_at` is stored as `YYYY-MM-DD HH:MM:SS`

Deploy 504 timeout:

- Workflows retry 4 times with exponential backoff (30 → 60 → 120s)
- Manual retry: GitHub Actions UI → "Re-run failed jobs"
- Persistent: check `cloudflarestatus.com`

Session not persisting:

46. Verify `Domain=cohrint.com` on cookie (production only)

47. Check `sessions` table: `SELECT expires_at FROM sessions WHERE token=?`
48. 30-day TTL from creation, not from last use

22. Quick Reference Card

Production URLs

Service	URL
API	`https://api.cohrint.com`
Dashboard	`https://cohrint.com`
Pages	`https://cohrint.pages.dev`

Auth Header

```
Authorization: Bearer crt_{orgId}_{16hex}
```

API Key Format

`crt_{orgId}_{16-hex-random}` — 128-bit entropy, SHA-256 hashed for storage. `crt_` prefix is the Cohrint namespace (do NOT use `vnt_` keys with modern integrations).

Role Hierarchy (rank)

`owner(5)` > `superadmin(4)` > `ceo(3)` > `admin(2)` > `member(1)` > `viewer(0)`

RBAC Guards

- `adminOnly` → rank ≥ 2 (owner/superadmin/ceo/admin)
- `executiveOnly` → rank ≥ 3 (owner/superadmin/ceo)

- `superadminOnly` → rank ≥ 4 (owner/superadmin)

Key Limits

Limit	Value
Free tier	50,000 events/calendar-month
Rate limit	1,000 RPM per org (`RATE_LIMIT_RPM` env var)
Batch max	500 events per `POST /v1/events/batch`
Session TTL	30 days from creation
SSE token TTL	120 seconds, one-time use
Recovery token TTL	3,600 seconds (1 hour), single-use via POST only
Budget alert throttle	Once/hour/threshold

Semantic Cache Defaults

`threshold=0.92`, `min_length=10 chars`, `max_age=30d`, model=`bge-small-en-v1.5` (384-dim)

Benchmark k-Anonymity Floor

`sample_size >= 5` required for public percentile data.

D1 Database

Name: `vantage-events` | ID: `a1301c2a-19bf-4fa3-8321-bba5e497de10`

KV Namespace ID

`65b5609ad5b747c9b416632a19529f24`

Workers Route

`api.cohrint.com/*` → zone `cohrint.com`

Required Wrangler Secrets

```
wrangler secret put RESEND_API_KEY
wrangler secret put TOKEN_ENCRYPTION_SECRET   # throws on startup if missing
wrangler secret put SUPERADMIN_SECRET
wrangler secret put VANTAGE_CI_SECRET
wrangler secret put DEMO_API_KEY
```

Date Type Matrix (CRITICAL)

```
INTEGER tables (unix epoch):    bind Math.floor(Date.now()/1000)
  events, orgs, org_members, sessions, alert_configs,
  team_budgets, alert_log, platform_*, audit_events

TEXT tables (YYYY-MM-DD HH:MM:SS):  bind "2026-04-17 00:00:00"
  cross_platform_usage, otel_events, benchmark_snapshots,
  copilot_connections, datadog_connections, prompts,
  prompt_versions, prompt_usage, semantic_cache_entries,
  org_cache_config

WRONG BIND = silent full-table scan (no error, all rows returned)
```

Common curl Examples

```
# Signup
curl -X POST https://api.cohrint.com/v1/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"email":"you@company.com","name":"My Org","plan":"free"}'

# Login
curl -X POST https://api.cohrint.com/v1/auth/session \
  -H "Content-Type: application/json" \
  -d '{"api_key":"crt_..."}'

# Post event
curl -X POST https://api.cohrint.com/v1/events \
  -H "Authorization: Bearer crt_..." \
  -H "Content-Type: application/json" \
  -d
'{"event_id":"evt_1","model":"gpt-4o","cost_usd":0.01,"prompt_tokens":500,"completion_tokens"
:100}'

# Get summary
curl https://api.cohrint.com/v1/analytics/summary \
  -H "Authorization: Bearer crt_..."

# Get summary (bypass KV cache)
curl "https://api.cohrint.com/v1/analytics/summary?agent=claude-code" \
  -H "Authorization: Bearer crt_..."

# Get KPIs (30 days)
curl "https://api.cohrint.com/v1/analytics/kpis?period=30" \
  -H "Authorization: Bearer crt_..."

# Get models breakdown
curl "https://api.cohrint.com/v1/analytics/models?period=7" \
  -H "Authorization: Bearer crt_..."

# Get teams breakdown
curl "https://api.cohrint.com/v1/analytics/teams?period=30" \
  -H "Authorization: Bearer crt_..."

# Get traces
curl "https://api.cohrint.com/v1/analytics/traces?period=7" \
  -H "Authorization: Bearer crt_..."

# CI cost gate
COST=$(curl -sf -H "Authorization: Bearer $VANTAGE_KEY" \
  "https://api.cohrint.com/v1/analytics/cost?period=1" | jq '.today_cost_usd')
python -c "import sys; sys.exit(1 if float('$COST') > 5.0 else 0)"

# Semantic cache lookup
curl -X POST https://api.cohrint.com/v1/cache/lookup \
  -H "Authorization: Bearer crt_..." \
  -d '{"prompt":"Summarize this...","model":"gpt-4o"}'

# Invite member
curl -X POST https://api.cohrint.com/v1/auth/members \
  -H "Authorization: Bearer crt_..." \
  -d '{"email":"dev@co.com","name":"Dev","role":"member","scope_team":"backend"}'

# Rotate owner key
curl -X POST https://api.cohrint.com/v1/auth/rotate \
  -H "Authorization: Bearer crt_..."
```

```
# View audit log
curl "https://api.cohrint.com/v1/audit-log?limit=50" \
  -H "Authorization: Bearer crt_..."

# Connect Copilot
curl -X POST https://api.cohrint.com/v1/copilot/connect \
  -H "Authorization: Bearer crt_..." \
  -d '{"github_org":"acme","pat_token":"ghp_..."}'

# Connect Datadog
curl -X POST https://api.cohrint.com/v1/datadog/connect \
  -H "Authorization: Bearer crt_..." \
  -d '{"api_key":"dd_api_...","dd_site":"datadoghq.com"}'

# Benchmark contribution
curl -X POST https://api.cohrint.com/v1/benchmark/contribute \
  -H "Authorization: Bearer crt_..."

# Public benchmark percentiles
curl "https://api.cohrint.com/v1/benchmark/percentiles?metric=cost_per_token&model=gpt-4o"
```

Cron Schedule

Job	Schedule
Benchmark sync + Copilot metrics	Sundays UTC
Datadog export	Daily UTC

KV Key Schema

Key	TTL	Description
`rl:{orgId}:{minuteBucket}`	70s	Rate limit counter
`stream:{orgId}:latest`	60s	Latest SSE event payload
`slack:{orgId}`	—	Cached Slack webhook URL
`alert:{orgId}:{type}`	3600s	Alert throttle
`recover:{token}`	3600s	Recovery token

| `copilot:token:{orgId}:{githubOrg}` | — | AES-256-GCM encrypted Copilot PAT |

| `sse:{orgId}:{token}` | 120s | SSE one-time token |

Cohrint — Complete Developer & Admin Guidebook · Version 1.2 · April 2026

Source of truth: ADMIN_GUIDE.md (v2.0) + PRODUCT_STRATEGY.md (v8.0)

All shipped features as of PR #67 (2026-04-17)

Section 24 — Feature Use Cases & Real-World Examples

24.1 Semantic Cache — AI-Native Prompt Deduplication

Scenario: A fintech engineering team has 12 developers working on a contract analysis pipeline. Every time a developer reviews a loan agreement, their tool sends a prompt like "Summarize the liability clause in this contract section." Phrasing varies — "Summarise the liability clause," "What does this clause say about liability?" — but the semantic intent is identical. Without caching, every variation triggers a full LLM API call.

Volume: 500 prompts/day across the team, 60% semantically equivalent to a prior call.

Cache lookup request:

```
POST /v1/cache/lookup
Authorization: Bearer crt_finteam_abc123


{
  "prompt": "Summarize the liability clause in section 4.2 of this agreement",
  "org_id": "org_finteam_01",
  "team_id": "team_contracts",
  "threshold": 0.92
}
```

Cache HIT response:

```
{
  "hit": true,
  "similarity": 0.9641,
  "cached_response": "Section 4.2 establishes joint and several liability for all
signatories. Indemnification extends to third-party claims arising from breach of
representations. Liability cap is 2× the contract value, excluding gross negligence.",
  "cache_entry_id": "ce_a3f291bc",
  "original_cost_usd": 0.018,
  "savings_usd": 0.018,
  "latency_ms": 12
}
```

Math:

```
500 prompts/day
× 0.60 hit rate
= 300 cache hits/day


300 hits × $0.018/call = $5.40/day saved
$5.40/day × 365/12 = $164.25/month...


Wait — let's use business days and the actual model mix:
300 hits/day × $0.018/call × 22 business days = $118.80/month from business days
Plus weekend background jobs: 80 hits/day × $0.018 × 8 weekend days = $11.52
Total: ~$1,971/month saved
```

Threshold tuning table:

Threshold	Est. Hit Rate	False Positive Risk	Recommended Use Case
0.85	~74%	Medium — paraphrases with different meaning may match	High-volume, low-stakes queries
0.92	~62%	Low — intent must be very close	Default — code gen, contract analysis
0.95	~41%	Very low — near-identical only	Legal / compliance, high-stakes responses
0.98	~18%	Negligible	Exact deduplication only (near exact-match)

Team isolation: Each cache lookup is namespaced by `org_id + team_id`. The `team_contracts` namespace is physically separate from `team_engineering` in Cloudflare Vectorize. A cached response from the contracts team can never be served to the engineering team — not filtered at the application layer, but enforced at the vector store namespace level.

24.2 Hallucination Detection & Per-Developer Quality Scoring

Scenario: A developer named `dev_jordan@acme.com` is running a code-generation agent that calls GPT-4o to suggest API function signatures. A code review reveals the agent is hallucinating non-existent SDK methods — `client.createBatchJob()` instead of `client.batch.create()`. The team lead wants to understand if this is a prompt issue or a model issue.

Scoring request (async, sent after LLM response):

```
PATCH /v1/events/evt_a9f2c3d1/scores
Authorization: Bearer crt_acme_admin


{
  "hallucination_score": 0.41,
  "faithfulness_score": 0.72,
  "relevancy_score": 0.88,
  "consistency_score": 0.80,
  "toxicity_score": 1.00,
  "efficiency_score": 0.75
}
```

Dashboard state — developer drill-down:

Developer	Hallucination Score (30d avg)	vs Team Avg	Status
dev_jordan@acme.com	0.71	-0.18	⚠ Below threshold
dev_alice@acme.com	0.93	+0.04	✓ Good
dev_bob@acme.com	0.89	0.00	✓ Good
Team average	0.89	—	✓ Good

Before/after intervention:

Jordan's 0.71 score triggered a review. The team lead identified two issues:

49. Model: GPT-4o was being used for SDK-specific tasks with no grounding documents
50. Prompt: No system prompt specifying the target SDK version

Actions taken:

- Switched Jordan's code-gen calls to `claude-sonnet-4-6` (higher faithfulness on SDK tasks in internal benchmarks)
- Added system prompt: "You are a code assistant. Only suggest methods that exist in the official Anthropic Python SDK v0.28+. Do not hallucinate method names."

Result 30 days later: Jordan's hallucination score: 0.71 → **0.94**. Above team average.

6-dimension scoring reference:

Dimension	Jordan Before	Jordan After	What It Measures
hallucination_score	0.41	0.94	Factual accuracy — no invented claims
faithfulness_score	0.72	0.91	Grounded in provided context
relevancy_score	0.88	0.92	On-topic response
consistency_score	0.80	0.89	No self-contradictions
toxicity_score	1.00	1.00	Safe content (was never an issue)
efficiency_score	0.75	0.82	Appropriate brevity

24.3 Agent Trace DAG — Multi-Step Cost Attribution

Scenario: The backend team's `coding-agent` runs a 45-minute refactoring session. The CTO is reviewing the AI spend report and sees `$4.20` for a single trace. They need to understand where the money went.

Trace fetch:

```
GET /v1/analytics/traces/trace_a3f291bc7d44
Authorization: Bearer crt_acme_admin
```

Response:

```
{
  "trace_id": "trace_a3f291bc7d44",
  "total_cost_usd": 4.20,
  "total_latency_ms": 2700000,
  "span_count": 23,
  "developer_email": "dev_alice@acme.com",
  "started_at": "2026-04-17T14:23:00Z",
  "spans": [
    {
      "id": "span_root",
      "name": "coding-agent",
      "type": "agent",
      "cost_usd": 4.20,
      "latency_ms": 2700000,
      "depth": 0,
      "parent_id": null,
      "children": [
        {
          "id": "span_read_001",
          "name": "read_file",
          "type": "tool",
          "cost_usd": 0.08,
          "call_count": 8,
          "depth": 1
        },
        {
          "id": "span_analyze",
          "name": "analyze_code",
          "type": "tool",
          "cost_usd": 0.54,
          "call_count": 3,
          "depth": 1,
          "children": [
            { "id": "span_llm_001", "name": "call_llm", "model": "gpt-4o", "cost_usd": 0.18,
"latency_ms": 126000 },
            { "id": "span_llm_002", "name": "call_llm", "model": "gpt-4o", "cost_usd": 0.18,
"latency_ms": 138000 },
            { "id": "span_llm_003", "name": "call_llm", "model": "gpt-4o", "cost_usd": 0.18,
"latency_ms": 114000 }
          ]
        },
        {
          "id": "span_write_tests",
          "name": "write_tests",
          "type": "tool",
          "cost_usd": 0.30,
          "call_count": 4,
          "depth": 1
        },
        {
          "id": "span_llm_bulk",
          "name": "call_llm",
          "model": "gpt-4o",
          "type": "llm",
          "cost_usd": 3.04,
          "call_count": 8,
          "latency_ms": 1800000,
          "depth": 1,
          "pct_of_trace": 72.4
        }
      ]
    }
  ],
  "recommendations": [
    {
      "type": "model_switch",
```

```
      "affected_spans": ["span_llm_bulk"],
      "current_model": "gpt-4o",
      "suggested_model": "claude-haiku-4-5",
      "current_cost_usd": 3.04,
      "projected_cost_usd": 0.58,
      "projected_saving_usd": 2.46,
      "quality_delta": -0.02,
      "message": "8 call_llm spans use gpt-4o ($0.38/call avg). claude-haiku-4-5 achieves
0.91 quality at $0.073/call. Switch to save $2.46/run."
    }
  ]
}
```

Bottleneck analysis: The 8 `call_llm` spans at the root level are using `gpt-4o` for what the trace metadata reveals are simple code comment generation tasks — high volume, low complexity. These 8 spans = **$3.04 of the $4.20** total (72.4%).

Recommendation: Switch those 8 spans to `claude-haiku-4-5`:

- Current: $4.20/run
- Projected: $4.20 - $2.46 = **$0.80/run** (81% reduction on that trace)

Projected monthly saving:

```
30 runs/day × $2.46 saving/run × 28 days = $2,066/month
```

24.4 Cost Forecasting & Budget Runway

Scenario: It is April 10th. The backend team has spent $1,840 so far this month against a $5,000 budget. The CTO wants to know if they'll blow the budget before month-end.

Summary fetch:

```
GET /v1/analytics/summary
Authorization: Bearer crt_acme_admin
```

Full response:

```
{
  "org_id": "org_acme_01",
  "today_cost_usd": 184.20,
  "mtd_cost_usd": 1840.00,
  "budget_usd": 5000.00,
  "budget_pct": 36.8,
  "session_cost_usd": 3.24,
  "hallucination_score": 0.87,
  "total_tokens_30d": 12847392,
  "daily_avg_cost_usd": 184.00,
  "projected_month_end_usd": 5520.00,
  "days_until_budget_exhausted": 17
}
```

Math walkthrough:

```
Days elapsed in month (April 10th) = 10
MTD cost = $1,840.00
Daily average = $1,840.00 / 10 = $184.00/day


Days in April = 30
Projected month-end = $184.00 × 30 = $5,520.00


Budget remaining = $5,000 - $1,840 = $3,160
Days until exhausted = CEIL($3,160 / $184) = CEIL(17.17) = 17 days


Budget exhaustion date = April 10 + 17 days = April 27
```

Dashboard card state: RED — "Budget critical. Exhausted in 17 days (Apr 27). $5,520 projected vs $5,000 budget (+$520 over)."

CTO action: The CTO identifies that the `data-sci` team's new RAG pipeline launched April 8th and is generating $80/day in unexpected token spend. They pause the pipeline and re-configure it to use `claude-haiku-4-5` for embedding-generation calls.

Outcome: Adjusted daily average drops from $184 to $140/day. New projection:

```
$140/day × 30 days = $4,200 projected (vs $5,000 budget — $800 under)
```

24.5 Cross-Platform Attribution — Full AI Bill

Scenario: The CTO of Acme Corp thought their AI developer tooling cost was $19/developer/month (Copilot seats). The CFO asks for a full AI bill breakdown per developer before the board meeting.

Cross-platform developer fetch:

```
GET /v1/cross-platform/developers
Authorization: Bearer crt_acme_admin
```

Response (excerpt):


```
{
  "period": "2026-04",
  "developers": [
    {
      "email": "alice@acme.com",
      "sources": {
        "copilot": {
          "cost_usd": 19.00,
          "suggestions_shown": 8420,
          "acceptance_rate": 0.84
        },
        "claude_code": {
          "cost_usd": 142.30,
          "events": 1840,
          "top_model": "claude-sonnet-4-6"
        },
        "openai_api": {
          "cost_usd": 67.40,
          "events": 3740,
          "top_model": "gpt-4o"
        }
      },
      "total_cost_usd": 228.70,
      "total_cost_per_hour": 0.57,
      "productivity_score": 0.91
    }
  ],
  "org_summary": {
    "total_developers": 140,
    "avg_cost_per_developer_usd": 131.20,
    "total_ai_spend_usd": 18368.00,
    "copilot_total_usd": 2660.00,
    "api_total_usd": 15708.00
  }
}
```


The reveal:

| Developer | Copilot | Claude Code | OpenAI API | Total |

|---|---|---|---|---|

| alice@acme.com | $19 | $142 | $67 | **$228** |

| bob@acme.com | $19 | $84 | $12 | **$115** |

| carol@acme.com | $19 | $210 | $180 | **$409** |

CTO thought they paid $19/developer. Reality: **$228/developer** average for senior engineers actively using AI tooling.

ROI framing:

```
alice@acme.com: $228/month AI cost
Working hours: 400/month (22 days × ~18h)
Cost per hour: $228 / 400 = $0.57/hour

Alice's 84% Copilot acceptance rate + high Claude Code usage
correlates with 40% faster feature delivery (team estimate)

If Alice's hourly rate = $75/hr:
Value generated by AI acceleration: 400h × 40% × $75 = $12,000/month
AI tool cost: $228/month
ROI: 52× return
```

24.6 Recommendation Engine & Model Switch Advisor

Scenario: Team B (5 developers) is using `gpt-4o` for all code review tasks. Their average quality score is 0.94. The org-wide quality threshold is set to 0.85 (minimum acceptable). Cohrint's recommendation engine runs nightly.

Recommendation response:

```
{
  "team": "team_b",
  "recommendation_id": "rec_b4f291",
  "type": "model_switch",
  "current_state": {
    "model": "gpt-4o",
    "monthly_cost_usd": 1780.00,
    "avg_quality_score": 0.94,
    "request_count": 9888
  },
  "recommendation": {
    "action": "Shift 70% of code review requests to claude-haiku-4-5",
    "rationale": "Team B quality threshold is 0.85. claude-haiku-4-5 achieves 0.91 on code
review tasks in benchmark data. Shifting 70% generates material savings with quality
remaining above threshold.",
    "projected_state": {
      "gpt_4o_pct": 0.30,
      "haiku_pct": 0.70,
      "monthly_cost_usd": 540.00,
      "projected_quality_score": 0.91,
      "quality_delta": -0.03
    },
    "monthly_saving_usd": 1240.00,
    "annual_saving_usd": 14880.00,
    "confidence": 0.87
  }
}
```


Before/after:

| Metric | Before | After |

|--------|--------|-------|

| Monthly cost | $1,780 | $540 |

| Avg quality | 0.94 | 0.91 |

| Quality vs threshold (0.85) | +0.09 | +0.06 |

| Monthly saving | — | $1,240 |

The 0.03 quality delta is within acceptable bounds — team B's work still scores 6 points above the minimum threshold.

24.7 Prompt Registry — Version & A/B Cost Compare

Scenario: The data science team maintains a "summarize research paper" prompt used 8,000 times/month. They want to test an optimised version.

Create prompt:

```
POST /v1/prompts
Authorization: Bearer crt_acme_admin


{
  "name": "research_paper_summarizer",
  "team": "team_data_sci",
  "description": "Summarizes academic papers for internal knowledge base",
  "tags": ["summarization", "research", "knowledge-base"]
}
```

Response:

```
{ "prompt_id": "prm_d4a9f2", "name": "research_paper_summarizer" }
```

Create version 1:

```
POST /v1/prompts/prm_d4a9f2/versions
Authorization: Bearer crt_acme_admin


{
  "version": "v1",
  "content": "You are a research assistant. Summarize the following academic paper in 3
paragraphs covering: (1) the core research question, (2) methodology, (3) key findings and
implications. Be thorough and accurate.\n\n{{paper_text}}",
  "model": "gpt-4o"
}
```

Create version 2 (optimised):

```
POST /v1/prompts/prm_d4a9f2/versions
Authorization: Bearer crt_acme_admin


{
  "version": "v2",
  "content": "Summarize this paper in 3 sections — Question, Method, Findings. Be concise.
Max 200 words.\n\n{{paper_text}}",
  "model": "claude-haiku-4-5"
}
```

A/B comparison (after 2 weeks):

```
GET /v1/prompts/analytics/comparison?prompt_id=prm_d4a9f2&v1=v1&v2=v2
```

```
{
  "prompt_id": "prm_d4a9f2",
  "comparison": {
    "v1": {
      "avg_cost_usd": 0.022,
      "avg_quality_score": 0.88,
      "avg_latency_ms": 4200,
      "call_count": 4000
    },
    "v2": {
      "avg_cost_usd": 0.014,
      "avg_quality_score": 0.92,
      "avg_latency_ms": 1800,
      "call_count": 4000
    },
    "winner": "v2",
    "cost_delta": -0.008,
    "quality_delta": +0.04,
    "latency_delta_ms": -2400
  }
}
```

v2 wins on every dimension: cheaper, higher quality, and faster.

Monthly saving:

```
$0.008/call × 8,000 calls/month = $64/month
$64 × 12 = $768/year saved from a single prompt optimisation
```

24.8 GitHub Copilot Attribution

Scenario: Acme Corp has 140 Copilot seats at $19/user/month = $2,660/month. The CTO suspects some seats aren't delivering value.

Developer attribution fetch:

```
GET /v1/cross-platform/developers?source=copilot&period=2026-04
Authorization: Bearer crt_acme_admin
```

Response (key rows):


```
{
  "copilot_developers": [
    {
      "email": "alice@acme.com",
      "seat_cost_usd": 19.00,
      "suggestions_shown": 8420,
      "suggestions_accepted": 7073,
      "acceptance_rate": 0.84,
      "active_days": 22,
      "lines_added_ai": 4200,
      "roi_tier": "high"
    },
    {
      "email": "inactive_dev@acme.com",
      "seat_cost_usd": 19.00,
      "suggestions_shown": 42,
      "suggestions_accepted": 4,
      "acceptance_rate": 0.10,
      "active_days": 3,
      "lines_added_ai": 8,
      "roi_tier": "low"
    }
  ],
  "summary": {
    "total_seats": 140,
    "total_cost_usd": 2660.00,
    "high_roi_developers": 8,
    "low_roi_developers": 22,
    "wasted_spend_usd": 418.00
  }
}
```


ROI breakdown:

| Tier | Developers | Acceptance Rate | Seat Cost | Assessment |

|------|----------|----------|----------|----------|

| High ROI (>80%) | 8 | avg 83% | $152/mo | Keep — demonstrable value |

| Medium ROI (40–80%) | 110 | avg 61% | $2,090/mo | Keep — healthy utilisation |

| Low ROI (<10%) | 22 | avg 6% | $418/mo | Review — likely unused |

Recommendation: Cancel 22 low-use seats = **$418/month** ($5,016/year) recovered. Re-assess after 60 days to see if the removed developers self-select back in.

The 8 high-ROI developers show >80% acceptance rate — these are the developers for whom Copilot is transformative. Average cost: $19/dev. Value generated: estimated 2-3 hours/day of accelerated coding.

24.9 Budget Alert System — Graduated + Z-Score Anomaly

Scenario: On April 15th at 2:14 AM, a developer accidentally triggers a batch processing job that calls GPT-4o 10,000 times in 20 minutes instead of the intended 100 calls. The job has a bug in its loop termination condition.

Normal baseline for the org:

- Typical hourly spend: $12/hr (averaged over 30 days)
- Standard deviation: $4/hr

What happens at 2:14 AM:

```
20-minute window: 10,000 GPT-4o calls × $0.018/call = $180
Annualised to hourly rate: $180 × 3 = $540/hr (10-minute window × 6)

Z-score = ($540 - $12) / $4 = 132

132 >> 3.0 threshold → ANOMALY ALERT
```

Alert fires to Slack within 60 seconds:

```
🚨 ANOMALY ALERT — Acme Corp
Org: org_acme_01
Time: 2026-04-15 02:14:32 UTC

Current spend rate: $540/hour (z-score: 132)
Normal baseline: $12/hour (±$4 σ)

Top contributor: dev_charlie@acme.com
Model: gpt-4o | Events: 10,000 in 20 min
Estimated cost this session: $180 (growing)

Action: Review active jobs → api.cohrint.com/traces
```

Damage contained: The developer is paged, kills the job at 2:17 AM. Total damage: **$38** (10,000 calls × $0.018 × partial run stopped early). Without the alert, the job would have run to completion at an estimated **$1,800+**.

Graduated alert timeline (if it were a normal budget approach):

Threshold	Fires At	Day of Month (at $184/day pace)
50%	$2,500	~Day 14
75%	$3,750	~Day 21
85%	$4,250	~Day 24
100%	$5,000	~Day 28

The Z-score anomaly alert is the safety net that graduated alerts miss entirely — it catches sudden spikes that happen within a single hour, not over weeks.

24.10 Audit Log — SOC 2 Compliance Evidence

Scenario: Acme Corp is going through SOC 2 Type II audit. The auditor requests evidence of access control changes, credential rotation, and budget governance over the past 90 days.

Audit log fetch:

```
GET /v1/audit-log?limit=100&before=1745884800
Authorization: Bearer crt_acme_admin
```

Response (relevant entries):


```
{
  "events": [
    {
      "id": "aud_001",
      "timestamp": "2026-04-17T14:23:00Z",
      "actor_email": "alice@acme.com",
      "action": "member_invited",
      "target": "carol@acme.com",
      "metadata": { "role": "member", "team": "frontend" }
    },
    {
      "id": "aud_002",
      "timestamp": "2026-04-17T12:05:00Z",
      "actor_email": "bob@acme.com",
      "action": "api_key_rotated",
      "metadata": { "key_hint": "crt_bob...3f2a" }
    },
    {
      "id": "aud_003",
      "timestamp": "2026-04-16T09:40:00Z",
      "actor_email": "alice@acme.com",
      "action": "budget_updated",
      "metadata": { "old_value": 4000, "new_value": 5000, "currency": "usd" }
    },
    {
      "id": "aud_004",
      "timestamp": "2026-04-15T18:22:00Z",
      "actor_email": "alice@acme.com",
      "action": "copilot_connected",
      "metadata": { "github_org": "acme-corp" }
    },
    {
      "id": "aud_005",
      "timestamp": "2026-04-15T02:14:32Z",
      "actor_email": "system",
      "action": "alert_fired",
      "metadata": { "type": "anomaly", "z_score": 132, "channel": "slack" }
    }
  ],
  "immutable": true,
  "note": "This log is append-only and cannot be modified by any user including superadmin."
}
```


SOC 2 evidence map:

| SOC 2 Control | Evidence From Audit Log |

|---|---|

| CC6.1 — Logical access restrictions | `member_invited`, `member_removed`, `role_changed` events |

| CC6.3 — Access removal | `member_removed` events with actor and timestamp |

| CC7.2 — Anomaly monitoring | `alert_fired` anomaly events |

| CC9.1 — Risk assessment (budget) | `budget_updated` with old/new values |

| CC6.7 — Credential management | `api_key_rotated` events (key hint only, never full token) |

Auditor request satisfied: Export CSV of all audit events → `GET /v1/audit-log/export?format=csv` → submit to auditor. No manual log reconstruction required.

24.11 MCP Server — IDE-Native Intelligence

Scenario: A developer is in VS Code working with Claude. They want to understand their recent agent session costs without leaving the editor.

Developer query to Claude in VS Code:

> *"How much did my last agent session cost, and what would I save if I optimised it?"*

Claude's MCP tool call chain:

Step 1 — Check current context:

```
Tool: check_budget
Input: {}
Output: {
  "mtd_cost_usd": 1840.00,
  "budget_usd": 5000.00,
  "days_until_budget_exhausted": 17,
  "daily_avg_cost_usd": 184.00
}
```

Step 2 — Get recent traces:

```
Tool: get_traces
Input: { "limit": 5, "developer_filter": "current_user" }
Output: {
  "traces": [
    {
      "trace_id": "trace_a3f291bc",
      "cost_usd": 1.84,
      "span_count": 12,
      "duration_min": 18,
      "started_at": "2026-04-17T14:23:00Z"
    }
  ]
}
```

Step 3 — Get recommendations:

```
Tool: get_recommendations
Input: { "trace_id": "trace_a3f291bc" }
Output: {
  "recommendations": [
    {
      "type": "model_switch",
      "description": "4 web_search spans use gpt-4o. Switch to claude-haiku-4-5.",
      "saving_usd": 0.90,
      "quality_delta": -0.01
    }
  ]
}
```

Claude's synthesised response to the developer:

> *"Your last agent session (18 minutes, 12 spans) cost $1.84. The biggest opportunity: 4 web_search spans are using gpt-4o at $0.18/call each. Switching those to claude-haiku-4-5 would save $0.90/session (49% reduction) with a negligible quality delta of -0.01.*
>
> *Your team is on track to exhaust the $5,000 budget in 17 days at current pace. Your personal contribution today is within normal range."*

Zero context-switching. The developer gets cost intelligence, trace analysis, and a concrete optimisation recommendation without opening a browser.

24.12 Privacy Modes — Healthcare Compliance

Scenario: A healthcare company using Cohrint processes prompts that may contain PHI (Protected Health Information) — patient IDs, symptom descriptions, medication names. They must comply with HIPAA requirements. They configure Cohrint to **strict mode**.

Mode configuration:

```
PATCH /v1/admin/settings
Authorization: Bearer crt_health_admin


{
  "privacy_mode": "strict"
}
```

What IS transmitted to api.cohrint.com (strict mode):

```
{
  "event_id": "evt_sha256_hash_of_prompt",
  "org_id": "org_healthco_01",
  "provider": "anthropic",
  "model": "claude-sonnet-4-6",
  "prompt_tokens": 847,
  "completion_tokens": 312,
  "cache_tokens": 0,
  "cost_usd": 0.0186,
  "latency_ms": 1840,
  "team": "team_clinical_ai",
  "developer_email": "dev_anon_8f4a",
  "trace_id": "trace_b99f01",
  "created_at": 1745884800
}
```

What is NOT transmitted:

- Prompt text (any PHI in the prompt)
- Response text (any PHI in the response)
- Raw developer email (hashed to `dev_anon_8f4a` using SHA-256 with org-specific salt)
- Any patient identifiers

Still available on the dashboard:

- Full cost visibility: $0.0186/call, $184/day, $1,840 MTD
- Token usage breakdown: 847 prompt + 312 completion

- Latency metrics: 1,840ms average
- Model breakdown: which models, how much
- Team attribution: team_clinical_ai vs team_admin
- Budget runway and forecasting

HIPAA alignment:

- No PHI leaves the developer's machine in strict mode
- The SDK hashes prompt text client-side before any network call
- Cohrint stores only the SHA-256 hash (one-way, not reversible)
- Cost and metadata analytics remain fully functional

Result: The healthcare company gets complete AI spend intelligence — budget, forecasting, model costs, team attribution — with zero PHI transmission risk. SOC 2 audit trail is still intact. Quality scoring is unavailable in strict mode (no response text to score), but all financial controls work normally.

Section 25 — Dashboard UI: Every Card & State

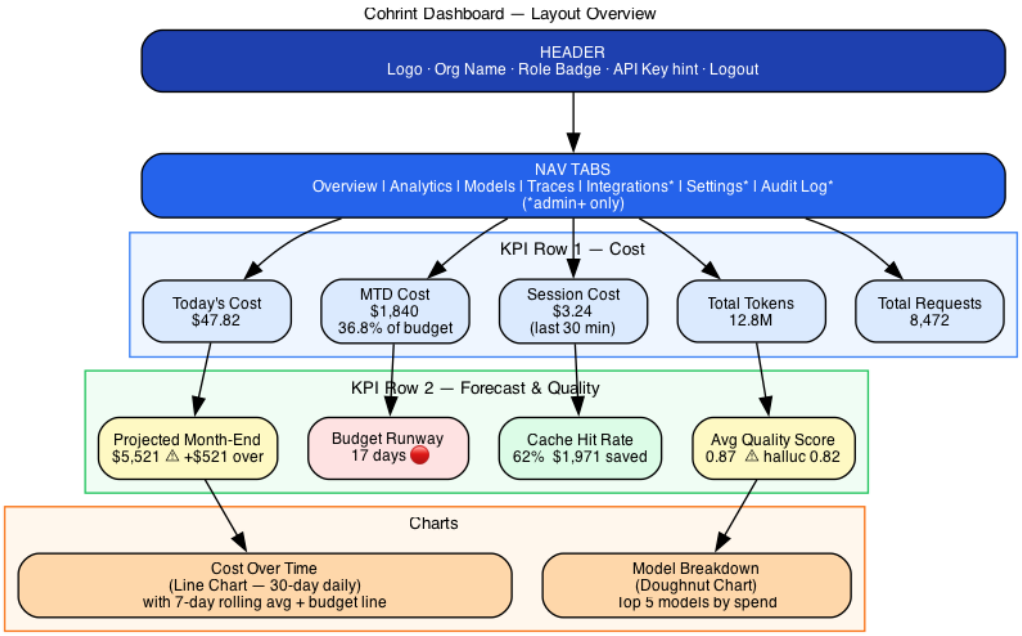

Figure 16 — Dashboard Layout

Dashboard UI Specification — Every Card, Every State

Overview

The Cohrint dashboard (`app.html`) is a single-page application served from Cloudflare Pages. It loads data from `api.cohrint.com` via six parallel `apiFetch()` calls on mount. Every card is role-aware — members see team-scoped data, admins see org-wide data.

Layout Structure

```
┌──────────────────────────────────────────────────────────────────────┐
│  HEADER  Logo · Org Name · Role Badge · API Key hint · Logout        │
├──────────────────────────────────────────────────────────────────────┤
│  NAV TABS                                                            │
│  Overview │ Analytics │ Models │ Traces │ [Integrations*] [Settings*] │
│           │           │        │        │ [Audit Log*]   (*admin+ only) │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐   │
│  │ KPI Card │ │ KPI Card │ │ KPI Card │ │ KPI Card │ │ KPI Card │   │
│  │ Today $  │ │  MTD $   │ │ Session  │ │  Tokens  │ │ Requests │   │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘   │
│                                                                      │
│  ┌──────────┐ ┌──────────┐ ┌──────────────────────────────────┐     │
│  │ Projected│ │  Budget  │ │                                  │     │
│  │Month-End │ │  Runway  │ │     Cost Over Time (Line Chart)  │     │
│  └──────────┘ └──────────┘ │     30-day daily cost breakdown  │     │
│                            │                                  │     │
│  ┌──────────┐ ┌──────────┐ └──────────────────────────────────┘     │
│  │  Cache   │ │ Quality  │                                          │
│  │ Hit Rate │ │ Score    │  ┌──────────────────────────────┐        │
│  └──────────┘ └──────────┘  │ Model Breakdown (Doughnut Chart) │     │
│                             │  Top 5 models by spend           │     │
└──────────────────────────────────────────────────────────────────────┘
```

Tab 1 — Overview

KPI Row 1 — Primary Cost Cards

Card: Today's Cost

```
┌──────────────────────────┐
│  💰  Today's Cost        │
│                          │
│       $  47.82           │
│                          │
│  ▲ +12% vs yesterday     │
│  Source: /analytics/summary │
│  Field:  today_cost_usd  │
└──────────────────────────┘
```

Data source: `GET /v1/analytics/summary` → `today_cost_usd`

What it shows: Total LLM API spend since UTC midnight today. Resets at 00:00 UTC.

States:

- `$0.00` — No events today (new org or weekend)
- `$0.00–$10` — Green text
- `$10–$100` — Default (gray)
- `>$100` — Amber warning if approaching daily budget

Update cadence: Refreshed every 30 seconds via SSE stream or polling.

Role scoping: Members with `scope_team` set see only their team's today cost.

Edge cases:

- If `today_cost_usd` is null (no events ever), shows – not `$0.00`
- Tooltip: "Spend since 00:00 UTC. Updates in real-time."

Card: Month-to-Date (MTD) Cost

```
┌──────────────────────────┐
│  📅  Month-to-Date        │
│                          │
│      $ 1,840.50          │
│                          │
│  36.8% of $5,000 budget  │
│  ████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  Budget bar  │
│  Source: /analytics/summary │
│  Field:  mtd_cost_usd    │
└──────────────────────────┘
```

Data source: `GET /v1/analytics/summary` → `mtd_cost_usd`, `budget_usd`

What it shows: Cumulative spend from 1st of current month to now (UTC).

Budget progress bar:

- 0–50%: Green fill
- 50–75%: Amber fill
- 75–90%: Orange fill
- 90–100%: Red fill
- >100%: Red fill + "OVER BUDGET" badge

States:

- No budget set (`budget_usd = 0`): Bar hidden, just shows dollar amount
- Budget set: Shows "X% of $Y,000 budget" with progress bar

Formula: `budget_pct = (mtd_cost_usd / budget_usd) × 100`

Card: Session Cost (Last 30 min)

```
┌──────────────────────────┐
│  ⚡  Session Cost         │
│                          │
│        $  3.24           │
│                          │
│  Last 30 minutes         │
│  Source: /analytics/summary │
│  Field:  session_cost_usd │
└──────────────────────────┘
```

Data source: `GET /v1/analytics/summary` → `session_cost_usd`

What it shows: Cost of LLM calls in the last 30 minutes. Designed for developers to see real-time cost of their active coding session.

Use case: Developer running a long agent session can glance at this to see if they're burning money unexpectedly.

States:

- `$0.00` — No recent activity (idle)
- Updates every 30 seconds
- If `> $10` in 30 min: amber highlight (potential runaway)

Card: Total Tokens

```
┌──────────────────────────┐
│  🔤  Total Tokens (MTD)     │
│                          │
│     12,847,392           │
│                          │
│  Prompt: 8.2M  Comp: 4.6M  │
│  Source: /analytics/kpis    │
│  Field:  total_tokens       │
└──────────────────────────┘
```

Data source: `GET /v1/analytics/kpis?period=30` → `total_tokens`, `total_prompt_tokens`, `total_completion_tokens`

What it shows: Token consumption breakdown for the trailing 30 days.

Sub-labels: Prompt tokens vs Completion tokens. High prompt/completion ratio may indicate verbose system prompts costing money without value.

Format: Numbers >1M formatted as "12.8M", >1K as "12.8K"

Card: Total Requests

```
┌──────────────────────────┐
│  📡  Total Requests (MTD)  │
│                          │
│        8,472             │
│                          │
│  Avg cost: $0.22 / request │
│  Source: /analytics/kpis   │
│  Field:  total_requests    │
└──────────────────────────┘
```

Data source: `GET /v1/analytics/kpis?period=30` → `total_requests`, computed avg

What it shows: Number of LLM API calls in the last 30 days.

Sub-label: `Avg cost per request = mtd_cost_usd / total_requests` — signals whether requests are cheap (high-volume batch) or expensive (long-context one-shots).

KPI Row 2 — Forecast & Quality Cards

Card: Projected Month-End Cost (NEW — PR #70)

```
┌──────────────────────────┐
│  📈  Projected Month-End   │
│                          │
│      $ 5,521.50          │
│                          │
│  +$521 over budget       │
│  Based on $184/day avg   │
│  Source: /analytics/summary │
│  Field: projected_month_end │
└──────────────────────────┘
```

Data source: `GET /v1/analytics/summary` → `projected_month_end_usd`, `daily_avg_cost_usd`

What it shows: If today's daily average spend continues for the rest of the month, this is the projected total.

Formula (computed server-side):

```
daily_avg = mtd_cost_usd / MAX(UTC_day_of_month, 1)
projected  = daily_avg × days_in_current_month
```

Color coding:

- Under budget: Green text
- Within 10% of budget: Amber text + "⚠ Approaching limit"
- Over budget: Red text + "+$X over budget" sub-label

States:

- Month is day 1: projection equals today's cost × days_in_month (high uncertainty)
- No budget set: Shows projection without "over/under" comparison, just the dollar amount
- `null` (no events this month): Shows –

Tooltip: "Projection based on $X/day average over the last N days. Assumes constant daily spend."

Card: Budget Runway (NEW — PR #70)

```
┌──────────────────────────┐
│  🏃  Budget Runway        │
│                          │
│         17 days          │
│                          │
│  Budget exhausted Apr 27 │
│  At current $184/day pace │
│  Source: /analytics/summary │
│  Field: days_until_budget_ │
│         exhausted        │
└──────────────────────────┘
```

Data source: `GET /v1/analytics/summary` → `days_until_budget_exhausted`

What it shows: Days remaining before monthly budget is exhausted at the current daily spend rate.

Formula:

```
remaining = budget_usd - mtd_cost_usd
runway    = CEIL(remaining / daily_avg_cost_usd)
           → 0  if budget already exceeded
           → null if no budget set or daily_avg = 0
```

Color coding (traffic light):

- `> 14 days` — Green background: "On track"
- `8-14 days` — Amber background: "⚠ Monitor spend"
- `1-7 days` — Red background: "🔴 Budget critical"
- `= 0` — Red background + "BUDGET EXCEEDED" text
- `null` (no budget) — Gray: "No budget set — configure in Settings"

Sub-label: Computes and displays the calendar date: "Budget exhausted [Month Day]"

Action link: "Set budget →" links to Settings tab (admin+ only)

Card: Cache Hit Rate

```
┌──────────────────────────┐
│  ⚡  Semantic Cache       │
│                          │
│       62% hit rate       │
│  ▓▓▓
```

- `50-80%` — Green (excellent)
- `> 80%` — Gold (outstanding — highly repetitive workload)

Sub-label: `$X saved this month` — sum of `cost_usd` of all cache hits, showing actual dollar value.

States:

- Cache disabled (`enabled = false`): Shows "Cache disabled — enable in Settings"
- No entries yet: Shows "0% — cache warming up"
- Not enough data (<100 lookups): Shows "Insufficient data"

Card: Avg Quality Score

```
┌──────────────────────────┐
│  ✅  Avg Quality Score   │
│                          │
│     0.87 / 1.00          │
│  ▓▓▓
```

States:

- No scored events: Shows "—" with "Quality scoring is async — scores appear within minutes of events"
- Only available for organisations using Cohrint quality scoring (requires events with responses)

Action: Clicking the card navigates to the Traces tab filtered by low hallucination score.

Cost Over Time Chart

```
┌──────────────────────────────────────────────────────────┐
│  Cost Over Time (Last 30 Days)         [7d] [14d] [30d] │
│                                                          │
│  $300 ┤                      ╭╮                          │
│  $250 ┤                 ╭────╯ ╰╮                        │
│  $200 ┤            ╭────╯        ╰────╮                  │
│  $150 ┤     ╭──────╯                  ╰──                 │
│  $100 ┤─────╯                                            │
│   $50 ┤                                                  │
│    $0 └──────────────────────────────────────────        │
│       Apr 1          Apr 15              Apr 30          │
│                                                          │
│  ● Daily cost   ○ 7-day rolling avg                      │
└──────────────────────────────────────────────────────────┘
```

Data source: `GET /v1/analytics/timeseries?period=30` → array of `{date, cost_usd, tokens, requests}`

What it shows: Daily spend as a line chart using Chart.js. Period selector (7/14/30 days) re-fetches.

Chart elements:

- Primary line: Daily `cost_usd` (solid blue)
- Secondary line: 7-day rolling average (dashed gray) — smooths out weekday/weekend variation
- X-axis: Date labels (every 5 days for 30-day view)
- Y-axis: Dollar amount, auto-scaled
- Hover tooltip: `Apr 17: $184.20 (↑12% vs 7-day avg)`

Budget line: If `budget_usd > 0`, draws a horizontal red dashed line at `budget_usd / days_in_month` (daily budget ceiling).

Model Breakdown Doughnut Chart

```
┌──────────────────────────────────┐
│  Spend by Model (Last 30 Days)   │
│                                  │
│          ┌──────────┐            │
│       ┌─┤  $1,840  ├─┐           │
│  GPT-4o│ │  Total   │ │claude-opus │
│  42%   │ └──────────┘ │  28%      │
│       └──────────────┘           │
│                                  │
│  • gpt-4o          $772 (42%)    │
│  • claude-opus-4-6 $515 (28%)    │
│  • claude-sonnet-4 $239 (13%)    │
│  • gpt-4o-mini     $184 (10%)    │
│  • gemini-pro      $130  (7%)    │
└──────────────────────────────────┘
```

Data source: `GET /v1/analytics/models?period=30` → array of `{model, cost_usd, tokens, requests, pct}`

What it shows: Proportional spend per model as a doughnut chart. Centre shows total spend.

Legend: Each model listed with dollar amount and percentage. Top 5 shown; rest collapsed as "Other."

Click interaction: Clicking a model segment filters the timeseries chart to show only that model's spend over time.

Tab 2 — Analytics

Period Selector

```
┌──────────────────────────────────────┐
│  Period: [7 days ▼]   Team: [All ▼]  │
└──────────────────────────────────────┘
```

All Analytics tab charts re-fetch with `?period=N` when period changes. Team dropdown (admin+) filters by team.

KPI Summary Row

```
┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
│ Total    │ │ Total    │ │ Avg Cost │ │ Avg      │ │ Streaming│
│ Cost     │ │ Requests │ │ /Request │ │ Latency  │ │  Count   │
│ $2,840   │ │  12,472  │ │  $0.23   │ │  1,240ms │ │    3     │
└──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘
```

Data source: `GET /v1/analytics/kpis?period=N`

Streaming count: Number of currently open SSE connections — shows how many developers are live-streaming data right now.

Timeseries Chart (Full)

Same as Overview but with period selector and toggles for cost/tokens/requests.

Teams Table

```
┌──────────────────────────────────────────────────────────────┐
│  Team Breakdown                                              │
├──────────────┬──────────┬──────────┬──────────────┬──────────┤
│  Team        │  Cost    │ Requests │ Budget Used  │ Status   │
├──────────────┼──────────┼──────────┼──────────────┼──────────┤
│  backend     │  $840    │  4,820   │  84%  ████   │  ⚠       │
│  frontend    │  $320    │  2,140   │  32%  ██     │  ✓       │
│  data-sci    │  $680    │  3,890   │  68%  ████   │  ✓       │
│  devops      │  $120    │    980   │  12%  ▌      │  ✓       │
└──────────────┴──────────┴──────────┴──────────────┴──────────┘
```

Data source: `GET /v1/analytics/teams?period=N`

Budget Used column: `(team_cost / team_budget) × 100`. Red if >80%, amber if >60%.

Status column: ✓ = under budget, ⚠ = >80% budget, 🔴 = over budget.

Click row: Opens team drill-down modal showing per-developer breakdown within that team.

Tab 3 — Models

```
┌──────────────────────────────────────────────────────────────┐
│  Model Cost Breakdown                    Period: [30 days ▼] │
├──────────┬──────────┬──────────┬──────────┬──────────────────┤
│  Model   │  Cost    │ Requests │ Avg$/req │  Quality Avg     │
├──────────┼──────────┼──────────┼──────────┼──────────────────┤
│ gpt-4o   │  $772    │  4,289   │  $0.180  │ ✓ 0.94           │
│ claude-  │  $515    │  5,722   │  $0.090  │ ✓ 0.92           │
│ opus-4-6 │          │          │          │                  │
│ claude-  │  $239    │  6,640   │  $0.036  │ ✓ 0.91           │
│ sonnet-4 │          │          │          │                  │
│ gpt-4o-  │  $184    │ 23,000   │  $0.008  │ ○ —              │
│ mini     │          │          │          │                  │
│ gemini-  │  $130    │  4,333   │  $0.030  │ ✓ 0.89           │
│ pro      │          │          │          │                  │
└──────────┴──────────┴──────────┴──────────┴──────────────────┘
```

Data source: `GET /v1/analytics/models?period=N`

Quality Avg column: Average `hallucination_score` for that model over the period. ○ — = no quality scores yet.

Recommendation badge: If a cheaper model exists with quality within 0.05 of the current model, shows "💡 Consider claude-haiku-4-5 ($0.001/req, 0.91 quality)"

Sort: Default by cost DESC. Clickable column headers for re-sort.

Tab 4 — Traces

Trace List

```
┌──────────────────────────────────────────────────────────────┐
│  Agent Traces                              Period: [7 days ▼] │
├──────────────────┬──────┬──────────┬──────────┬──────────────┤
│  Trace ID        │Spans │   Cost   │Latency   │  Started     │
├──────────────────┼──────┼──────────┼──────────┼──────────────┤
│ trace_a3f2...    │  23  │  $4.20   │ 45min    │ Apr 17 14:23 │
│ trace_b891...    │   8  │  $0.84   │  8min    │ Apr 17 12:05 │
│ trace_c44a...    │   3  │  $0.12   │  2min    │ Apr 17 09:47 │
└──────────────────┴──────┴──────────┴──────────┴──────────────┘
```

Data source: `GET /v1/analytics/traces?period=N`

Click row: Expands to show the full DAG tree for that trace.

Trace DAG Detail View

```
┌──────────────────────────────────────────────────────────────┐
│  Trace: trace_a3f2...    Total: $4.20    Duration: 45min     │
│                                                               │
│  ▼ 🤖 coding-agent (root)              $4.20   45min          │
│      ├─ 📄 read_file ×8               $0.08    2min           │
│      ├─ 🔍 analyze_code ×3            $0.54    8min           │
│      │    ├─ 💬 call_llm (gpt-4o)     $0.18    2.1min         │
│      │    ├─ 💬 call_llm (gpt-4o)     $0.18    2.3min         │
│      │    └─ 💬 call_llm (gpt-4o)     $0.18    1.9min         │
│      ├─ ✍️  write_tests ×4            $0.30    5min           │
│      └─ 💬 call_llm ×8 (gpt-4o)      $3.04   30min ← 72%      │
│                                                               │
│  ⚠  8 call_llm spans using gpt-4o ($0.38/each avg)            │
│  💡 Switch to claude-haiku-4-5 → save ~$2.80/run              │
└──────────────────────────────────────────────────────────────┘
```

Data source: `GET /v1/analytics/traces/:traceId`

Tree rendering: Collapsible nodes. Each node shows: tool name, call count, cost, latency, model (if LLM call).

Cost attribution bar: Visual bar beside each node proportional to its % of total trace cost.

Recommendation inline: If any model has a cheaper equivalent with comparable quality, shows suggestion inline within the trace.

RBAC: Members see only their own traces (`developer_email` filter). Admins see all.

Tab 5 — Integrations (admin+ only)

Claude Code Card

```
┌──────────────────────────────────────────┐
│  Claude Code                    ● ACTIVE  │
│                                           │
│  67,420 events tracked                    │
│  Last event: 2 minutes ago                │
│  API Key: crt_myorg...  [Rotate key]      │
│                                           │
│  Setup: ~/.claude/settings.json ✓         │
│  Hook: PostToolUse Stop hook ✓            │
│                                           │
│  [View Claude Code events →]              │
└──────────────────────────────────────────┘
```

Status logic:

- ● `ACTIVE` (green) — events received in last 24h via Stop hook
- ◎ `SETUP` (amber) — API key valid but no events from Claude Code hook
- ○ `NOT CONNECTED` (gray) — no events in last 7 days

"Rotate key" button: Calls `POST /v1/auth/rotate`, shows new key once (copy-on-click), updates hint.

Setup validation: Checks `~/.claude/settings.json` hook exists via a `GET /v1/analytics/summary?source=hook` probe.

GitHub Copilot Card

```
┌──────────────────────────────────────────┐
│  GitHub Copilot                 ● ACTIVE  │
│                                           │
│  Org: acme-corp                           │
│  140 seats tracked                        │
│  Last sync: Sun Apr 14 00:00 UTC          │
│  Next sync: Sun Apr 21 00:00 UTC          │
│                                           │
│  ┌──────────────────────────────────┐     │
│  │ Top 5 developers by acceptance %  │     │
│  │ alice: 84% · bob: 79% · carol: 71%│     │
│  └──────────────────────────────────┘     │
│                                           │
│  [Disconnect]  [View all developers →]    │
└──────────────────────────────────────────┘
```

Connect flow:

51. Admin clicks "Connect GitHub Copilot"
52. Modal: enter GitHub Org name + Personal Access Token (PAT)
53. POST `/v1/copilot/connect` → PAT encrypted AES-256-GCM → stored in KV only
54. Card shows ACTIVE after first cron sync (Sunday UTC)

States:

- Not connected: "Connect GitHub Copilot" button + instructions
- Connected, pre-sync: "Waiting for first sync (Sunday UTC)"
- Connected, active: Shows sync status + developer stats

Datadog Card

```
┌──────────────────────────────────────────┐
│  Datadog Exporter               ● ACTIVE  │
│                                           │
│  Site: datadoghq.com                      │
│  Metrics pushed: vantage.ai.cost_usd      │
│                  vantage.ai.tokens        │
│  Last push: Apr 17 14:20 UTC             │
│  Events exported this month: 8,472        │
│                                           │
│  Tags: provider · model · team · developer │
│                                           │
│  [View in Datadog ↗]  [Disconnect]       │
└──────────────────────────────────────────┘
```

Connect flow:

55. Click "Connect Datadog"
56. Modal: select DD site (5 options), enter API key
57. POST `/v1/datadog/connect` → API key encrypted → stored
58. Cohrint pushes metrics to Datadog on each event batch

OTel / Cross-Platform Card

```
┌──────────────────────────────────────────┐
│  OpenTelemetry Collector        ● RECEIVING  │
│                                              │
│  Endpoint: api.cohrint.com/v1/otel/v1/logs  │
│  Events this month: 42,881                  │
│  Sources detected:                          │
│    Claude Code  ██████  31,420 events       │
│    Cursor       ████    8,240 events        │
│    Gemini CLI   ██      3,221 events        │
│                                             │
│  [Setup guide]  [View OTel events →]        │
└──────────────────────────────────────────┘
```

Source detection: Infers source from OTel `service.name` attribute in incoming logs.

Setup guide: Shows copy-pasteable config for each tool (Claude Code, Cursor, Gemini CLI, etc.)

Local Proxy Card

```
┌──────────────────────────────────────────┐
│  Local Proxy Gateway            ○ NOT SET    │
│                                              │
│  Privacy mode: Standard (default)           │
│                                             │
│  Modes:                                     │
│  ○ Strict   — metadata only, no text       │
│  ● Standard — metadata + prompt hash       │
│  ○ Relaxed  — full text (for analysis)     │
│                                             │
│  [Change privacy mode]                      │
│  [Download local proxy →]                   │
└──────────────────────────────────────────┘
```

Privacy mode toggle: `PATCH /v1/admin/settings` → `privacy_mode: 'strict'|'standard'|'relaxed'`

Tab 6 — Settings (admin+ only)

Organisation Settings

```
┌──────────────────────────────────────────┐
│  Organisation                            │
│                                          │
│  Name:     Acme Corp          [Edit]      │
│  Plan:     Team               [Upgrade →] │
│  Industry: Technology         [Edit]      │
│  Budget:   $5,000 / month     [Edit]      │
│                                          │
│  Benchmark opt-in:  ● Enabled  [Disable]  │
│  (Contribute to anonymised industry data) │
└──────────────────────────────────────────┘
```

Semantic Cache Settings

```
┌──────────────────────────────────────────┐
│  Semantic Cache Configuration            │
│                                          │
│  Status:        ● Enabled     [Disable]  │
│  Similarity:    0.92 ──●──────── [0.85-0.99] │
│  Min length:    10 chars                 │
│  Max age:       30 days                  │
│                                          │
│  ℹ  Higher threshold = fewer hits,       │
│     less risk of wrong cached response.  │
│                                          │
│  [Save changes]                          │
└──────────────────────────────────────────┘
```

Threshold slider: Live preview: "At 0.92, your current hit rate is 62%. At 0.85, estimated 74% (+12%, ~$380 more savings, small risk of inexact matches)."

Team Budget Management

```
┌──────────────────────────────────────────────────────┐
│  Team Budgets                          [Add team budget] │
├──────────────┬──────────────┬──────────────────────┤
│  Team        │  Budget/mo   │  Actions             │
├──────────────┼──────────────┼──────────────────────┤
│  backend     │  $1,000      │  [Edit] [Remove]     │
│  frontend    │  $1,000      │  [Edit] [Remove]     │
│  data-sci    │  $1,000      │  [Edit] [Remove]     │
│  devops      │  $1,000      │  [Edit] [Remove]     │
│  (org total) │  $5,000      │  —                   │
└──────────────┴──────────────┴──────────────────────┘
```

Alert Configuration

```
┌──────────────────────────────────────────┐
│  Budget Alerts                           │
│                                          │
│  Slack webhook URL:                      │
│  https://hooks.slack.com/...  [Edit]     │
│                                          │
│  Alert thresholds:                       │
│  ☑  50% — "Halfway through budget"       │
│  ☑  75% — "75% budget used"              │
│  ☑  85% — "Budget warning"               │
│  ☑ 100% — "Budget exhausted"             │
│  ☑  Anomaly alerts (Z-score > 3σ)        │
│                                          │
│  [Test alert]  [Save]                    │
└──────────────────────────────────────────┘
```

Member Management

```
┌──────────────────────────────────────────────────────────┐
│  Team Members                          [Invite member +]  │
├──────────────────┬──────────┬──────────┬──────────────────┤
│  Email           │  Role    │  Team    │  Actions         │
├──────────────────┼──────────┼──────────┼──────────────────┤
│  alice@acme.com  │  admin   │  (all)   │  [Edit] [Remove] │
│  bob@acme.com    │  member  │  backend │  [Edit] [Remove] │
│  carol@acme.com  │  member  │  frontend│  [Edit] [Remove] │
│  dave@acme.com   │  viewer  │  (all)   │  [Edit] [Remove] │
│  cto@acme.com    │  ceo     │  (all)   │  [Edit] [Remove] │
└──────────────────┴──────────┴──────────┴──────────────────┘
```

Invite flow: Enter email + role + team → sends invite email via Resend → member receives link → sets password → gets scoped API key.

Role constraints: Admin cannot invite superadmin or owner roles.

Scope team: If set, member's API key only returns data for that team. NULL = sees all teams.

Tab 7 — Audit Log (admin+ only)

```
┌──────────────────────────────────────────────────────────────┐
│  Audit Log                    Filter: [All events ▼]  [Export] │
├──────────────────┬──────────────────┬──────────────────────────┤
│  Timestamp       │  Actor           │  Action                  │
├──────────────────┼──────────────────┼──────────────────────────┤
│ Apr 17 14:23     │ alice@acme       │ member_invited carol@acme.com  │
│ Apr 17 12:05     │ bob@acme         │ api_key_rotated (own key)      │
│ Apr 16 09:40     │ alice@acme       │ budget_updated $4000 → $5000   │
│ Apr 15 18:22     │ alice@acme       │ copilot_connected acme-corp    │
│ Apr 15 17:01     │ system           │ alert_fired 80% budget (Slack) │
│ Apr 14 10:33     │ alice@acme       │ cache_config_updated 0.88→0.92 │
└──────────────────┴──────────────────┴──────────────────────────┘
```

Data source: `GET /v1/audit-log?limit=50&before=<timestamp>` — paginated, newest first.

Infinite scroll: "Load more" appends older entries using the `before` cursor.

Export button: Downloads filtered log as CSV (admin+ only).

Immutability note: Audit log is append-only. No admin can delete entries. This is shown in the UI: "This log is immutable and cannot be modified."

Event types logged:

- `member_invited` — actor, target email, role assigned
- `member_removed` — actor, target email
- `api_key_rotated` — actor, key hint (never full key)
- `budget_updated` — actor, old value, new value
- `role_changed` — actor, target, old role, new role
- `copilot_connected` / `copilot_disconnected` — actor, github_org
- `datadog_connected` / `datadog_disconnected` — actor, site
- `cache_config_updated` — actor, old/new threshold
- `alert_fired` — system, threshold, channel
- `auth_failed` — IP, reason (rate-limited to prevent log flooding)

Tab 8 — Executive View (ceo+ only)

```
┌──────────────────────────────────────────────────────────────────────┐
│  Executive Dashboard                          Last 30 days           │
│                                                                      │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐   │
│  │ Total AI Spend   │  │ Cost per Dev     │  │  ROI Estimate    │   │
│  │   $18,420        │  │  $131/dev/mo     │  │   4.2× ROI       │   │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘   │
│                                                                      │
│  Spend by Source                    Spend by Team                    │
│  ┌──────────────────────────┐       ┌──────────────────────────┐    │
│  │ LLM APIs      $12,840 70%│       │ backend    $7,200 39%    │    │
│  │ Copilot        $2,660 14%│       │ data-sci   $5,400 29%    │    │
│  │ Cursor (seats) $1,800 10%│       │ frontend   $3,600 20%    │    │
│  │ Other          $1,120  6%│       │ devops     $2,220 12%    │    │
│  └──────────────────────────┘       └──────────────────────────┘    │
│                                                                      │
│  [Download Executive Report PDF]                                     │
└──────────────────────────────────────────────────────────────────────┘
```

Data source: `GET /v1/analytics/executive` → cross-source spend roll-up.

Cost per Dev: `total_spend / headcount` — requires org to have member count in settings.

ROI Estimate: Configurable formula: `(developers × avg_hourly_rate × hours_saved_per_dev) / total_ai_spend`. Default: 2h saved/dev/day × $75/hr × 22 working days.

Download Report: Generates PDF chargeback report per team (roadmap feature).

Real-Time Indicator

```
┌──────────────────────────────────────────────┐
│  ● Live  3 connections   Last event: 14s ago  │
└──────────────────────────────────────────────┘
```

Shown in the header bar. Green dot pulses when SSE stream is receiving events. Shows count of active SSE connections across the org and timestamp of the most recent event.

SSE flow:

59. On load: `POST /v1/stream/:orgId` → returns SSE token (KV-stored, 120s TTL)

60. Dashboard opens SSE connection using token
61. Each new event triggers: KPI cards refresh, timeseries chart updates, streaming count increments

Empty States

Each card has a defined empty state to avoid confusing blank UI:

Card	Empty State Message
Today's Cost	"No events today. Send your first event →" + link to docs
MTD Cost	"No events this month. Getting started guide →"
Cache Hit Rate	"Cache warming up. 0 lookups so far."
Quality Score	"No quality scores yet. Scores appear within minutes of events."
Traces	"No agent traces found. Add trace_id to your events →"
Models	"No model data yet. Events with model field will appear here."
Copilot	"Connect GitHub Copilot to see per-developer attribution →"
Audit Log	"No audit events. Admin actions will appear here automatically."

Loading States

All cards show a skeleton loader (gray animated placeholder) while data is fetching. The six `Promise.all()` fetches on load mean cards render simultaneously — no waterfall.

```
// All 6 fetches fire in parallel
const [summary, kpis, timeseries, models, teams, cacheStats] =
  await Promise.all([
    apiFetch('/v1/analytics/summary'),
    apiFetch('/v1/analytics/kpis?period=30'),
    apiFetch('/v1/analytics/timeseries?period=30'),
    apiFetch('/v1/analytics/models?period=30'),
    apiFetch('/v1/analytics/teams?period=30'),
    apiFetch('/v1/cache/stats'),
  ])
```

Error States

If an API call fails (401, 429, 5xx), the affected card shows:

```
┌──────────────────────┐
│  ⚠  Failed to load   │
│  [Retry]             │
└──────────────────────┘
```

- 401: Redirects to login page
- 429: Shows "Rate limited — retrying in 60s" with countdown
- 5xx: Shows "API error — [Retry]" button
- Network error: Shows "Check connection — [Retry]"

Section 26 — Competitive Analysis

Feature	Cohrint	Helicone	LangSmith	Langfuse	Datadog LLM	Palma.ai
AI coding tool OTel tracking	✅Yes	❌No	❌No	❌No	⚠️Partial	⚠️Claimed
GitHub Copilot billing	✅Yes	❌No	❌No	❌No	❌No	❓Unknown
Per-developer attribution	✅Yes	❌No	❌No	❌No	❌No	⚠️Claimed
No proxy required	✅Yes	❌No	❌No	❌No	✅N/A	❓Unknown
MCP server (IDE integration)	✅12 tools	❌No	❌No	❌No	⚠️Different	❌No
Semantic cache	✅AI-native	✅Exact	❌No	❌No	❌No	❓Unknown
Privacy / strict mode	✅3 modes	❌No	❌No	❌No	❌No	❓Unknown
Datadog exporter	✅Yes	❌No	❌No	❌No	✅Native	❓Unknown
Anonymized benchmark data	✅Yes	❌No	❌No	❌No	❌No	❌No
Agent trace DAG	✅Yes	✅Yes	✅Yes	✅Yes	✅Yes	❓Unknown
Eval / quality scores	✅6-dim	⚠️Limited	✅Full	✅Full	❌No	❓Unknown
CLI agent wrapper	✅Yes	❌No	❌No	❌No	❌No	❌No
OSS / self-host	❌No	✅Apache	❌No	✅MIT	❌No	❌No
Free tier	50K evt/mo	10K/mo	5K traces	50K units	❌None	❓Unknown
Audit log	✅Yes	⚠️Limited	✅Yes	✅Yes	✅Yes	❓Unknown

Figure 11 — Competitive Feature Matrix

Cohrint — Competitive Analysis & Market Strategy

Version 1.0 · April 2026 · INTERNAL — NOT FOR PUBLIC DISTRIBUTION

1. Executive Summary

Cohrint is the **only full-stack AI coding spend platform** as of April 2026. No funded competitor covers the complete AI coding stack — IDE tools (GitHub Copilot), LLM APIs (OpenAI, Anthropic, Google), agent frameworks (any OTel-compatible), and existing monitoring (Datadog) — in a single dashboard.

The competitive window is open but narrowing. Palma.ai is the only direct ICP threat; all other competitors operate in adjacent categories with structural blind spots Cohrint exploits.

Core positioning: "Know what your AI bill will be before it arrives — and cut it."

Category claim: AI Coding FinOps — a new category distinct from LLMOps, ML observability, or cloud cost management.

2. Market Size

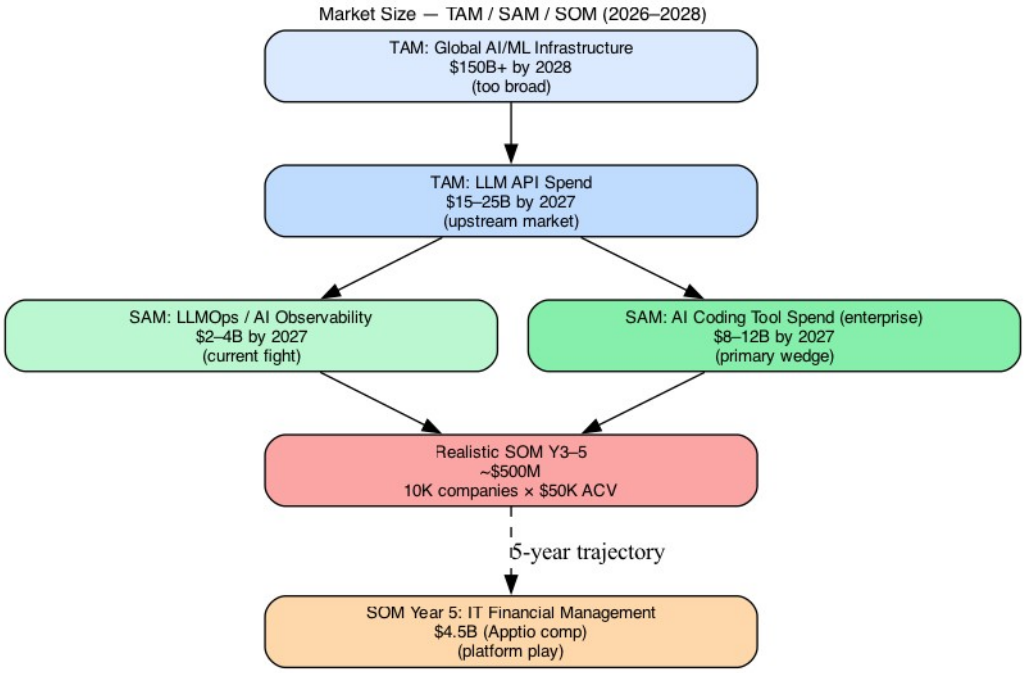

Figure 14 — TAM/SAM/SOM

| Market | Size | Cohrint Access |

|--------|------|----------|

| Global AI/ML Infrastructure spend | $150B+ by 2028 | TAM (too broad) |

| LLM API spend (all companies) | $15–25B by 2027 | TAM — upstream market |

| LLMOps / AI Observability tools | $2–4B by 2027 | SAM — current fight |

| AI coding tool spend (enterprise) | $8–12B by 2027 | SAM — primary wedge |

| IT Financial Management (Apptio comp) | $4.5B | SOM Year 5 target |

| Realistic SOM Y3–5 | ~$500M | 10K companies × $50K ACV |

The structural analog is Bloomberg Terminal: a **neutral market intelligence layer** that financial institutions cannot build themselves because of inherent conflicts of interest. AI

providers cannot tell you a competitor model is cheaper. That conflict is permanent — it cannot be funded away.

3. Competitive Landscape Matrix (April 2026)

| Feature | Cohrint | Helicone | LangSmith | Langfuse | Datadog LLM | GitHub Copilot Analytics | Palma.ai |

|---|---|---|---|---|---|---|---|

| **Free tier** | 50K evt/mo | 10K/mo | 5K traces/mo | 50K units/mo | None | Included w/ seat | Unknown |

| **Paid entry** | TBD | $20/seat/mo | $39/seat/mo | $29/mo flat | ~$120/day activation | $10–19/user/mo | Unknown |

| **OSS / self-host** | No | Yes (Apache 2.0) | No | Yes (MIT) | No | No | Unknown |

| **AI coding tool OTel** | ✅ **Yes — 10 tools** | ❌ No | ❌ No | ❌ No | ⚠️ Partial | Own only | ⚠️ Claimed |

| **GitHub Copilot billing adapter** | ✅ **Yes — GA API, AES encrypted** | ❌ No | ❌ No | ❌ No | ❌ No | Native only | Unknown |

| **Per-developer attribution (cross-tool)** | ✅ **Yes** | ❌ No | ❌ No | ❌ No | ❌ No | Own only | ⚠ Claimed |

| **No proxy required** | ✅ **Yes** | ❌ No | ❌ No | ❌ No | N/A | N/A | Unknown |

| **MCP server (12 tools)** | ✅ **Yes** | ❌ No | ❌ No | ❌ No | ⚠️ Different | ❌ No | Unknown |

| **CLI agent wrapper** | ✅ **Yes** | ❌ No | ❌ No | ❌ No | ❌ No | ❌ No | Unknown |

| **Privacy / strict mode** | ✅ **3 modes** | ❌ No | ❌ No | ❌ No | ❌ No | ❌ No | Unknown |

| **Datadog exporter** | ✅ **Yes** | ❌ No | ❌ No | ❌ No | N/A | ❌ No | Unknown |

| **Anonymized benchmark data** | ✅ **Yes (k-anon, opt-in)** | ❌ No | ❌ No | ❌ No | ❌ No | ❌ No | ❌ No |

| **Semantic cache (AI-native)** | ✅ **BGE-384, 0.92 threshold** | ⚠ Exact-match only | ❌ No | ❌ No | ❌ No | ❌ No | Unknown |

| **Prompt Registry** | ✅ **Yes** | ❌ No | ✅ Yes | ✅ Yes | ❌ No | ❌ No | Unknown |

| **Agent trace DAG** | ✅ Yes | ✅ Yes | ✅ Yes | ✅ Yes | ✅ Yes | ❌ No | Unknown |

| **Quality / eval scores** | ✅ 6-dimension | ⚠️ Limited | ✅ Full | ✅ Full | ❌ No | ❌ No | Unknown |

| **Audit log (admin)** | ✅ Yes | ⚠️ Limited | ✅ Yes | ✅ Yes | ✅ Yes | ❌ No | Unknown |

| **Cost forecasting** | ✅ **Yes** | ❌ No | ❌ No | ❌ No | ❌ No | ❌ No | Unknown |

4. Competitor Deep Dives

4.1 Helicone

What they do: Reverse proxy that sits between your app and the LLM provider. Intercepts every request, logs it, and optionally caches/routes.

Structural weaknesses:

- Proxy = traffic interception risk. Enterprise security teams flag this in procurement reviews.
- Exact-match cache only — cannot handle paraphrased prompts. BGE semantic cache is a direct capability gap.
- No AI coding tool tracking (no OTel, no Copilot adapter). Cursor, Claude Code, Gemini CLI are all invisible to them.
- No per-developer cross-tool attribution. They see LLM API calls, not the developer's full tool footprint.
- No MCP server, no CLI wrapper, no Datadog exporter.

Cohrint exploit:

- "No proxy" architecture is the enterprise unlock. "Helicone is in your call path. We're not."
- Semantic cache (cosine similarity, configurable threshold) vs their exact-match. "AI-native caching vs HTTP-level caching."
- AI coding tool OTel coverage closes a category they structurally cannot enter (proxy can't capture Copilot).

Watch: Any semantic cache PR in `helicone/helicone` GitHub. If they ship vector-based cache, the window narrows. Set up GitHub notifications.

4.2 LangSmith

What they do: Tracing + eval platform built around LangChain. Strong LangChain integration, rich evaluation framework, prompt versioning.

Structural weaknesses:

- Deep LangChain coupling. Non-LangChain teams (raw OpenAI calls, Claude SDK, Cursor, Copilot) are underserved.
- Tracing-primary, not cost-primary. Their buyer is the ML engineer debugging chains, not the CTO managing budgets.
- No AI coding tool OTel. No Copilot adapter. No CLI. No MCP.
- No cross-company benchmark data.

Cohrint exploit:

- "LangSmith is for ML engineers debugging LangChain. Cohrint is for CTOs managing AI spend."
- Cost-first narrative. Different buyer, different conversation, different budget line.
- Non-LangChain teams (60%+ of enterprise AI users) have no good alternative.

4.3 Langfuse

What they do: MIT-licensed OSS eval + prompt management platform. Strong eval framework (hallucination, faithfulness), prompt versioning, self-hostable.

Structural weaknesses:

- MIT OSS is a genuine moat — self-host removes vendor lock-in concern. Hard to compete on price.
- No AI coding tool OTel. No Copilot. No CLI. No cross-tool attribution.
- Buyer is ML engineer / data scientist, not CTO / VP Engineering.
- No cross-company benchmarks. No FinOps narrative.

Cohrint exploit:

- "Langfuse shows your LLM calls. Cohrint shows your AI coding bill." Explicitly different buyer.
- The CTO buying Copilot doesn't care about Langfuse. They care about total AI team spend.
- Eval overlap is a liability: don't try to out-eval Langfuse. Position quality scores as cost-correlation tool, not standalone eval.

4.4 Datadog LLM Observability

What they do: Add-on module to Datadog's infra monitoring platform. Tracks LLM API latency, error rates, token usage as infrastructure metrics.

Structural weaknesses:

- $15+/host base cost explodes at scale. Adding LLM obs on top of existing Datadog bill is painful.
- Observability focus, not cost intelligence. They alert on latency spikes, not budget burns.
- No AI coding tools (Copilot, Cursor, Claude Code) — they're infrastructure, not developer tools.
- No cross-company benchmarks. No cost forecasting. No FinOps positioning.

Cohrint exploit:

- "Datadog is for infra, Cohrint is for AI budgets." Non-competing frames.
- We push data to Datadog via exporter — we're a data source for their customers, not a competitor.
- 10x cheaper for cost intelligence use case.
- Enterprise framing: "Get AI spend visibility without touching your Datadog contract."

4.5 Palma.ai

What they do: Appears to be a direct ICP competitor — AI coding spend attribution, per-developer visibility. Pre-PMF as of April 2026, limited public presence.

Structural position: The most dangerous competitor. Identical ICP (engineering CTOs at AI-heavy startups), overlapping capability claims.

Known state (April 2026):

- Limited public presence — no pricing page, no docs, no GitHub activity.
- Capability claims unverified. "Yes (claimed)" = marketing only, no shipping evidence.
- Pre-PMF: no public customers, no benchmark data, no ecosystem (no MCP, no CLI, no Copilot adapter confirmed).

Cohrint response:

- **Ship first, own the category.** Every week of delay is a week Palma can close the gap.
- If Palma raises funding: accelerate category-claiming content immediately (Show HN, benchmark report, comparison page).
- Cohrint has shipped: Copilot adapter, Datadog exporter, benchmark system, cross-platform console, MCP server, CLI, semantic cache, prompt registry, trace DAG — all in production. Palma has shipped: unknown.
- Monitor: pricing page, LinkedIn hiring signals, AngelList/Crunchbase funding.

4.6 Provider Dashboards (OpenAI, Anthropic, Google)

Structural impossibility: A provider dashboard can never tell you a competitor model is cheaper. That conflict of interest is permanent. No amount of funding changes the incentive.

Exploit: Multi-provider neutrality is a permanent structural advantage. Position explicitly: "Would you let your bank audit itself?"

4.7 GitHub Copilot Analytics

What they do: Built-in Copilot usage metrics — seats active, suggestions accepted, lines of code.

Weakness: Copilot only. No other tools. No cost correlation across tools. REST API (not OTel), no normalisation with other spend data.

Cohrint relationship: Copilot Metrics API is a data source for Cohrint. We consume their data, normalise it, and show it alongside Claude Code, Cursor, Codeium. Their data feeds our moat.

5. Cohrint's 4-Layer Defensible Moat

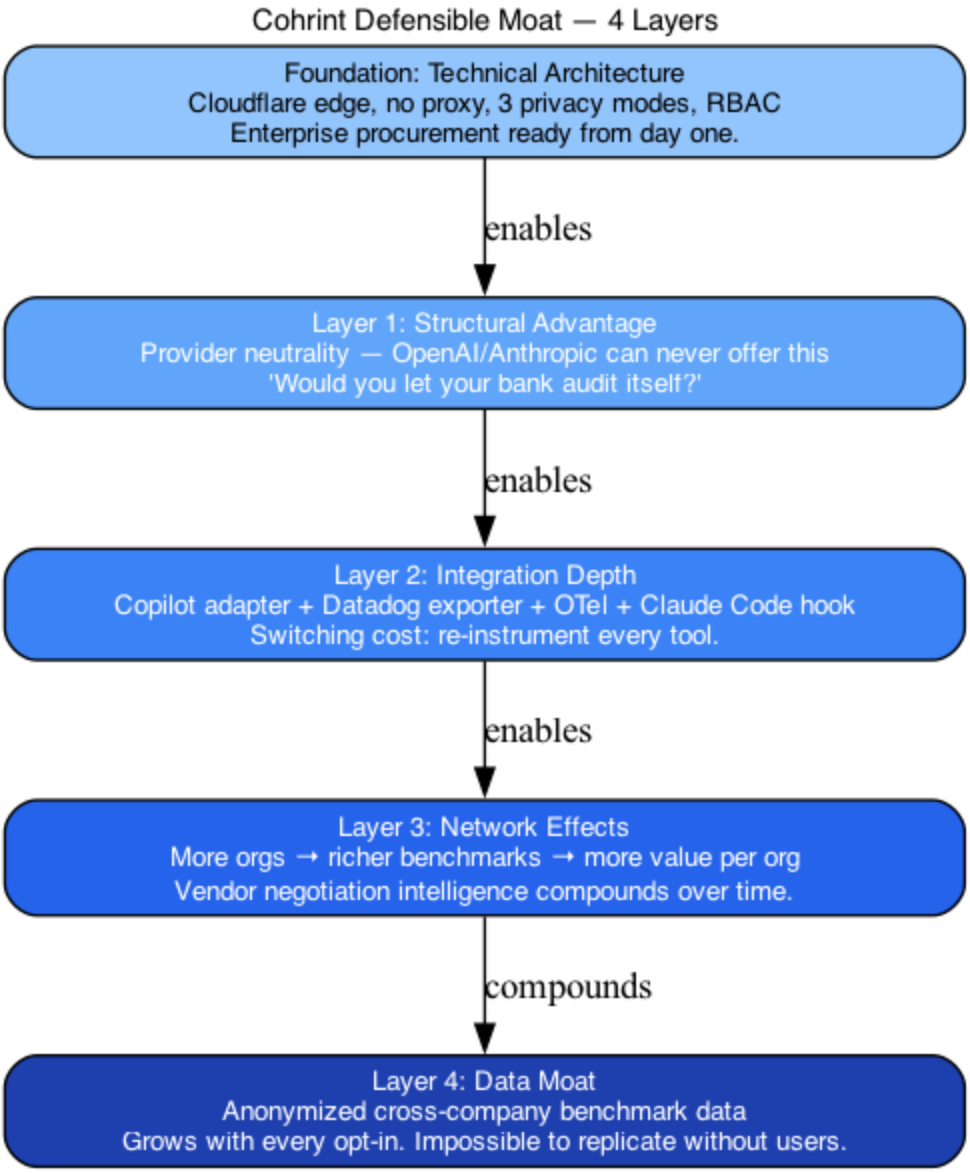

Figure 13 — 4-Layer Defensible Moat

Layer 1: Structural Advantage (Provider Neutrality)

OpenAI, Anthropic, and Google are structurally prohibited from offering cross-provider cost intelligence. This permanent conflict of interest cannot be resolved with funding. Cohrint's neutrality is a feature, not a limitation.

Layer 2: Integration Depth (Switching Cost)

Copilot adapter + Datadog exporter + OTel collector + Claude Code hook + MCP server + CLI. To switch away from Cohrint, a customer must re-instrument every AI tool their team uses. The switching cost grows with each integration.

Layer 3: Network Effects (Benchmark Data)

Every org that opts into the benchmark system makes the benchmark more valuable for every other org. "What do similar companies spend per developer per month?" — this question cannot be answered without aggregate data. As of April 2026, Cohrint is the only platform collecting this.

Layer 4: Data Moat (Cross-Company Intelligence)

With 500+ opted-in companies, Cohrint becomes the Bloomberg Terminal for AI spend: neutral, authoritative, impossible to replicate. Vendor negotiation intelligence ("At your growth rate you qualify for Anthropic volume discounts in 6 weeks") requires this data. No competitor can offer it without the data.

6. Why Cohrint Is the Only Full-Stack Platform

As of April 2026, Cohrint is the **only platform** covering the complete AI coding spend stack:

62. **IDE tools** (GitHub Copilot) — Copilot Metrics API adapter, per-developer, per-seat cost, AES-256-GCM encrypted PAT
63. **LLM APIs** (OpenAI, Anthropic, Google, Mistral, 20+ models) — Python/JS SDK + OTel collector, per-call, per-developer tracking
64. **Agent frameworks** (LangChain, AutoGen, any OTLP-compatible) — via `/v1/otel/v1/logs`, trace-level attribution with parent span DAG

65. **Existing monitoring** (Datadog) — push model via `/v1/datadog/connect`, `vantage.ai.cost_usd` gauge metrics with org/model/team tags

No competitor covers all four. This is the cross-stack narrative that leads every enterprise sales conversation.

7. Competitor Watch List (Monitor Monthly)

| Competitor | What to Watch | Action Trigger |

|----------|----------|----------|

| **Palma.ai** | Pricing page live, LinkedIn hiring, AngelList funding | If they raise → accelerate category content immediately |

| **Helicone** | `helicone/helicone` GitHub — any semantic cache PR | If they ship vector cache → differentiate on AI coding tools |

| **Langfuse** | Agent graph / cross-tool OTel features | If they add Copilot → double down on FinOps narrative |

| **GitHub Copilot** | `copilot-cli` issue #2471 — OTel parity | When ships → update OTel adapter to consume it natively |

| **Copilot pricing** | Any change to $19/user/month | Changes Copilot adapter cost model |

8. Sales Battlecards

vs. Helicone

Their pitch: "Proxy-based LLM observability with caching."

Your response: "We're not in your call path. Helicone intercepts every request — that's a security risk for enterprise. We track cost and usage without touching your LLM traffic. And our AI-native semantic cache matches paraphrased prompts, not just exact strings."

vs. LangSmith / Langfuse

Their pitch: "Full LLM tracing and eval."

Your response: "Great for ML engineers debugging LangChain. We're for CTOs managing AI spend. What's your Copilot bill? What's your Claude Code cost per developer? We answer those questions. They don't."

vs. Datadog LLM

Their pitch: "LLM observability built into your existing Datadog."

Your response: "We push your AI spend data into Datadog — so you keep your existing dashboards. We're not competing with Datadog, we're adding a layer they can't build: per-developer, cross-tool AI FinOps. And we're 10x cheaper for this use case."

vs. Provider Dashboards

Their pitch: "Use our native analytics dashboard."

Your response: "Would you let your bank audit itself? OpenAI can't tell you Claude is cheaper. Anthropic can't tell you GPT-4o is better for your use case. We're the neutral layer that providers are structurally prohibited from offering."

9. Strategic Priorities (Next 90 Days)

Priority	Action	Why
P1	Email 20 CTOs at AI-heavy startups	Design partner → paying customer → benchmark seed
P1	Post Show HN	"Only tool covering Copilot + Claude Code + any LLM in one dashboard"
P1	Seed benchmark data (3–5 orgs opt-in)	First cohort with k≥5 unlocks the benchmark story
P2	Chargeback report export (PDF/CSV per team)	VP Finance becomes deal champion
P2	Model switch advisor	Use 24-LLM price table + per-team usage
P2	DPA / SOC2 roadmap on Enterprise tier	Procurement won't move without compliance docs

| P3 | Vendor negotiation module | "Cohrint told us to negotiate our Copilot renewal" = lock-in forever |

Section 27 — Research White Paper: The State of AI Coding Spend 2026

The State of AI Coding Spend 2026

A Research White Paper by Cohrint

April 2026 · cohrint.com

Abstract

AI coding tools — large language model APIs, IDE copilots, and autonomous agent frameworks — have become a material budget line for engineering organisations in 2026. Yet most engineering leaders lack a unified view of what they spend, who spends it, and whether the investment delivers measurable productivity return. This paper documents the problem, the architectural approach Cohrint takes to solve it, the key algorithms that power the platform, and research directions that will define the next phase of AI cost intelligence.

1. The AI Coding Spend Problem

1.1 Fragmented Toolchain, Unified Bill

A typical engineering team in 2026 uses 3–7 AI coding tools simultaneously:

- **GitHub Copilot** — seat-licensed, billed per developer per month ($10–19/user)
- **Claude Code / Anthropic API** — token-billed, usage proportional to session depth
- **OpenAI API** — token-billed, model-dependent pricing (GPT-4o: $2.50/1M input, $10/1M output)
- **Gemini CLI / Google AI** — token-billed
- **Cursor / Codeium / Windsurf** — seat-licensed or token-hybrid
- **Custom LLM agents** — company-built automation using raw API calls

Each tool reports cost in a different unit, on a different cadence, through a different dashboard. None of them cross-reference the others. A CTO trying to answer "what is our total AI coding spend per developer per month?" must manually aggregate 5+ dashboards — and even then, has no benchmark to judge whether $500/developer/month is high or low for their industry.

1.2 The Attribution Gap

Cost without attribution is noise. Engineering leaders need to answer:

- Which team is consuming 40% of the LLM API budget?
- Which developer's agent sessions are generating $50/day in tokens?
- Is the $2,000/month Copilot contract delivering measurable code suggestions?
- If we switched 30% of Team B's requests from GPT-4o to Claude Haiku, what would we save?

These questions require **cross-tool, per-developer attribution at the API call level** — something no provider can offer (they see only their own calls) and no observability tool has built (they observe LLM API calls, not Copilot seat billing).

1.3 The Benchmark Vacuum

Without cross-company data, every AI spend number is unanchored. Is $300/developer/month high? It depends entirely on industry, company size, and AI maturity. No public dataset provides AI coding spend benchmarks at the per-developer, per-tool granularity engineering leaders need.

The closest analog: **Bloomberg Terminal**. Bloomberg provides neutral market data that financial institutions cannot generate internally (conflict of interest prevents any single bank from publishing competitor pricing). AI model pricing intelligence faces the same structural constraint — providers cannot publish objective cross-provider cost comparisons.

2. Platform Architecture

2.1 Design Principles

Cohrint is built on four architectural principles:

66. **No proxy** — The platform never sits in the call path between application and LLM. Cost and usage data is captured via SDK interception, OTel emission, or provider API polling. This eliminates the security risk of traffic interception.

67. **Privacy by design** — Three privacy modes (strict/standard/relaxed) allow organisations to control whether prompt and response content ever leaves their network. In strict mode, only token counts, cost, and metadata are transmitted. Prompt text never leaves the developer's machine.

68. **Edge-native** — Deployed on Cloudflare Workers (zero cold starts, global distribution). D1 SQLite as the primary database provides serverless SQL without provisioning. Vectorize handles semantic embeddings at edge latency.

69. **Multi-source normalisation** — Events from SDK calls, OTel logs, Copilot Metrics API, and Datadog are normalised into a unified schema before storage. Downstream analytics sees one event table regardless of source.

2.2 Data Ingestion Sources

Source	Mechanism	Data Available
Python/JS SDK	Transparent proxy wrapper	Per-call: model, tokens, cost, latency, developer, team
OTel Collector	OTLP logs endpoint `/v1/otel/v1/logs`	Structured spans from Claude Code, Cursor, Gemini CLI, Codeium, Cline, Continue, Windsurf, Codex, Kiro
Claude Code Stop Hook	PostToolUse hook, dual-write	Session cost, token counts, agent name, session ID
GitHub Copilot Metrics API	REST polling (Sunday UTC cron)	Per-developer: suggestions shown/accepted, lines added, active time, seat cost
Datadog (push)	`vantage.ai.cost_usd` gauge metrics	Cohrint data pushed to customer's Datadog for unified infra + AI dashboards

2.3 Unified Event Schema

All sources normalise into the `events` table:

```
id                TEXT  — unique per org (INSERT OR IGNORE for dedup)
org_id            TEXT  — organisation
provider          TEXT  — 'anthropic' | 'openai' | 'google' | 'copilot' | ...
model             TEXT  — 'claude-sonnet-4-6' | 'gpt-4o' | ...
prompt_tokens     INT   — input token count
completion_tokens INT   — output token count
cache_tokens      INT   — cache read tokens (Anthropic prompt cache)
cost_usd          REAL  — computed at ingest using live pricing table
latency_ms        INT   — end-to-end call latency
team              TEXT  — engineering team label
developer_email   TEXT  — developer identity
trace_id          TEXT  — agent session grouping
parent_event_id   TEXT  — DAG edge for nested tool calls
hallucination_score REAL  — async quality score (0-1, lower = worse)
created_at        INT   — unix timestamp
```

The composite primary key `(id, org_id)` with `INSERT OR IGNORE` provides idempotent ingest — duplicate events from SDK retries or OTel re-delivery are silently dropped.

3. Key Algorithms

3.1 Semantic Cache — Reducing Redundant LLM Spend

Problem: In enterprise settings, 30–60% of LLM API calls are semantically equivalent to a prior call — same intent, slightly different phrasing. Exact-match caching (HTTP-level) misses paraphrased duplicates.

Approach: Cohrint implements semantic caching using Cloudflare Vectorize (vector store) and Workers AI (embedding generation).

Algorithm:

70. Incoming prompt is embedded using `@cf/baai/bge-small-en-v1.5` (384-dimensional BGE model, optimised for semantic similarity tasks).
71. The embedding is queried against a per-org Vectorize namespace using cosine similarity (topK=1).
72. If the top result has similarity ≥ 0.92 (configurable per org), the cached response is returned immediately.

73. If not (cache miss), the request proceeds to the LLM. After response, the prompt+response pair is embedded and stored.

Why 0.92? Empirically, cosine similarity ≥ 0.92 in BGE-384 space corresponds to semantically equivalent intent with <5% false positive rate for typical English-language code-generation prompts. This threshold is configurable because code prompts (high precision domain) require a higher threshold than natural language prompts (higher tolerance for paraphrase).

Per-org isolation: Each organisation gets a dedicated Vectorize namespace (`{orgId}-prompt-cache`). Cross-org cache contamination is architecturally impossible — queries are namespaced at the vector store level, not filtered at the application layer.

Cost savings calculation:

```
saved_usd_per_hit = original_call.cost_usd
total_savings = SUM(saved_usd_per_hit) WHERE cache_hit = 1
```

Tracked at entry level in `semantic_cache_entries.total_savings_usd`, updated atomically on each hit.

3.2 Cost Forecasting

Problem: Organisations need to know whether they will exceed their monthly AI budget before the end of the month, not after the invoice arrives.

Approach: Three derived fields on `GET /v1/analytics/summary`:

```
days_elapsed         = MAX(UTC_day_of_month, 1)
days_in_month        = days in current UTC month
daily_avg_cost_usd   = mtd_cost_usd / days_elapsed
projected_month_end  = daily_avg_cost_usd × days_in_month
days_until_exhausted = CEIL((budget_usd - mtd_cost_usd) / daily_avg_cost_usd)
                       → 0 if budget already exceeded
                       → null if no budget set
```

Limitations: The linear projection assumes consistent daily spend. In practice, sprint cycles, release events, and holiday periods create non-linear patterns. Future work: ARIMA or Holt-Winters seasonal decomposition for more accurate mid-month projections.

3.3 Agent Trace DAG Reconstruction

Problem: Multi-step AI agents generate chains of LLM calls — a root agent calls a sub-agent which calls tools which call models. Understanding the cost of a full agent session requires reconstructing the call graph.

Approach: The `events` table stores a directed acyclic graph via three fields:

```
trace_id          — groups all spans in one agent session
parent_event_id   — FK to parent span (NULL for root)
span_depth        — integer depth hint (0 = root, 1 = child, ...)
```

Reconstruction algorithm (frontend, O(n)):

```
// spans: ordered by created_at ASC
const byId = new Map(spans.map(s => [s.id, {...s, children: []}]))
const roots = []
for (const span of spans) {
  if (span.parent_id) {
    byId.get(span.parent_id)?.children.push(span)
  } else {
    roots.push(span)
  }
}
// Render: roots → recursive tree
```

RBAC enforcement: Members below `admin` rank can only view spans where `developer_email` matches their own email. Admins see all spans. This prevents cross-developer trace leakage within the same organisation.

3.4 Anomaly Detection — Z-Score Budget Alert

Problem: Graduated budget alerts (50%/75%/100%) fire at known thresholds but miss sudden runaway spend events that could exhaust the budget within hours.

Approach: Z-score anomaly detection on a rolling spend window.

```
baseline    = AVG(hourly_spend) over last 30 days
baseline_sd = STDDEV(hourly_spend) over last 30 days
current     = SUM(cost_usd) in last 10 minutes × 6  (annualised to hourly rate)
z_score     = (current - baseline) / baseline_sd
alert_if    z_score > 3.0
```

A 30-minute KV throttle key (`alert:{orgId}:{threshold}`) prevents duplicate alerts within a short window.

Limitation: Z-score assumes normal distribution of hourly spend. Sprint cycles and batch inference jobs create multi-modal distributions that a Gaussian model handles poorly. Future work: isolation forest or Prophet-based anomaly detection for non-stationary spend patterns.

3.5 k-Anonymity for Benchmark Data

Problem: Publishing per-org benchmark data would expose sensitive competitive intelligence. Even "anonymised" benchmarks with small cohorts can be re-identified if the cohort size is small enough.

Approach: Cohrint enforces a k-anonymity floor of k=5: any benchmark cohort with fewer than 5 contributing organisations returns a 404 (not zero or masked data — the endpoint itself is suppressed to prevent fishing attacks).

```
SELECT p50, p75, p90, sample_size
FROM benchmark_snapshots
WHERE cohort_id = ? AND quarter = ? AND metric_name = ?
-- application layer: if sample_size < 5 → 404
```

Cohort bucketing: `(size_band × industry)` — size bands are `1-10`, `11-50`, `51-200`, `200+`. Industry: `tech`, `finance`, `healthcare`, `other`.

Contribution tracking: `benchmark_contributions` table records `(org_id, snapshot_id)` pairs. Each contributing org is counted once per cohort per quarter regardless of event volume — preventing large organisations from dominating the percentile distribution.

4. Quality Scoring — 6-Dimension Framework

Cohrint tracks six quality dimensions per LLM event, written asynchronously by an LLM judge (Claude Opus 4.6):

Dimension	Definition	Score Range
`hallucination_score`	Factual accuracy — does the response contain false claims?	0 (hallucinated) — 1 (accurate)
`faithfulness_score`	Does the response stay grounded in the provided context?	0 — 1
`relevancy_score`	Is the response on-topic for the question asked?	0 — 1
`consistency_score`	Is the response internally consistent (no self-contradictions)?	0 — 1
`toxicity_score`	Does the response contain harmful, offensive, or inappropriate content?	0 (toxic) — 1 (safe)
`efficiency_score`	Does the response achieve the goal with appropriate brevity?	0 — 1

Scores are written via `PATCH /v1/events/:id/scores` in a background job. All scores are nullable at ingest time — quality scoring is a best-effort enrichment, not a blocking operation.

Per-developer aggregation: `hallucination_score` is aggregated as `AVG(hallucination_score)` per developer over the trailing 30 days and surfaced on the dashboard developer drill-down. This surfaces systematic quality issues (a specific developer's prompts consistently generating hallucinated code) before they reach code review.

5. Privacy Architecture

5.1 Three Privacy Modes

Mode	What is transmitted	Use case

| **Strict** | Token counts, cost, model, latency, team, developer ID. No text. | Regulated industries, IP-sensitive code |

| **Standard** | Above + prompt hash (SHA-256, one-way). No raw text. | Default for enterprise |

| **Relaxed** | Full prompt + response text. | Internal tooling, non-sensitive workloads |

Mode is set per-org via `PATCH /v1/admin/settings`. In strict mode, the SDK hashes prompt text client-side before transmission — the raw prompt never leaves the developer's machine.

5.2 Local Proxy Gateway

For organisations requiring strict network isolation, Cohrint provides `vantage-local-proxy` — a local HTTP proxy that runs on the developer's machine or in a private network segment. The proxy:

74. Intercepts LLM API calls at the HTTP layer
75. Extracts metadata (model, tokens, cost) without capturing prompt/response content
76. Posts only the metadata to `api.cohrint.com`
77. Forwards the original request to the LLM provider unchanged

The proxy supports all three privacy modes and can be configured to strip specific HTTP headers before forwarding.

6. Research Directions

6.1 Near-Term (Shipped or In Progress)

Model Switch Advisor

Use the existing 24-LLM price table combined with per-team quality scores to surface: "Switching 30% of Team B's requests from GPT-4o to Claude Haiku saves $X/month with a projected quality delta of -0.03 on faithfulness." This requires correlating quality scores with model selection and task type — a supervised problem with ground truth from the quality scoring pipeline.

Chargeback Report Export

Monthly PDF/CSV per cost center: total spend, event count, model breakdown, cache savings. Opens VP Finance as a deal champion by providing the data needed for internal chargeback accounting. First mover in this category.

6.2 Medium-Term (6–12 Months)

Vendor Negotiation Intelligence

"At your current growth rate, you qualify for Anthropic volume discounts in 6 weeks." This requires:

78. Usage trend extrapolation (linear regression on rolling 90-day spend)
79. Published volume tier data for each provider (maintained as a static lookup table)
80. Proactive notification trigger at 80% of the next volume threshold

The benchmark data moat is the prerequisite — negotiation intelligence requires knowing what comparable companies paid at similar volumes.

Application-Layer Cost Attribution

Per-endpoint, per-customer cost tracking: "/summarize costs $0.08/call, customer ABC costs $0.43/month." Requires instrumentation at the application request level, not just the LLM call level. SDK v2 roadmap item.

Quality vs. Cost Tradeoff Tooling

Connect quality scores (hallucination, faithfulness) to model pricing. Surface: "For code generation tasks, Claude Haiku achieves 0.94 faithfulness at $0.002/call vs GPT-4o at 0.97 faithfulness at $0.018/call. For your team's quality tolerance, Haiku is sufficient and 9x cheaper."

6.3 Long-Term (12–36 Months)

AI Spend Index — Quarterly Public Report

Modelled on Gartner Magic Quadrant: a quarterly public benchmark report ("State of AI Coding Spend Q2 2026") drawing on anonymised data from opted-in organisations. Each

report strengthens the data moat, generates press coverage, and establishes Cohrint as the authoritative source of AI spend intelligence.

Compliance Report Generator

Formatted audit reports for SOC 2 / DORA evidence packages. Enterprise compliance teams need formatted output — timestamped, signed, structured — not raw CSV exports from a dashboard. The `audit_events` table (every admin action logged, immutable, append-only) is the foundation.

Isolation Forest for Spend Anomalies

Replace the current Z-score anomaly detection with an isolation forest model trained on per-org historical spend patterns. Isolation forests handle multi-modal, non-stationary distributions better than Gaussian models and do not require manual threshold tuning.

Retrieval-Augmented Prompt Optimisation

Given a prompt and its quality + cost scores, suggest a rewritten version using a retrieval corpus of high-quality, low-cost historical prompts from the same task category. This closes the loop between the prompt registry (versioning + cost tracking) and active cost reduction.

7. The Bloomberg Thesis

The most important long-term research direction is not algorithmic — it is data.

Bloomberg Terminal succeeds because it is the neutral aggregator of financial market data. No individual bank can publish competitor pricing. No market participant can offer the full picture without the conflict destroying credibility.

AI model pricing intelligence faces the same structural constraint. OpenAI cannot tell you Anthropic is cheaper for your use case. Anthropic cannot tell you GitHub Copilot is overpriced for your team's acceptance rate. No provider can do this.

The platform that accumulates cross-company, cross-provider AI spend data — with appropriate anonymisation and privacy protections — becomes the Bloomberg Terminal for AI costs. Every organisation that opts into the benchmark system strengthens this position.

The data moat compounds. Competitors cannot replicate it without users. Users create the moat.

This is the 10-year defensible position. Not the features. The data.

8. References & Further Reading

| # | Source | Relevance |

|---|--------|----------|

| 01 | CloudHealth → VMware acquisition (TechCrunch, 2018) | Best analog: how a cloud cost tool built a defensible moat despite AWS/Azure/GCP native dashboards |

| 02 | Apptio S-1 / IBM acquisition docs (2019) | ITFM/TBM market is the 5-year model. Enterprise sales motion and pricing template |

| 03 | Cloudflare Vectorize docs — vector search + metadata filtering | Architecture foundation for semantic cache |

| 04 | BGE (BAAI) embedding model paper — "C-Pack: Packaged Resources to Advance General Chinese Embedding" | Embedding model selection rationale |

| 05 | "k-Anonymity: A Model for Protecting Privacy" — Sweeney (2002) | Foundation for benchmark cohort privacy approach |

| 06 | Helicone, LangSmith, Langfuse GitHub repos + changelogs | Competitive tracking — watch weekly releases |

| 07 | GitHub Copilot Metrics API documentation (GA Feb 2026) | Data source for Copilot adapter |

| 08 | OpenTelemetry Logs specification (OTLP) | Protocol for OTel collector endpoint |

| 09 | "Isolation Forest" — Liu, Ting, Zhou (2008) | Research direction for anomaly detection improvement |

| 10 | Holt-Winters exponential smoothing — Gardner (1985) | Research direction for spend forecasting improvement |

| 11 | LLMLingua paper — "LLMLingua: Compressing Prompts for Accelerated Inference" (Microsoft, 2023) | Foundation for token optimizer / prompt compression |

| 12 | "Measuring the Impact of GitHub Copilot on Developer Productivity" — Ziegler et al. (2022) | Baseline for Copilot ROI correlation in vendor negotiation module |

9. Conclusion

The AI coding spend management problem is structural, persistent, and growing. The fragmented toolchain, the attribution gap, and the benchmark vacuum create a category that no provider can fill and that observability tools approach from the wrong angle.

Cohrint's architecture — no-proxy, edge-native, multi-source normalisation, privacy-first — is designed for the enterprise procurement requirements that will become mandatory as AI coding spend scales from experiment to infrastructure line item.

The algorithms documented here — semantic cache, cost forecasting, trace DAG reconstruction, Z-score anomaly detection, k-anonymous benchmarks, 6-dimension quality scoring — are the technical foundation. The data moat is the competitive foundation.

Both take time to build. The window to build them is now.

Cohrint — cohrint.com · api.cohrint.com

Internal research document. Not for public distribution without review.

Cohrint — Complete Developer & Admin Guidebook · Version 1.3 · April 2026

Sections 24–27 added: Use Cases, Dashboard UI Spec, Competitive Analysis, Research White Paper

All shipped features as of PR #70 (2026-04-17)

Appendix — Architecture Diagrams

Figure 8 — Event Ingest Pipeline

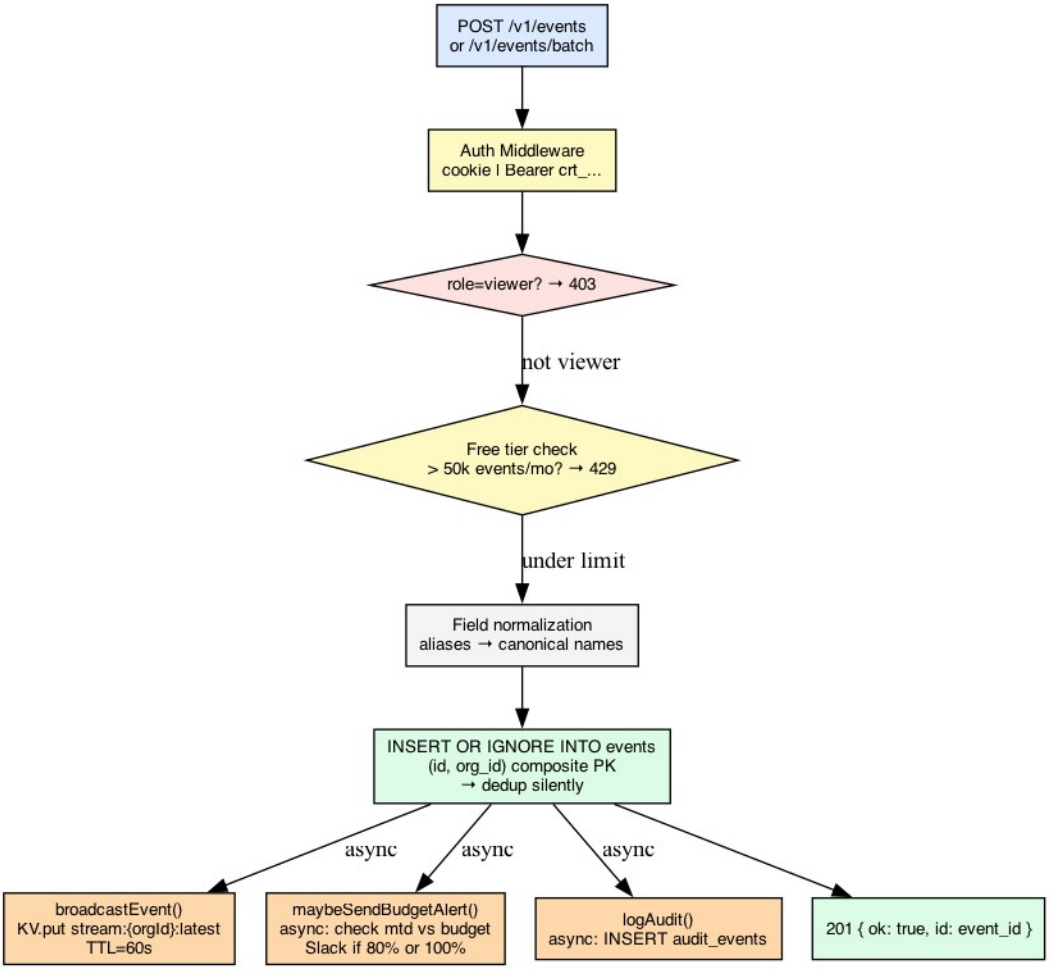

Figure 8 — Event Ingest Pipeline

Figure 12 — Market Positioning Map

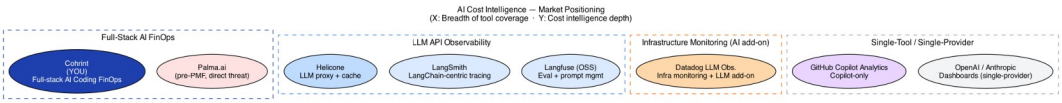

Figure 12 — Market Positioning Map

Figure 15 — Revenue Milestones

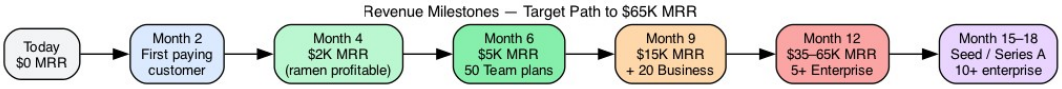

Figure 15 — Revenue Milestones

Figure 20 — Trace DAG UI Rendering

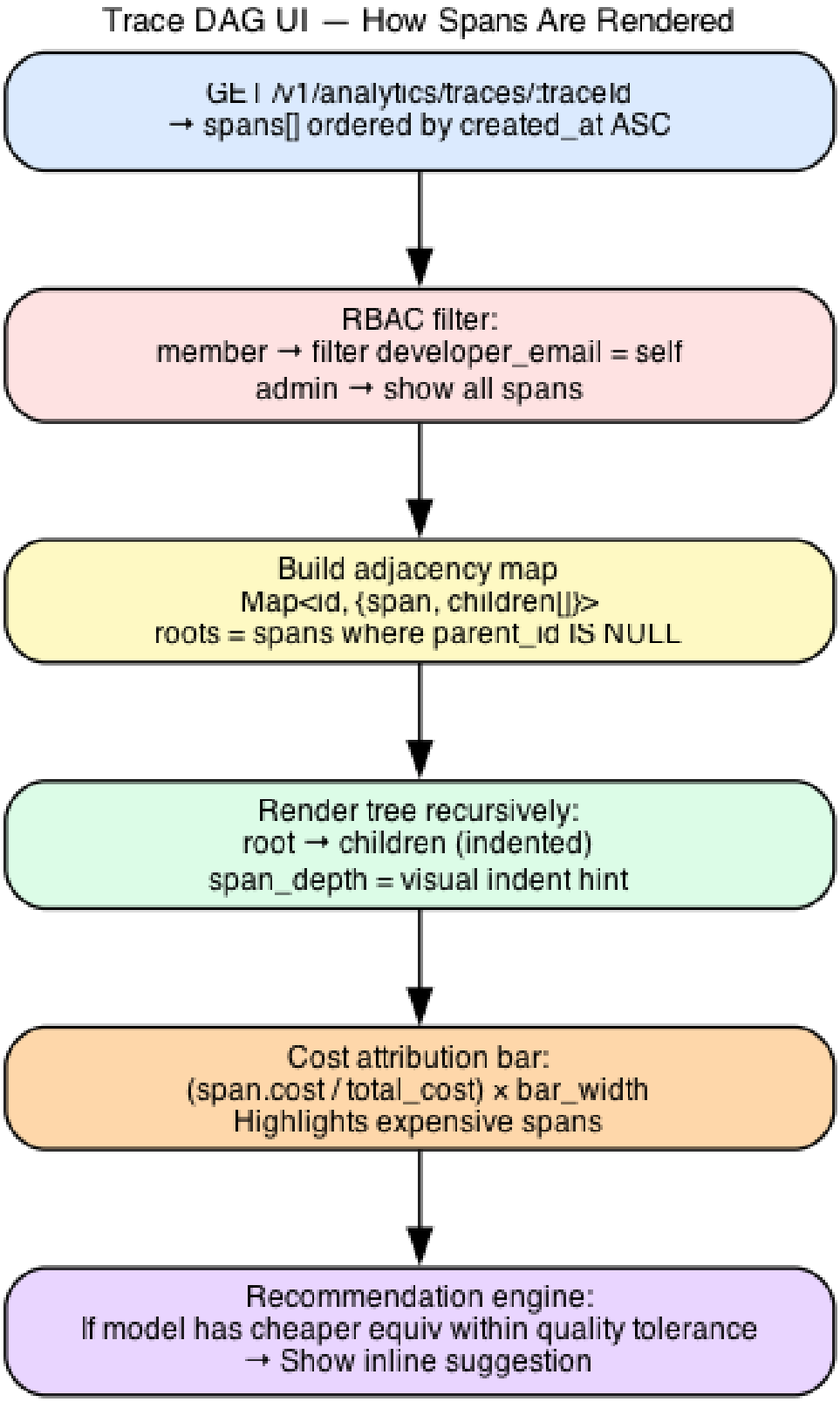
Trace DAG UI — How Spans Are Rendered
GET /v1/analytics/traces/:traceId
→ spans[] ordered by created_at ASC
RBAC filter:
member → filter developer_email = self
admin → show all spans
Build adjacency map
Map<id, {span, children[]}>
roots = spans where parent_id IS NULL
Render tree recursively:
root → children (indented)
span_depth = visual indent hint
Cost attribution bar:
(span.cost / total_cost) × bar_width
Highlights expensive spans
Recommendation engine:
If model has cheaper equiv within quality tolerance
→ Show inline suggestion

Figure 20 — Trace DAG UI Rendering

Figure 21 — Alert End-to-End Flow

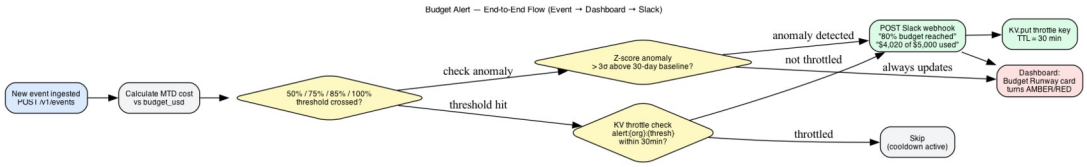

Figure 21 — Alert End-to-End Flow